

คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพ
พรรณี

คู่มือนวัตกรรมเกษตรดิจิทัลรุ่น เยาว์

การบูรณาการปัญญาประดิษฐ์และอินเทอร์เน็ตของ
สรรพสิ่งในแปลงเกษตรแม่นยำ

ผู้เขียน/วิทยากรบรรยายหลัก:

รศ.ดร. ชีวะ ทักสนา และ ผศ.อรรถกร คำฉัตร

โครงการศูนย์พัฒนานวัตกรรมการเกษตรดิจิทัลรำไพพรรณี ประณีต
วิทยาคม

สนับสนุนโดย ยุทธศาสตร์การพัฒนากิจการเกษตรแม่นยำอัจฉริยะ
พ.ศ. 2569

Contents

1	โครงการศูนย์พัฒนานวัตกรรมการเกษตรดิจิทัลไร่โพพรรณี ประณีต วิทยาคม	14
1.1	คู่มือนวัตกรรมเกษตรดิจิทัลรุ่นเยาว์: การบูรณาการปัญญา ประดิษฐ์และอินเทอร์เน็ตของสรรพสิ่งในแปลงเกษตรแม่นยำ	14
1.1.1	คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏ ไร่โพพรรณี	14
	สารบัญรายละเอียดวิชาเรียนอบรม 3 วัน	15
1.2	โครงการศูนย์พัฒนานวัตกรรมการเกษตรดิจิทัลไร่โพพรรณี ประณีต วิทยาคม	15
	หน้าปก คำนำ และการวิเคราะห์วัตถุประสงค์เชิงเป้าหมายการ เรียนรู้ (Preface & Educational Objectives Matrix)	17
2	คู่มือนวัตกรรมเกษตรดิจิทัลรุ่นเยาว์:	18
3	การบูรณาการปัญญาประดิษฐ์และอินเทอร์เน็ตของสรรพสิ่งใน แปลงเกษตรแม่นยำ	19
3.1	คำนำ (Preface)	19
3.2	การวิเคราะห์วัตถุประสงค์เชิงเป้าหมายการเรียนรู้ (Edu- cational Objectives Matrix)	21
3.3	โครงการปฏิบัติการ: การสร้าง "กล่องนวัตกรรมฟาร์ม" (Smart Farm Innovator Box)	23
3.3.1.1	หลักการและเหตุผลเชิงวิชาการ (Academic Ra- tionale)	23
3.3.2.2	องค์ประกอบทางฟิสิกส์และเทคโนโลยีที่ใช้ (Phys- ical and Technological Components)	23

3.4	รายการสัญลักษณ์และตัวย่อทางวิชาการ (List of Symbols and Abbreviations)	25
4	บทที่ 1: ฟิสิกส์อุตุนิยมวิทยาเกษตรและการประมวลผลข้อมูล อนุกรมเวลา (Day 1)	27
4.1	1.1 ดุลภูมิอากาศและสรีรวิทยาของทุเรียนตะวันออก (Agro- Climatology and Eastern Durian Physiology)	27
5	ตำราวิชาการ: ฟิสิกส์เกษตรและระบบนิเวศดิจิทัล	28
5.1	บทที่ 1.1 ดุลภูมิอากาศและสรีรวิทยาของทุเรียนตะวันออก (Agro-Climatology and Eastern Durian Physiology) .	28
5.1.1	บทนำ: ความสำคัญของการทำความเข้าใจปฏิสัมพันธ์ ระหว่างสิ่งแวดล้อมกับสรีรวิทยาพืช	28
5.1.21	การวิเคราะห์กายภาพและสรีรวิทยาการเจริญเติบโต ของทุเรียนในระยะวิกฤต	29
5.1.32	แนวคิดทางฟิสิกส์เกษตร: ดัชนีความตึงเครียด จากน้ำและอุณหภูมิเรือนยอด	30
5.1.43	การวิเคราะห์สมการดุลพลังงานของเรือนยอดพืช (Canopy Energy Balance Equation)	31
5.1.54	ความสัมพันธ์ระหว่างอุณหภูมิ ความชื้น และความ ดันไอน้ำ (Vapor Pressure Deficit: VPD)	34
5.1.65	ตัวอย่างการคำนวณและการจำลองทางคอมพิวเตอร์ (Computational Modeling Example)	35
5.2	1.2 พื้นฐาน Python Sandbox สำหรับบัณฑิตเกษตร (Introduction to Python Programming & Variables)	36
6	ภาควิชาวิศวกรรมศาสตร์และเทคโนโลยีเกษตรอัจฉริยะ	37
7	สาขาวิชาปัญญาประดิษฐ์และการเกษตรดิจิทัล	38
8	ระดับปริญญาตรี/บัณฑิตศึกษา	39
8.1	บทที่ 1.2 พื้นฐานภาษาไพธอน (Python) สำหรับบัณฑิต เกษตร: การจัดการข้อมูลและตรรกะการเขียนโปรแกรม .	39

8.1.1 คำนำเชิงวิชาการ (Academic Preface)	39
8.1.21.2.1 สภาพแวดล้อมการเขียนโปรแกรมไพธอน (Python Environment and Sandbox)	40
8.1.31.2.2 ชนิดข้อมูลพื้นฐานและโครงสร้างข้อมูล (Data Types and Data Structures)	41
8.1.41.2.3 โครงสร้างควบคุมทิศทางโปรแกรม (Control Flow Structures)	43
8.1.51.2.4 การประยุกต์ใช้: การจำลองระบบตรวจจับสถานะ วิกฤตความชื้นดิน (Soil Moisture Crisis Detection Simulation)	45
8.2 1.3 การดึงข้อมูลสภาพอากาศ TMD Open Data API และ โครงสร้าง JSON	46
9 1.3 การดึงข้อมูลสภาพอากาศจาก TMD Open Data API และโครงสร้าง JSON	47
9.1 1.3.1 แนวคิดพื้นฐานของคลังข้อมูลเปิด (Open Data Repository) และ Web API	47
9.1.11.3.1.1 คลังข้อมูลเปิด (Open Data Repository)	48
9.1.21.3.1.2 สถาปัตยกรรม Web API (Web Application Programming Interface)	48
9.2 1.3.2 กลไกการร้องขอข้อมูลผ่าน API (API Request Mechanism)	49
9.2.11.3.2.1 องค์ประกอบของการร้องขอ (Request Components)	49
9.2.21.3.2.2 การเปรียบเทียบ HTTP GET และ HTTP POST	49
9.2.31.3.2.3 การใช้ API Key (Authentication)	50
9.3 1.3.3 โครงสร้างข้อมูล JSON (JavaScript Object Notation)	51
9.3.11.3.3.1 โครงสร้างพื้นฐานของ JSON	51
9.3.21.3.3.2 การวิเคราะห์พารามิเตอร์สภาพอากาศใน JSON	52
9.3.31.3.3.3 กระบวนการแปลงค่า (JSON Parsing)	52

9.4 1.3.4 การประยุกต์ใช้เชิงปฏิบัติ: การดึงข้อมูลสภาพอากาศด้วย Python	53
9.4.1 1.3.4.1 โค้ด Python สำหรับการดึงและวิเคราะห์ข้อมูล	53
9.5 1.4 การวิเคราะห์อนุกรมเวลาสภาพอากาศด้วย Pandas (Weather Time Series Analysis with Pandas & Seaborn)	56
10 ภาควิชาวิทยาศาสตร์การเกษตรดิจิทัลและฟิสิกส์ประยุกต์	57
11 หัวข้อ 1.4 การวิเคราะห์อนุกรมเวลาสภาพอากาศด้วย Pandas (Weather Time Series Analysis with Pandas & Seaborn)	58
11.1 บทนำ: ความสำคัญของการวิเคราะห์อนุกรมเวลาในอุตุนิยมวิทยาการเกษตร	58
11.2 องค์ประกอบทางทฤษฎี: การจัดการข้อมูลอนุกรมเวลาด้วย Pandas	59
11.2.1.1 การโหลดและการจัดการโครงสร้างข้อมูล (Data Loading and Structure Management)	60
11.2.2.2 การจัดการข้อมูลสูญหาย (Handling Missing Values: NaN)	60
11.3 การวิเคราะห์แนวโน้มด้วยค่าเฉลี่ยเคลื่อนที่ (Rolling Moving Average)	61
11.3.2.1 หลักการทางคณิตศาสตร์ (Mathematical Principle)	61
11.4 การปฏิบัติการ: การวิเคราะห์อนุกรมเวลาด้วย Python	62
11.4.1.1 การเตรียมสภาพแวดล้อมและข้อมูลจำลอง	62
11.4.2.2 โค้ด Python สำหรับการวิเคราะห์ (Python Code for Analysis)	62
11.5 การวิเคราะห์เชิงลึกและขอสรุปทางวิชาการ	65
11.6.5 แบบจำลอง Drought Risk Index (DRI) Dashboard และระบบสนับสนุนการตัดสินใจ (Decision Support System)	65

12บทที่ 1.5 แบบจำลอง Drought Risk Index (DRI) Dashboard และระบบสนับสนุนการตัดสินใจ (Decision Support System)	66
12.1บทนำเชิงทฤษฎี: ระบบสนับสนุนการตัดสินใจในการเกษตร (Farm Decision Support System: DSS)	66
12.1.1.5.1 องค์ประกอบหลักของ DSS ในการเกษตร	67
12.2.1.5.2 การพัฒนาแบบจำลอง Drought Risk Index (DRI)	68
12.2.1.5.2.1 การนิยามตัวแปรทางคณิตศาสตร์	68
12.2.2.5.2.2 การสร้างแบบจำลองเชิงคณิตศาสตร์ (Mathematical Model Derivation)	69
12.3.1.5.3 การประยุกต์ใช้เชิงปฏิบัติ: การสร้าง Dashboard และระบบสนับสนุนการตัดสินใจด้วย Python	71
12.3.1.5.3.1 โค้ด Python สำหรับการคำนวณ DRI และ DSS	71
12.4.1.5.4 การวิเคราะห์เปรียบเทียบเชิงวิพากษ์: TMD Open API กับระบบตรวจวัดในฟาร์ม (In-situ Sensors)	74
12.5.1.5.5 คำศัพท์สำคัญประจำบทเรียน (Key Terms)	74
12.6.1.5.6 แบบฝึกหัดทบทวนและโจทย์คิดวิเคราะห์ (Review Exercises)	74
13บทที่ 2: คอมพิวเตอร์วิทัศน์เชิงแสงและโครงข่ายประสาทเทียม ลึกลับวิเคราะห์โรคพืช (Day 2)	77
13.1.2.1 ดัชนีสเปกตรัมพืชพรรณ (NDVI, EVI) และหลักการของคอมพิวเตอร์วิทัศน์ (Spectral Vegetation Indices & Computer Vision)	77
14.2.1 ดัชนีสเปกตรัมพืชพรรณ (Spectral Vegetation Indices) และหลักการของคอมพิวเตอร์วิทัศน์ (Computer Vision)	78
14.1.2.1.1 หลักการทางฟิสิกส์ของการแผ่รังสีและการปฏิสัมพันธ์กับพืชพรรณ	79
14.1.1.1 การดูดกลืนและการสะท้อนของคลื่นแม่เหล็กไฟฟ้า (Absorption and Reflection)	79
14.2.2.1.2 ดัชนีพืชพรรณผลต่างปกติ (Normalized Difference Vegetation Index: NDVI)	80

14.2.1. สมการทางคณิตศาสตร์ (Mathematical Equation)	81
14.2.2. การตีความเชิงลึก (In-Depth Interpretation) . .	81
14.3.1.3 ดัชนีสเปกตรัมขั้นสูง (Advanced Spectral Indices)	82
14.3.1. ดัชนีพืชพรรณที่ปรับปรุงแล้ว (Enhanced Vegetation Index: EVI)	82
14.3.2. ดัชนีสเปกตรัมขอบเขตสีแดง (Normalized Difference Red Edge: NDRE)	83
14.4.1.4 การประยุกต์คอมพิวเตอร์วิทัศน์ในการเกษตร (Computer Vision in Agriculture)	84
14.4.1. หลักการทำงานของ Computer Vision ในการเกษตร	84
14.4.2. ขั้นตอนการวิเคราะห์โรคและสุขภาพพืชด้วย Computer Vision	85
14.5.1.5	86
14.6.2 โครงสร้างคณิตศาสตร์ของ CNN (Convolutional Neural Networks: Convolution, ReLU, Max Pooling, Softmax)	86
15.2.2 โครงสร้างคณิตศาสตร์ของโครงข่ายประสาทเทียมแบบคอนโวลูชัน (Mathematical Structure of Convolutional Neural Networks: CNN)	87
15.2.2.1 ภาพรวมสถาปัตยกรรม CNN	88
15.2.2.2 ชั้นคอนโวลูชัน (Convolutional Layer)	88
15.2.2.2.1 หลักการทางคณิตศาสตร์ของกัวรังวัตนาการ	89
15.2.2.3 ฟังก์ชันกระตุ้นค่าเชิงเส้นแบบปรับแก้ (Rectified Linear Unit: ReLU)	90
15.2.2.4 ชั้นลดขนาดพื้นที่สูงสุด (Max Pooling Layer)	91
15.2.2.5 ชั้นเชื่อมต่อสมบูรณ์และฟังก์ชันซอฟต์แวร์แมกซ์ (Fully Connected Layer and Softmax Function)	92
15.2.2.5.1 การคำนวณในชั้นเชื่อมต่อสมบูรณ์	92
15.2.2.5.2 ฟังก์ชันซอฟต์แวร์แมกซ์ (Softmax Function)	93
15.2.2.6 การประยุกต์ใช้ในโค้ด (Python Implementation Example)	94

15.2.3 ปฏิบัติการสะสมชุดภาพถ่าย การขยายข้อมูลภาพ การฝึกโมเดลบน Google Colab และการส่งออก TFLite	96
16.2.3 ปฏิบัติการสะสมชุดภาพถ่าย การขยายข้อมูลภาพ การฝึกโมเดลบน Google Colab และการส่งออก TFLite	97
16.1 บทนำเชิงทฤษฎี (Theoretical Introduction)	97
16.2.3.1 กระบวนการเตรียมและรวบรวมชุดข้อมูลภาพถ่ายใบพืช (Dataset Collection Protocol)	98
16.2.2.3.1.1 หลักการเก็บข้อมูลภาคสนาม (Field Data Acquisition Principles)	98
16.2.2.3.1.2 การจัดโครงสร้างชุดข้อมูล (Dataset Structure)	99
16.3.2.3.2 ปัญหาการเรียนรู้เกินและการแก้ไขด้วย Data Augmentation	100
16.3.2.3.2.1 ปัญหาการเรียนรู้เกิน (Overfitting)	100
16.3.2.3.2.2 หลักการเพิ่มขยายข้อมูลภาพ (Data Augmentation Theory)	100
16.4.2.3.3 การปฏิบัติการบน Google Colab: การสร้างและฝึกโมเดล (Implementation on Google Colab)	101
16.4.2.3.3.1 โค้ด Python สำหรับการเตรียมข้อมูลและการสร้างโมเดล	102
16.5.2.4 การประดิษฐ์แอปพลิเคชันมือถือ Doctor AI App บน Kodular/App Inventor (Client-Server and Mobile Block Logic)	105
17.2.4 การประดิษฐ์แอปพลิเคชันมือถือ Doctor AI App บน Kodular/App Inventor (Client-Server and Mobile Block Logic)	106
17.1 บทนำเชิงทฤษฎี: แนวคิดการประยุกต์ใช้ปัญญาประดิษฐ์ในการเกษตรปลายทาง (Agro-AI Deployment)	106
17.2.4.1 การออกแบบสถาปัตยกรรมของแอปพลิเคชันมือถือในการเกษตรปลายทาง (Agro-Mobile App Architecture)	107

17.2.1.	ชั้นส่วนหน้า (Client Layer: Mobile Device) . .	107
17.2.2.	ชั้นการสื่อสาร (Communication Layer: Internet/API)	108
17.2.3.	ชั้นส่วนหลัง (Server Layer: Cloud Computing)	108
17.2.4.2	แนวทางการพัฒนารูปแบบไร้โค้ด (No-code / Low-code Mobile App)	109
17.2.4.3	กลไกการสื่อสารข้อมูล: Base64 Encoding และ HTTP POST Request	110
17.4.1.	หลักการของ Base64 Encoding	110
17.4.2.	การส่งข้อมูลด้วย HTTP POST Request	110
17.2.4.4	การจำลองโค้ดส่วนหลัง (Server-Side Backend Simulation)	111
17.2.4.5	การออกแบบฝั่งการตอบลือกคำสั่งใน Kodular (Client-Side Block Logic)	112
17.6.1	องค์ประกอบหลัก (Components) ที่ต้องใช้:	112
17.6.2.	บล็อกคำสั่งการเปิดกล้องและการเลือกรูปภาพ (Image Acquisition Logic)	113
17.6.3.	บล็อกคำสั่งของ Web Component ในการแปลงภาพเป็น Base64 และโพสต์ส่งไปยัง API Endpoint (Encoding and Posting Logic)	113
17.6.4.	บล็อกคำสั่งการรับผลลัพธ์ JSON ตอบกลับและแยกแยะระดับความน่าจะเป็น (Receiving and Parsing Response Logic)	114
17.2.4.6	การวิเคราะห์เปรียบเทียบเชิงวิพากษ์: การแบ่งสืระดับสมเดนชัด (Traditional Thresholding) กับโครงข่ายประสาทเทียมลึก (CNN Deep Learning)	115
17.2.4.7	คำศัพท์สำคัญประจำบทเรียน (Key Terms)	117
17.2.4.8	แบบฝึกหัดทบทวนและโจทย์คิดวิเคราะห์ (Review Exercises)	117
18	บทที่ 3: ชลศาสตร์เขตรากพืช ระบบสมองกลฝังตัว และระบบนำทางอัจฉริยะ (Day 3)	119

18.B.1 ฟิสสิกส์โซลาร์เซลล์ ความนำสารกึ่งตัวนำ และระบบ พลังงานเก็บกักแบตเตอรี่ (Photovoltaic Semiconduc- tor Physics & Battery Systems)	119
19ภาควิชาวิศวกรรมพลังงานและเกษตรดิจิทัล	120
20ตำราเรียน: ฟิสสิกส์พลังงานทางเลือกและระบบเกษตรอัจฉริยะ	121
21หัวข้อ 3.1: ฟิสสิกส์โซลาร์เซลล์ ความนำสารกึ่งตัวนำ และระบบ พลังงานเก็บกักแบตเตอรี่	122
21.B.1 ฟิสสิกส์โซลาร์เซลล์ ความนำสารกึ่งตัวนำ และระบบ พลังงานเก็บกักแบตเตอรี่	122
21.1.B.1.1 หลักการทางฟิสสิกส์ของปรากฏการณ์โฟโตวอล เทอริก (Photovoltaic Effect)	123
21.1.B.1.2 การคำนวณกำลังไฟฟ้าสูงสุดของแผงโซลาร์เซลล์ (Solar Panel Power Output Modeling)	125
21.1.B.1.3 หลักการเคมีไฟฟ้าและระบบประจุพลังงานแบตเตอรี่ (Electrochemistry and Battery Charging)	125
21.1.B.1.4 พารามิเตอร์จำลองสถานะของแบตเตอรี่ (Bat- tery State Parameters)	128
21.1.B.1.5 การจำลองการทำงานของระบบพลังงาน (Python Simulation Example)	129
21.B.2 บอร์ดฝังตัว ESP32, ไดรเวอร์ CH340 และสิ่งแวดล้อม โปรแกรม Arduino IDE	130
22.2 บอร์ดฝังตัว ESP32, ไดรเวอร์ CH340 และสิ่งแวดล้อม โปรแกรม Arduino IDE	131
22.B.2.1 สถาปัตยกรรมของไมโครคอนโทรลเลอร์ ESP32 . . .	131
22.1.B.2.1.1 โครงสร้างทางสถาปัตยกรรม (Architectural Structure)	132
22.1.B.2.1.2 หลักการทำงานของ Wi-Fi และ Bluetooth .	133
22.B.2.2 การทำความเข้าใจแผนผังขาพิน (Pinout Map) ของ ESP32	133
22.2.B.2.2.1 ประเภทของขาพิน (Pin Types)	135
22.2.B.2.2.2 การวิเคราะห์ขา GPIO และ ADC	135

22.3.2.3	ความสำคัญของชิปแปลงสัญญาณ USB-to-Serial (CH340 Driver)	135
22.3.3.2.3.1	บทบาทของ USB-to-Serial Converter . . .	136
22.3.3.2.3.2	ขั้นตอนการติดตั้งไดรเวอร์ (Driver Installation)	136
22.4.2.4	การตั้งค่าสภาพแวดล้อมการพัฒนา Arduino IDE . .	137
22.4.3.2.4.1	การเพิ่มการรองรับบอร์ด ESP32 ผ่าน Boards Manager	137
22.4.3.2.4.2	โครงสร้างหลักของโค้ด (The Sketch Structure)	138
22.4.3.2.4.3	ตัวอย่างโค้ด C++ สำหรับการทดสอบการทำงานพื้นฐาน (Blink Test)	139
22.5	สรุปเชิงวิชาการ (Academic Summary)	140
22.6.3	การเขียนโค้ดอ่านเซนเซอร์ วิเคราะห์กราฟคาลิเบรตดินเหนียวตราด (Soil Moisture Sensor Calibration for Trat Clay)	140
23.3	การเขียนโค้ดอ่านเซนเซอร์ วิเคราะห์กราฟคาลิเบรตดินเหนียวตราด (Soil Moisture Sensor Calibration for Trat Clay)	141
23.1.3.1	หลักการทำงานทางฟิสิกส์ของเซนเซอร์วัดความชื้นแบบเก็บประจุ (Capacitive Principle)	142
23.1.3.1.1	ความแตกต่างระหว่างเซนเซอร์แบบต้านทานและแบบเก็บประจุ	142
23.1.3.1.2	กลไกทางฟิสิกส์ของการวัดค่าความจุไฟฟ้า . .	143
23.2.3.2	ทฤษฎีการสอบเทียบเซนเซอร์ (Sensor Calibration Theory)	144
23.2.3.2.1	การแปลงค่าความจุไฟฟ้าเป็นความชื้นเชิงปริมาตร (VWC)	144
23.2.3.2.2	การกำหนดจุดสอบเทียบ (Defining Calibration Points)	145
23.2.3.2.3	การคำนวณค่าความชื้นและจุดตัดแกน Y . . .	146
23.3.3.3	การประยุกต์ใช้โค้ดและการกรองสัญญาณรบกวน (Implementation and Noise Filtering)	146

23.3.3.3.1 การกรองค่าเฉลี่ยเคลื่อนที่ (Moving Average Filter)	147
23.3.3.3.2 โค้ดตัวอย่าง C++ สำหรับ Arduino (ESP32)	147
23.4.4 ตรรกะระบบควบคุมป้อนกลับ (Feedback Control Loop) สิ่งงานวงจรไฟฟ้าปั้มน้ำด้วยรีเลย์	149
24.4.4 ตรรกะระบบควบคุมป้อนกลับ (Feedback Control Loop) สำหรับการสิ่งงานวงจรไฟฟ้าปั้มน้ำด้วยรีเลย์	150
24.1.4.4.1 หลักการควบคุมระบบอัตโนมัติ	150
24.1.4.4.2 หลักการควบคุมป้อนกลับ (Feedback Control Loop) และระบบ On-Off Controller	151
24.1.4.4.3 การเชื่อมต่อทางไฟฟ้า: รีเลย์และหลักการแยกสัญญาณ (Isolation)	153
24.1.4.4.4 การออกแบบตรรกะระบบควบคุมเชิงปฏิบัติ (Algorithmic Design)	154
24.1.4.4.5 การประยุกต์ใช้โค้ด (C++ Arduino Implementation)	155
24.5.5 แล็บวิจัยเกษตรขั้นสูง: 1D Richards Infiltration Solver (Python) และระบบนำทาง Extended Kalman Filter (EKF)	158
25.5.5 แล็บวิจัยเกษตรขั้นสูง: การจำลองการซึมผ่านน้ำในดิน	
1 มิติด้วยสมการริชาร์ดส และระบบนำทางด้วย Extended Kalman Filter	159
25.1.5.5.1 หลักการบูรณาการทางวิศวกรรมและวิทยาศาสตร์ดิน (Introduction: Principles of Engineering and Soil Science Integration)	159
25.2.5.5.1 ส่วนที่ 1: การจำลองการซึมผ่านน้ำในดิน 1 มิติด้วยสมการริชาร์ดส (1D Richards Infiltration Solver)	160
25.2.5.5.1.1 ทฤษฎีพื้นฐาน: สมการริชาร์ดส (Richards Equation Theory)	160
25.2.5.5.1.2 ความสัมพันธ์ระหว่างตัวแปร (Constitutive Relationships)	161

25.2.3.3 การประยุกต์ใช้กรณีศึกษา: ดินเหนียวจันทบุรี (Chanthaburi Clay)	162
25.2.4.4 การแก้สมการด้วยระเบียบวิธีผลต่างอันดับสี่ (Finite Difference Method: FDM)	162
25.2.5.5 ซอร์สโค้ด Python: ChanthaburiClayDiffusionSolver	163
25.3.6 การประกอบกล่องภายนอกอาคารมาตรฐาน IP65 และ Telemetry ส่งพารามิเตอร์ขึ้น Cloud (MQTT)	166
26.3.6 การประกอบกล่องภายนอกอาคารมาตรฐาน IP65 และ Telemetry ส่งพารามิเตอร์ขึ้น Cloud (MQTT)	167
26.1.1 บทนำเชิงทฤษฎี: การออกแบบระบบ IoT สำหรับสภาพแวดล้อมภายนอกอาคาร (Outdoor IoT Deployments)	167
26.1.3.6.1 หลักการป้องกันการแทรกซึม (Ingress Protection: IP Rating)	168
26.1.3.6.2 แนวทางการประกอบกล่องภายนอกอาคารตามมาตรฐาน IP65	169
26.3.6.3 แนวคิดการสื่อสารข้อมูลระยะไกลด้วยโปรโตคอล MQTT (Telemetry via MQTT)	171
26.2.1. สถาปัตยกรรม Publish/Subscribe (Pub/Sub Architecture)	171
26.2.2. คุณสมบัติของ MQTT (MQTT Properties)	172
26.3.6.4 การประยุกต์ใช้โค้ด: การส่งข้อมูลเซนเซอร์ผ่าน MQTT (C++ Implementation)	172
26.3.1 โค้ดตัวอย่าง (Arduino IDE / C++)	173
26.4.6.5 การวิเคราะห์เปรียบเทียบเชิงวิพากษ์: ควบคุมเปิด-ปิด (On-Off Control) กับการควบคุมคาดการณ์เชิง ชลศาสตร์ (Richards PDE-based Predictive Control)	176
26.5.6.6 คำศัพท์สำคัญประจำบทเรียน (Key Terms)	176
26.6.6.7 แบบฝึกหัดทบทวนและโจทย์คิดวิเคราะห์ (Review Exercises)	178

คู่มือนวัตกรรมเกษตรดิจิทัลรุ่นศูนย์ไทยธานีศาสตร์และเทคโนโลยี มรภ.รำไพพรรณี	
บรรณานุกรมเชิงวิชาการมาตรฐานสากล (References - APA and IEEE Styles)	180
27บทที่ 5: การจัดการข้อมูลและการสื่อสารทางวิชาการ	181
ภาคผนวก: หลักการและรูปแบบการอ้างอิงเชิงวิชาการมาตรฐานสากล (Academic Citation Standards)	182
27.0.๕.๑ บทนำ: ความสำคัญของการอ้างอิงเชิงวิชาการ (Introduction to Academic Referencing)	182
27.0.๕.๒ กรอบแนวคิดทางทฤษฎีของการอ้างอิง (Theoretical Framework of Citation)	183
27.0.๕.๓ การประยุกต์ใช้รูปแบบ APA 7th Edition (American Psychological Association)	184
27.0.๕.๔ การประยุกต์ใช้รูปแบบ IEEE Style (Institute of Electrical and Electronics Engineers)	185
27.0.๕.๕ การปฏิบัติการสร้างบรรณานุกรมเชิงวิชาการ (Practical Bibliography Construction)	186
27.0.๖ บรรณานุกรม (Bibliography)	187
27.0.๕.๖ การประยุกต์ใช้โค้ดเพื่อการจัดการข้อมูลอ้างอิง (Code Implementation for Citation Management)	189
ภาคผนวก ก: ดัชนีคำศัพท์วิชาการ (Subject Index)	192
ภาคผนวก ข: คู่มือแผนการจัดการติดตั้งกล่องภาคสนาม (Field Deployment Project Plan)	194
27.0.๘ โรงเรียนประณีตวิทยาคม อำเภอเขาสมิง จังหวัดตราด	194

Chapter 1

โครงการศูนย์พัฒนานวัตกรรม เกษตรดิจิทัลไร่ไพพรรณี ประณีตวิทยาคม

1.1 คู่มือนวัตกรรมเกษตรดิจิทัลรุ่นใหม่: การบูรณาการ
ปัญญาประดิษฐ์และอินเทอร์เน็ตของสรรพสิ่งใน
แปลงเกษตรแม่นยำ

1.1.1 คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพ
พรรณี

แหล่งที่มาของงบประมาณ: งบประมาณรายจ่ายประจำปีงบประมาณ
พ.ศ. 2569 (งบแผ่นดิน 76,000 บาท - รหัสโครงการ 110100463) ผู้
อำนวยการโครงการ: ผู้ช่วยศาสตราจารย์ ดร.ชีวะ ทัดนา และ ผู้ช่วย
ศาสตราจารย์อรรถกร คำฉัตร

สารบัญรายละเอียดวิชาเรียน อบรม 3 วัน

1.2 โครงการศูนย์พัฒนานวัตกรรมเกษตรดิจิทัลไร่ไฟ พรรณี ประณีตวิทยาคม

กิจกรรม อบรมเชิงปฏิบัติการ การพัฒนานวัตกรรมเกษตรดิจิทัลด้วยปัญญา
ประดิษฐ์ ณ โรงเรียนประณีตวิทยาคม อ.เขาสมิง จ.ตราด

เรียนรู้ (Preface) (file:///preface.md)

อบรมเวลา (Day 1) (file:///ch1/ch1_s1.md)

ทุเรียนตะวันออก (file:///ch1/ch1_s1.md)

ับนวัตกรรมเขาวขน (file:///ch1/ch1_s2.md)

โครงสร้าง JSON (file:///ch1/ch1_s3.md)

าศด้วย Pandas (file:///ch1/ch1_s4.md)

สนุนการตัดสินใจ (file:///ch1/ch1_s5.md)

ห์โรคพืช (Day 2) (file:///ch2/ch2_s1.md)

อมพิวเตอร์วิทัศน์ (file:///ch2/ch2_s1.md)

oling, Softmax) (file:///ch2/ch2_s2.md)

ารส่งออก TFLite (file:///ch2/ch2_s3.md)

r/App Inventor (file:///ch2/ch2_s4.md)

อัจฉริยะ (Day 3) (file:///ch3/ch3_s1.md)

เก็บกักแบตเตอรี่ (file:///ch3/ch3_s1.md)

รวม Arduino IDE (file:///ch3/ch3_s2.md)

ติดตั้งนิวทราด (file:///ch3/ch3_s3.md)

ปั้มน้ำด้วยรีเลย์ (file:///ch3/ch3_s4.md)

han Filter (EKF) (file:///ch3/ch3_s5.md)

Cloud (MQTT) (file:///ch3/ch3_s6.md)

ล (References) (file:///references.md)

—

หน้าปก คำนำ และการ วิเคราะห์วัตถุประสงค์เชิงเป้า หมายการเรียนรู้ (Preface & Educational Objectives Matrix)

(หมายเหตุ: เนื้อหาต่อไปนี้จะถูกเขียนขึ้นในรูปแบบของตำราเรียนวิชาการ
ระดับสูง โดยใช้ภาษาไทยทางการวิชาการที่เคร่งครัดและสละสลวย เพื่อ
ให้สอดคล้องกับบทบาทของศาสตราจารย์ผู้เชี่ยวชาญ)

Chapter 2

คู่มือนวัตกรรมเกษตรดิจิทัล
เยาวชน:

Chapter 3

การบูรณาการปัญญาประดิษฐ์ และอินเทอร์เน็ตของสรรพสิ่ง ในแปลงเกษตรแม่นยำ

(Digital Agri-Innovator Handbook for Youth: Integrating Artificial Intelligence and Internet of Things in Precision Farming)

โดย [ชื่อ-นามสกุล ศาสตราจารย์ผู้เชี่ยวชาญ] รองศาสตราจารย์ ดร.
(รศ.ดร.) สาขาฟิสิกส์ประยุกต์และเกษตรดิจิทัล

สถาบันวิชาการแห่งการพัฒนาเทคโนโลยีเกษตรยั่งยืน ปีการศึกษา
พ.ศ. 2567

3.1 คำนำ (Preface)

ในฐานะที่โลกกำลังเผชิญหน้ากับพลวัตของการเปลี่ยนแปลงสภาพภูมิอากาศ (Climate Change) อย่างที่ไม่เคยปรากฏมาก่อน ภาคเกษตรกรรมซึ่งเป็นรากฐานทางเศรษฐกิจและความมั่นคงทางอาหารของประเทศได้รับผลกระทบอย่างรุนแรงและซับซ้อน การเปลี่ยนแปลงของรูปแบบสภาพอากาศ (Climate Pattern Shift) ได้ส่งผลให้เกิดความแปรปรวนของทรัพยากรธรรมชาติอย่างมีนัยสำคัญ โดยเฉพาะอย่างยิ่งในพื้นที่ภาคตะวันออกของประเทศไทย ตั้งแต่จังหวัดจันทบุรีจนถึงตราด ซึ่งเป็นแหล่งปลูก

ทุเรียนที่มีความสำคัญทางเศรษฐกิจระดับประเทศ

ปรากฏการณ์ที่เด่นชัดที่สุดคือการเกิดภาวะภัยแล้ง (Drought) ที่มีความรุนแรงและยาวนานขึ้น ควบคู่ไปกับการเกิดช่วงฝนทิ้งช่วง (Dry Spells) ที่ทำให้วงจรการเพาะปลูกแบบดั้งเดิม (Traditional Farming) ไม่สามารถตอบสนองต่อความผันผวนทางสิ่งแวดล้อมได้อีกต่อไป การพึ่งพาองค์ความรู้และประสบการณ์เชิงประจักษ์ (Empirical Knowledge)

เพียงอย่างเดียวจึงไม่เพียงพอต่อการรับมือกับความท้าทายระดับโลกนี้

ด้วยเหตุนี้ องค์ความรู้ด้านการเกษตรจึงจำเป็นต้องได้รับการยกระดับครั้งใหญ่ จากการทำเกษตรแบบเหมารวม (Uniform Farming) ไปสู่แนวคิด เกษตรแม่นยำ (Precision Agriculture) ซึ่งเป็นศาสตร์ที่มุ่งเน้นการจัดการทรัพยากร (Resource Management) อย่างจำเพาะเจาะจงในระดับพื้นที่ย่อย (Site-Specific Management) โดยอาศัยการรวบรวมวิเคราะห์ และการประมวลผลข้อมูลขนาดใหญ่ (Big Data)

ตำราเล่มนี้จึงถูกจัดทำขึ้นเพื่อเป็นเครื่องมือในการบูรณาการศาสตร์หลายแขนงเข้าด้วยกัน ได้แก่ ฟิสิกส์ประยุกต์ (Applied Physics) ในการทำความเข้าใจปฏิสัมพันธ์ของพลังงานและสสารในระบบนิเวศ, อินเทอร์เน็ตของสรรพสิ่ง (Internet of Things: IoT) ในการสร้างเครือข่ายเซนเซอร์อัจฉริยะ, และ ปัญญาประดิษฐ์ (Artificial Intelligence: AI) ในการวิเคราะห์รูปแบบ (Pattern Recognition) และการตัดสินใจเชิงรุก (Proactive Decision Making)

การเรียนรู้ตามแนวทางนี้สอดคล้องกับแนวคิด เศรษฐกิจชีวภาพ เศรษฐกิจหมุนเวียน และเศรษฐกิจสีเขียว (Bio-Circular-Green Economy: BCG) ซึ่งไม่ได้มุ่งเน้นเพียงแค่การเพิ่มผลผลิต (Yield Maximization) เท่านั้น แต่ยังรวมถึงการลดการใช้ทรัพยากร (Resource Minimization) การสร้างระบบนิเวศที่ยั่งยืน และการสร้างมูลค่าเพิ่มทางเทคโนโลยีอย่างเป็นระบบ

เป้าหมายสูงสุดของตำราเล่มนี้คือการส่งมอบองค์ความรู้เชิงปฏิบัติ (Practical Knowledge) ให้แก่นักเรียนรุ่นเยาว์ เพื่อให้พวกเขาสามารถ

ก้าวจากการเป็นเพียงผู้รับความรู้ ไปสู่การเป็น นวัตกรรม (Innovator) ที่สามารถออกแบบ ประดิษฐ์ และติดตั้งระบบเกษตรอัจฉริยะด้วยตนเอง โดยมีโครงการหลักคือการสร้างและติดตั้ง "กล่องนวัตกรรมฟาร์ม" (Smart Farm Innovator Box) ในแปลงทุเรียนจริง ณ โรงเรียนประณีตวิทยาคม อำเภอเขาสมิง จังหวัดตราด เพื่อให้เกิดการเรียนรู้แบบวงจรปิด (Closed-Loop Learning) ที่เชื่อมโยงทฤษฎีทางวิชาการเข้ากับการปฏิบัติจริงในบริบททางภูมิศาสตร์และเศรษฐกิจท้องถิ่นอย่างสมบูรณ์ที่สุด

ขอให้ผู้เรียนทุกท่านจงใช้ตำราเล่มนี้เป็นจุดเริ่มต้นของการเดินทางทางปัญญาที่ยิ่งใหญ่ เพื่อสร้างสรรค์อนาคตเกษตรกรรมที่ยั่งยืนด้วยพลังแห่งเทคโนโลยีและปัญญาของตนเอง

3.2 การวิเคราะห์วัตถุประสงค์เชิงเป้าหมายการเรียนรู้ (Educational Objectives Matrix)

เพื่อให้การเรียนรู้เป็นไปอย่างมีโครงสร้างและสามารถวัดผลได้อย่างเป็นระบบ ตำราเล่มนี้จึงได้กำหนดวัตถุประสงค์การเรียนรู้ (Learning Objectives) โดยอ้างอิงตามกรอบอนุกรมวิธานของบลูม (Bloom's Taxonomy) ซึ่งเป็นกรอบแนวคิดที่ใช้วัดระดับความลึกของการคิด (Depth of Thinking) ตั้งแต่ระดับพื้นฐานที่สุดไปจนถึงระดับการสร้างสรรค์นวัตกรรมขั้นสูงสุด

วัตถุประสงค์เหล่านี้ถูกออกแบบมาเพื่อนำพานักเรียนให้พัฒนาทักษะการคิดอย่างเป็นลำดับขั้น (Hierarchical Thinking Skills) ดังตารางเมทริกซ์ต่อไปนี้:

ระดับการคิด (Cognitive Level)	คำนิยามเชิง ทฤษฎี (Theoretical Definition)	วัตถุประสงค์ การเรียนรู้ (Learning Objectives)	ตัวชี้วัดเชิง ปฏิบัติ (Practical Indicators)
1. ความรู้ (Knowledge)	การจดจำและ เรียกคืนข้อมูล ข้อเท็จจริง หรือคำจำกัด ความที่เคย เรียนรู้มา (Recall and Recognition).	ผู้เรียนสามารถ ระบุหลักการ พื้นฐานทาง ฟิสิกส์ (Physics Principles) และองค์ ประกอบทาง เทคโนโลยี (Technologi- cal Compo- nents) ที่ เกี่ยวข้องกับ ระบบเกษตร แม่นยำได้อย่าง ถูกต้อง.	1.1 อธิบาย หลักการทำงานของ เซนเซอร์วัด ความชื้นในดิน (Soil Moisture Sensor) และ เซนเซอร์วัดค่า pH (pH Sensor) ได้. 1.2 ระบุ ความแตกต่าง ระหว่าง สถาปัตยกรรม IoT (Internet of Things) และระบบ ควบคุมแบบ ดั้งเดิม (Traditional Control System).
2. ความเข้าใจ (Comprehen- sion)	การตีความ การอธิบาย ความหมาย หรือการสรุป แนวคิดจาก ข้อมูลที่ได้รับ (Interpreta- tion and Explanation).	ผู้เรียนสามารถ อธิบายความ สัมพันธ์เชิง สาเหตุและ ผลลัพธ์ (Cause-and- Effect Relationship) ระหว่างตัวแปร ทางสิ่งแวดล้อม (Environmen- tal Variables) กับสุขภาพของ พืชได้อย่างสม เหตุสมผล.	2.1 อธิบาย ว่าการ เปลี่ยนแปลง ของค่าความ ตึงเครียดของ น้ำในดิน (Soil Water Tension) ส่งผลต่ออัตราการ คายน้ำ (Tran- spiration Rate) ของ พืชเรียนอย่างไร. 2.2 แปล ผลของผลที่ได้

3.3 โครงการปฏิบัติการ: การสร้าง ”กล่องนวัตกรรมสมาร์ทฟาร์ม” (Smart Farm Innovator Box)

3.3.1 1. หลักการและเหตุผลเชิงวิชาการ (Academic Rationale)

โครงการนี้มีใช้เพียงการประกอบชิ้นส่วนอิเล็กทรอนิกส์ (Electronic Assembly) แต่เป็นการจำลองระบบนิเวศทางเทคโนโลยี (Technological Ecosystem) ที่สมบูรณ์แบบ ซึ่งต้องอาศัยการบูรณาการทางวิศวกรรมไฟฟ้า (Electrical Engineering), วิทยาการคอมพิวเตอร์ (Computer Science), และหลักการทางฟิสิกส์ของสิ่งแวดล้อม (Environmental Physics) เข้าด้วยกัน

กล่องนวัตกรรมสมาร์ทฟาร์ม ถูกออกแบบให้เป็นศูนย์กลางการรวบรวมข้อมูล (Data Aggregation Hub) ที่สามารถวัดและประมวลผลตัวแปรทางสิ่งแวดล้อมหลักที่ส่งผลต่อการเจริญเติบโตของพืช (Plant Growth Variables) ในแปลงทุเรียนจริง โดยเฉพาะอย่างยิ่งการวัดค่าที่เกี่ยวข้องกับ ความเครียดของน้ำ (Water Stress) และ ความเครียดจากความร้อน (Heat Stress)

3.3.2 2. องค์ประกอบทางฟิสิกส์และเทคโนโลยีที่ใช้ (Physical and Technological Components)

ระบบนี้ประกอบด้วย 3 ส่วนหลักที่ทำงานร่วมกัน:

2.1 ส่วนการรับรู้ (Sensing Layer)

- เซนเซอร์ความชื้นในดิน (Soil Moisture Sensor): วัดค่าความชื้นในดิน โดยทั่วไปจะวัดค่าความต้านทานไฟฟ้า (Electrical Resistance) หรือค่าศักย์ไฟฟ้า (Potential Voltage) ของน้ำในดิน ซึ่งมี

ความสัมพันธ์โดยตรงกับปริมาณน้ำอิสระ (Free Water Content) ในช่องว่างของดิน (Soil Pore Space).

- หลักการทางฟิสิกส์: การวัดค่าความต้านทานไฟฟ้าของสารละลายอิเล็กโทรไลต์ (Electrolyte Solution) ที่อยู่ในดิน ซึ่งค่าความต้านทานจะเพิ่มขึ้นเมื่อความชื้นลดลง.
- เซนเซอร์วัดอุณหภูมิและความชื้นสัมพัทธ์ (Temperature and Relative Humidity Sensor): วัดค่าอุณหภูมิอากาศ (T) และความชื้นสัมพัทธ์ (RH) ซึ่งเป็นตัวแปรสำคัญในการคำนวณอัตราการคายน้ำ (Transpiration Rate).
- เซนเซอร์วัดแสง (Light Sensor/Photodiode): วัดค่าความเข้มของแสง (Light Intensity) ในหน่วยลักซ์ (Lux) หรือวัตต์ต่อตารางเมตร (W/m^2) ซึ่งส่งผลต่อการสังเคราะห์ด้วยแสง (Photosynthesis).

2.2 ส่วนการประมวลผลและควบคุม (Processing and Control Layer)

- ไมโครคอนโทรลเลอร์ (Microcontroller Unit: MCU): เช่น Arduino หรือ ESP32 ทำหน้าที่เป็นสมองของระบบ ทำการอ่านค่าจากเซนเซอร์ (Analog-to-Digital Conversion) และประมวลผลตามตรรกะที่กำหนด (Control Logic).
- โมดูลการสื่อสาร (Communication Module): เช่น ชิพ Wi-Fi/Bluetooth ที่ติดตั้งกับบอร์ด ESP32 เพื่อทำหน้าที่ส่งข้อมูลผ่านระบบเครือข่ายอินเทอร์เน็ตไปยังคลาวด์เซิร์ฟเวอร์แบบไร้สาย
- ตู้ควบคุมมาตรฐานอุตสาหกรรม (Industrial Enclosure): กล่องพลาสติกอะคริลิกมาตรฐาน IP65 เพื่อป้องกันฝุ่นละอองและไอน้ำความชื้นจากน้ำฝนในแปลงเกษตรจริงภายนอกอาคาร

2.3 ส่วนการตอบสนองและสั่งการ (Actuator Layer)

- โมดูลรีเลย์ (Relay Module): ทำหน้าที่เป็นสวิตช์แม่เหล็กไฟฟ้าในการเปิดและปิดกระแสไฟกระแสสลับ (AC) หรือกระแสตรง (DC) เพื่อควบคุมการทำงานของปั้มน้ำหรือสปริงเกลอร์จ่ายน้ำทุเรียน
- ระบบโซลาร์เซลล์พลังงานแสงอาทิตย์ (Off-Grid Solar Power System): ประกอบด้วยแผงโซลาร์เซลล์ (Solar Panel), เครื่องควบคุมการชาร์จประจุ (Solar Charge Controller), และแบตเตอรี่เก็บกักพลังงาน (เช่น LiFePO_4) เพื่อจ่ายไฟฟ้ากระแสตรงให้แก่ไมโครคอนโทรลเลอร์อย่างยั่งยืนตลอด 24 ชั่วโมง

3.4 รายการสัญลักษณ์และตัวย่อทางวิชาการ (List of Symbols and Abbreviations)

เพื่อความเข้าใจที่เป็นเอกภาพในการศึกษาทฤษฎีทางฟิสิกส์ เกษตรดิจิทัล และวิศวกรรมควบคุมในตำราเล่มนี้ ตารางต่อไปนี้สรุปสัญลักษณ์และคำย่อสำคัญที่ปรากฏตลอดทั้งเล่ม:

สัญลักษณ์	ชื่อภาษาไทย	ชื่อภาษาอังกฤษ	หน่วยหลัก (SI Unit)	บริบททางวิชาการ
R_n	รังสีสุทธิ ของเรือน ยอด	Net Radiation	W/m^2 (วัดต่อตา รางเมตร)	ฟิสิกส์ พลังงาน และภูมิ อากาศพืช
G	ฟลักซ์ความ รอนนำใน ดิน	Soil Heat Flux	W/m^2	ชลศาสตร์ และฟิสิกส์ ชั้นดิน
H	ฟลักซ์ความ รอนสัมผัส อากาศ	Sensible Heat Flux	W/m^2	การถ่ายเท ความร้อน ในอากาศ
$\lambda E / \lambda ET_o$	ฟลักซ์ความ รอนแฝง การระเหย	Latent Heat Flux	W/m^2	อัตราการ คายระเหย น้ำของ ทุเรียน
VPD	ความต่าง แรงดันไอน้ำ ในอากาศ	Vapor Pressure Deficit	kPa (กิโล ปาสคาล)	สรีรวิทยา และการ คายน้ำของ ใบพืช
θ	ความชื้นดิน เชิงปริมาตร	Volumetric Water Content	m^3/m^3 หรือ %	ฟิสิกส์ของ ดินเขตราก พืช
ψ	ศักย์แรงดึง น้ำในดิน	Soil Water Matrix Potential	kPa หรือ cm (หัวน้ำ)	ความยาก ง่ายในการ ดูดน้ำของ พืช
$K(\theta)$	ความนำไฮ ดรอลิกของ น้ำในดิน	Hydraulic Conduc- tivity	cm/s หรือ m/day	การไหล และ เคลื่อนที่ ของน้ำใน ดิน
NDVI	ดัชนีพืช พรรณผล ต่างปกติ	Normalized Difference Vegeta- tion Index	ไม่มีหน่วย (ค่าระหว่าง -1 ถึง 1)	ฟิสิกส์ของ คลื่นแสง และ ดาวเทียม
EVI	ดัชนีพืช พรรณ ปรับปรุง	Enhanced Vegeta- tion Index	ไม่มีหน่วย	การ วิเคราะห์ ข้อมูล

Chapter 4

บทที่ 1: ฟิสิกส์อุตุนิยมวิทยา เกษตรและการประมวลผล ข้อมูลอนุกรมเวลา (Day 1)

ระดับวิชา: กายภาพอุตุนิยมวิทยาการเกษตรและวิทยาการวิเคราะห์ข้อมูล
ระดับเบื้องต้น ผู้บรรยาย: ผู้ช่วยศาสตราจารย์ ดร.ชีวะ ทัดนา และ ผู้
ช่วยศาสตราจารย์อรรถกร คำจันทร์

4.1 1.1 ดุลภูมิอากาศและสรีรวิทยาของทุเรียนตะวันออก (Agro-Climatology and Eastern Durian Physiology)

Chapter 5

ตำราวิชาการ: ฟิสิกส์เกษตร และระบบนิเวศดิจิทัล

5.1 บทที่ 1.1 ดุลภูมิอากาศและสรีรวิทยาของทุเรียน ตะวันออก (Agro-Climatology and Eastern Durian Physiology)

5.1.1 บทนำ: ความสำคัญของการทำความเข้าใจปฏิสัมพันธ์ระหว่าง สิ่งแวดล้อมกับสรีรวิทยาพืช

การเพาะปลูกพืชเศรษฐกิจที่มีมูลค่าสูง เช่น ทุเรียน (*Durio zibethinus*) ไม่ได้ขึ้นอยู่กับการจัดการปัจจัยการผลิตเพียงด้านเดียว แต่ต้องอาศัยความเข้าใจอย่างลึกซึ้งในปฏิสัมพันธ์ระหว่างปัจจัยทางภูมิอากาศ (Climatic Factors) และกระบวนการทางสรีรวิทยา (Physiological Processes) ของพืช การที่พืชจะสามารถเจริญเติบโตได้อย่างสมบูรณ์ โดยเฉพาะอย่างยิ่งในระยะวิกฤต เช่น ระยะออกดอก (Flowering Stage) และระยะติดผล (Fruiting Stage) จำเป็นต้องมีการจัดการความเครียด (Stress Management) อย่างแม่นยำ

ในบริบทของทุเรียนตะวันออก (Eastern Durian) ซึ่งเป็นสายพันธุ์ที่มีความต้องการสภาพแวดล้อมที่เฉพาะเจาะจง การวิเคราะห์ทางฟิสิกส์

เกษตร (Agro-Physics) จึงเข้ามามีบทบาทสำคัญในการสร้างแบบจำลองทางคณิตศาสตร์เพื่อทำนายสภาวะความเครียดที่พืชอาจเผชิญ เพื่อให้เกษตรกรสามารถปรับปรุงการจัดการน้ำและธาตุอาหารได้อย่างเหมาะสมที่สุด

5.1.2 1. การวิเคราะห์กายภาพและสรีรวิทยาการเจริญเติบโตของทุเรียนในระยะวิกฤต

1.1 ลักษณะสรีรวิทยาของทุเรียนในระยะออกดอกและติดผล

ทุเรียนเป็นพืชที่จัดอยู่ในกลุ่มไม้ยืนต้นเขตร้อน (Tropical Evergreen Tree) การเจริญเติบโตของทุเรียนมีความซับซ้อนทางสรีรวิทยา โดยเฉพาะอย่างยิ่งในช่วงที่เกิดการสืบพันธุ์ (Reproductive Stage) ซึ่งเป็นระยะที่พืชมีความไวต่อสภาวะตึงเครียด (Stress Susceptibility) สูงมาก

ระยะออกดอก (Flowering Stage): ในช่วงนี้ พืชต้องใช้พลังงานจำนวนมากในการสร้างโครงสร้างสืบพันธุ์ (Reproductive Structures) เช่น ดอกและรังไข่ การที่ต้นทุเรียนจะสามารถสร้างดอกที่สมบูรณ์และติดผลได้นั้น ต้องอาศัยการจัดการน้ำที่เหมาะสมอย่างยิ่ง หากเกิดภาวะขาดน้ำ (Drought Stress) ในช่วงนี้ จะส่งผลให้เกิดการยุบของดอก (Flower Abortion) หรือการติดผลที่ไม่สมบูรณ์ (Poor Fruit Set) เนื่องจากกระบวนการทางสรีรวิทยาที่เกี่ยวข้องกับการสังเคราะห์แสง (Photosynthesis) และการลำเลียงน้ำ (Transpiration) ถูกรบกวน

ระยะติดผล (Fruiting Stage): เมื่อผลเริ่มขยายขนาด (Fruit Enlargement) ความต้องการน้ำของพืชจะเพิ่มขึ้นอย่างก้าวกระโดด (Exponential Increase in Water Demand) การขาดน้ำอย่างรุนแรงจะกระตุ้นให้เกิดการปิดปากใบ (Stomatal Closure) เพื่อลดการสูญเสียน้ำ (Water Loss) อย่างไรก็ตาม การปิดปากใบที่มากเกินไปจะส่งผลให้การแลกเปลี่ยนก๊าซ (Gas Exchange) ลดลง ทำให้การสังเคราะห์แสง

ลดลงตามไปด้วย ซึ่งส่งผลโดยตรงต่อการสะสมมวลชีวภาพ (Biomass Accumulation) และคุณภาพของผลผลิต

1.2 กลไกการตอบสนองต่อความเครียดจากน้ำ (Water Stress Response Mechanism)

เมื่อพืชเผชิญกับภาวะขาดน้ำ ความเครียดจากน้ำ (Water Stress) จะส่งผลให้ศักย์น้ำในเนื้อเยื่อ (Water Potential, ψ) ของพืชลดต่ำลง (become more negative) กลไกหลักที่พืชใช้ในการตอบสนองคือการลดอัตราการคายน้ำ (Transpiration Rate) ผ่านการควบคุมการเปิดปิดของปากใบ (Stomatal Control)

ความสัมพันธ์นี้สามารถอธิบายได้ด้วยสมการที่เกี่ยวข้องกับความดันไอน้ำ (Vapor Pressure) ซึ่งเป็นหัวใจสำคัญในการทำนายความเข้าใจการถ่ายเทน้ำจากใบสู่บรรยากาศ

5.1.3 2. แนวคิดทางฟิสิกส์เกษตร: ดัชนีความตึงเครียดจากน้ำและอุณหภูมิเรือนยอด

2.1 ดัชนีความตึงเครียดจากน้ำของพืช (Crop Water Stress Index: CWSI)

นิยามและหลักการ: ดัชนีความตึงเครียดจากน้ำของพืช (CWSI) เป็นตัวชี้วัดเชิงปริมาณ (Quantitative Index) ที่ใช้ประเมินระดับความเครียดจากน้ำที่พืชกำลังเผชิญอยู่ โดยอาศัยการเปรียบเทียบระหว่างสถานะความชื้นที่พืชต้องการ (Potential Stress) กับสถานะความชื้นที่พืชได้รับจริง (Actual Stress)

หลักการทางฟิสิกส์: CWSI มักคำนวณโดยใช้ข้อมูลจากความชื้นในดิน (Soil Moisture) และข้อมูลสภาพอากาศ (Meteorological Data) โดยเฉพาะอย่างยิ่งการคำนึงถึงความแตกต่างระหว่างความชื้นที่ดินที่เหมาะสม (Optimal Soil Moisture) กับความชื้นที่ดินปัจจุบัน (Current Soil Moisture)

โดยทั่วไปแล้ว ค่า CWSI จะมีค่ามากกว่า 1.0 เมื่อพืชอยู่ในภาวะเครียดจากน้ำปานกลางถึงรุนแรง และค่าเข้าใกล้ 0 เมื่อพืชอยู่ในสภาวะน้ำที่เหมาะสม (Optimal Water Status)

การประยุกต์ใช้: การติดตามค่า CWSI อย่างต่อเนื่องช่วยให้ผู้ปฏิบัติงานสามารถวางแผนการให้น้ำ (Irrigation Scheduling) ได้อย่างแม่นยำ แทนที่จะใช้วิธีการให้น้ำตามตารางเวลา (Time-based Irrigation) ซึ่งอาจนำไปสู่การสูญเสียทรัพยากรน้ำอย่างไม่จำเป็น

2.2 อุณหภูมิเรือนยอด (Canopy Temperature)

แนวคิด: อุณหภูมิเรือนยอด (Canopy Temperature, T_c) คืออุณหภูมิเฉลี่ยของกลุ่มใบและกิ่งก้านของพืชในพื้นที่จำกัด (Micro-environment) ซึ่งแตกต่างจากอุณหภูมิอากาศรอบข้าง (Air Temperature, T_a)

ความสำคัญทางสรีรวิทยา: ในสภาวะที่พืชมีการคายน้ำสูง (High Transpiration) พลังงานความร้อนส่วนหนึ่งจะถูกนำไปใช้ในการเปลี่ยนสถานะของน้ำ (Latent Heat of Vaporization) ซึ่งมีผลให้ T_c มีค่าต่ำกว่า T_a (Cooling Effect)

เมื่อพืชประสบภาวะขาดน้ำ (Water Stress) การคายน้ำจะลดลงอย่างมาก (Stomatal Closure) ส่งผลให้กลไกการระบายความร้อนนี้ถูกปิดตัวลง ทำให้ T_c มีแนวโน้มที่จะสูงขึ้นและเข้าใกล้ T_a มากขึ้น การวัดความแตกต่างของอุณหภูมินี้จึงเป็นตัวบ่งชี้ทางอ้อม (Indirect Indicator) ที่มีประสิทธิภาพว่าพืชกำลังประสบกับภาวะตึงเครียดจากน้ำหรือไม่

5.1.4 3. การวิเคราะห์สมการดุลพลังงานของเรือนยอดพืช (Canopy Energy Balance Equation)

การทำความเข้าใจว่าพลังงานความร้อนและรังสีถูกถ่ายเทอย่างไรในระบบเรือนยอดพืช (Canopy System) เป็นหัวใจสำคัญของฟิสิกส์เกษตร สมการ

ที่ใช้ในการวิเคราะห์นี้คือสมการดุลพลังงาน (Energy Balance Equation)

สมการหลัก: พลังงานสุทธิที่ได้รับโดยเรือนยอดพืช (R_n) จะต้องถูกถ่ายโอนออกไปในรูปแบบต่างๆ ตามหลักการอนุรักษ์พลังงาน (Law of Conservation of Energy)

$$R_n = G + H + \lambda ET_o$$

3.1 คำอธิบายตัวแปรทางฟิสิกส์และนิยามเชิงลึก

การวิเคราะห์เชิงลึกของแต่ละเทอม:

1. **R_n (Net Radiation):** R_n คือพลังงานทั้งหมดที่ขับเคลื่อนกระบวนการทางชีวภาพ (Photosynthesis) และการระเหยน้ำ พลังงานนี้ประกอบด้วยรังสีคลื่นสั้น (Shortwave Radiation) และรังสีคลื่นยาว (Long-wave Radiation) การคำนวณที่แม่นยำต้องพิจารณาการดูดซับ (Absorption) และการสะท้อน (Reflection) ขององค์ประกอบต่างๆ ในเรือนยอด
2. **G (Soil Heat Flux):** G สะท้อนถึงความร้อนที่สะสมอยู่ในชั้นดิน การเปลี่ยนแปลงของ G บ่งชี้ถึงความสามารถในการกักเก็บความร้อนของระบบรากและดิน ซึ่งมีผลต่อการควบคุมอุณหภูมิของรากพืช
3. **H (Sensible Heat Flux):** H เป็นตัวบ่งชี้ถึงความแตกต่างของอุณหภูมิระหว่างเรือนยอดกับอากาศ หาก H มีค่าสูง หมายความว่ามีการถ่ายเทความร้อนสูง ซึ่งมักเกิดขึ้นเมื่อความแตกต่างของอุณหภูมิมีมาก
4. **λET_o (Latent Heat Flux):** λ คือค่าความร้อนแฝงของการกลายเป็นไอ (Latent Heat of Vaporization) ซึ่งมีค่าคงที่ทางกายภาพ (ประมาณ 2.45 kJ/kg ที่อุณหภูมิมาตรฐาน) ET_o คืออัตราการคายระเหยน้ำอ้างอิง (Reference Evapotranspiration) ซึ่งเป็นตัวกำหนดปริมาณน้ำที่ระบบต้องสูญเสียไปในรูปของไอน้ำ

ตัวแปร	ชื่อเต็ม (English Term)	หน่วย SI	คำอธิบายเชิง ฟิสิกส์
R_n	Net Radiation (รังสีสุทธิ)	W/m^2	คือพลังงานรังสีสุทธิที่เข้าสู่เรือนยอดพืช ($R_{net} = R_{solar} - R_{reflected}$ โดย R_{solar} คือรังสีดวงอาทิตย์ และ $R_{reflected}$ คือรังสีที่สะท้อนกลับ)
G	Soil Heat Flux (ฟลักซ์ความร้อนในดิน)	W/m^2	คือการถ่ายเทความร้อนระหว่างพื้นผิว (ดิน) กับบรรยากาศผ่านการนำความร้อน (Conduction) และการพาความร้อน (Convection)
H	Sensible Heat Flux (ฟลักซ์ความร้อนสัมผัส)	W/m^2	คือการถ่ายเทความร้อนที่เกี่ยวข้องกับอุณหภูมิ (Temperature Gradient) ระหว่างเรือนยอดกับอากาศโดยรอบ (Convection)
λET_o	Latent Heat Flux (ฟลักซ์ความร้อนแฝง)	W/m^2	คือพลังงานความร้อนที่ใช้ในการเปลี่ยนสถานะของน้ำจากของเหลวเป็นไอ (Phase

การนำไปใช้ในการจัดการน้ำ: สมการนี้ช่วยให้เราเข้าใจว่า หากเราต้องการให้พืชมีการคายน้ำสูง (เพื่อระบายความร้อนและดึงสารอาหาร) เราต้องมั่นใจว่ามีพลังงานรังสี (R_n) เพียงพอ และต้องมีการจัดการน้ำที่เพียงพอเพื่อรองรับการสูญเสียพลังงานในรูปของ λET_o .

5.1.5 4. ความสัมพันธ์ระหว่างอุณหภูมิ ความชื้น และความดันไอน้ำ (Vapor Pressure Deficit: VPD)

4.1 แนวคิดพื้นฐานของความดันไอน้ำ (Vapor Pressure)

****ความดันไอน้ำ (Vapor Pressure, P_v):**** คือความดันที่ไอน้ำในสถานะแก๊ส (Water Vapor) ออกฤทธิ์เหนือของเหลว (Liquid Water) ในระบบปิด (เช่น ในใบไม้ หรือน้ำในดิน)

****ความชื้นสัมพัทธ์ (Relative Humidity, RH):**** คืออัตราส่วนระหว่างความดันไอน้ำจริง (P_v) กับความดันไอน้ำอิ่มตัว (P_{sat}) ที่ระดับอุณหภูมินั้นๆ

$$RH = \frac{P_v}{P_{sat}} \times 100\%$$

4.2 ความแตกต่างของความดันไอน้ำ (Vapor Pressure Deficit: VPD)

นิยาม: VPD คือความแตกต่างระหว่างความดันไอน้ำที่อากาศสามารถบรรจุได้สูงสุด (Saturation Vapor Pressure, P_{sat}) กับความดันไอน้ำจริง (P_v) ที่มีอยู่ในอากาศ ณ ขณะนั้น

$$VPD = P_{sat} - P_v$$

ความสำคัญทางสรีรวิทยา: VPD เป็นตัวแปรทางสภาพอากาศที่สำคัญที่สุดตัวหนึ่งในการควบคุมอัตราการคายน้ำ (Transpiration Rate) ของพืช

1. VPD สูง (High VPD): หมายความว่าอากาศแห้งมาก (Air is Dry) ความแตกต่างของความดันไอน้ำระหว่างภายในใบ (ซึ่งมี P_v สูง) กับภายนอก (ซึ่งมี P_v ต่ำ) มีมาก ทำให้แรงขับเคลื่อน (Driving Force) ในการคายน้ำสูงมาก พืชจึงต้องเปิดปากใบกว้างเพื่อระบายความร้อน แต่เสี่ยงต่อการสูญเสียน้ำอย่างรวดเร็ว
2. VPD ต่ำ (Low VPD): หมายความว่าอากาศมีความชื้นสูง (Air is Humid) ความแตกต่างของความดันไอน้ำมีน้อย ทำให้การคายน้ำช้าลง ซึ่งเป็นสภาวะที่เหมาะสมต่อการเจริญเติบโตของทุเรียนในแง่ของการลดความเครียดจากน้ำ

การจัดการเชิงปฏิบัติ: การเฝ้าระวังค่า VPD ช่วยให้ผู้ปฏิบัติงานสามารถคาดการณ์การสูญเสียน้ำของพืชล่วงหน้า หากค่า VPD สูงอย่างต่อเนื่อง ควรพิจารณาการให้น้ำแบบสม่ำเสมอเพื่อรักษาศักยภาพน้ำในดินให้สูงพอที่จะรองรับอัตราการคายน้ำที่สูงนั้นได้

5.1.6 5. ตัวอย่างการคำนวณและการจำลองทางคอมพิวเตอร์ (Computational Modeling Example)

เพื่อสาธิตการประยุกต์ใช้หลักการทางฟิสิกส์เกษตร เราจะเขียนโค้ด Python เพื่อคำนวณค่า VPD และประมาณการอัตราการคายน้ำ (Transpiration Rate) โดยใช้สมการที่อ้างอิงจากหลักการทางฟิสิกส์

วัตถุประสงค์: คำนวณ P_{sat} และ VPD จากอุณหภูมิและความชื้นสัมพัทธ์ที่วัดได้

สมการที่ใช้ (Clausius-Clapeyron Equation Approximation): เราจะใช้สมการที่ประมาณค่าความดันไอน้ำอิ่มตัว (P_{sat}) จากอุณหภูมิ (T) โดยใช้สูตรของ Magnus:

$$P_{sat} = 6.112 \cdot \exp\left(\frac{17.67 \cdot T}{237.3 + T}\right)$$

โดยที่ P_{sat} มีหน่วยเป็น kPa และ T มีหน่วยเป็น $^{\circ}\text{C}$
โค้ด Python สำหรับการคำนวณ VPD:

```
import numpy as np
import math

# -----
#  (Saturation Vapor Pressure: P_sat)
#  Magnus (Magnus' Formula)
#  P_sat  kPa, T  Celsius
#  -----

def calculate_psat(temperature_celsius: float) -> float:
    """
    (P_sat)
    :param temperature_celsius:  ( $^{\circ}\text{C}$ )
    :return:  (kPa)
    """
    #  (Constants)
    A = 17.67 #  Magnus
    B = 237.3 #  Magnus

    #  P_sat :  $P_{sat} = 6.112 * \exp((A * T) / (B + T))$ 
    psat = 6.112 * np.exp((A * temperature_celsius) / (B + temperature_celsius))

    return psat

# -----
#  (Vapor Pressure Deficit: VPD)
#  -----

def calculate_vpd(temperature_celsius: float, relative_humidity_percent: float)
    -> float:
    """
    VPD (kPa)
    """

    psat = calculate_psat(temperature_celsius)
    pv = psat * (relative_humidity_percent / 100.0)
    vpd = psat - pv
    return vpd

# 
temp = 32.5 #  ()
rh = 65.0 #  (%)
vpd_val = calculate_vpd(temp, rh)
print(f" VPD {vpd_val:.4f} kPa")
```

5.2 1.2 พื้นฐาน Python Sandbox สำหรับนักกร เยาวชน (Introduction to Python Program- ming & Variables)

Chapter 6

ภาควิชาวิศวกรรมศาสตร์และ
เทคโนโลยีเกษตรอัจฉริยะ

Chapter 7

สาขาวิชาปัญญาประดิษฐ์และ
การเกษตรดิจิทัล

Chapter 8

ระดับปริญญาตรี/บัณฑิตศึกษา

8.1 บทที่ 1.2 พื้นฐานภาษาไพธอน (Python) สำหรับ นวัตกรเยาวชน: การจัดการข้อมูลและตรรกะการ เขียนโปรแกรม

(Introduction to Python Programming & Variables for Young Innovators)

โดย รศ.ดร. [ชื่อสมมติของศาสตราจารย์] ผู้เชี่ยวชาญด้านฟิสิกส์ประยุกต์ (Applied Physics), เกษตรแม่นยำ (Precision Agriculture), และปัญญาประดิษฐ์ (Artificial Intelligence)

8.1.1 คำนำเชิงวิชาการ (Academic Preface)

เรียน ท่านนักนวัตกรเยาวชนผู้เปี่ยมด้วยศักยภาพทุกท่าน

ในยุคแห่งการปฏิวัติอุตสาหกรรมครั้งที่สี่ (Industry 4.0) การบูรณาการศาสตร์แขนงต่างๆ ไม่ว่าจะเป็นฟิสิกส์ (Physics), วิศวกรรมศาสตร์ (Engineering), และวิทยาการคอมพิวเตอร์ (Computer Science) ได้กลายเป็นหัวใจสำคัญของการสร้างสรรค์นวัตกรรม การเกษตรแม่นยำ (Precision Agriculture) ที่เรากำลังศึกษาอยู่นี้ ไม่ได้อาศัยเพียงแค่ความ

รู้ทางชีววิทยาเท่านั้น แต่ยังต้องอาศัยเครื่องมือทางคณิตศาสตร์และภาษาโปรแกรมมิ่ง (Programming Language) ที่ทรงประสิทธิภาพ

ภาษาไพธอน (Python) ได้รับการยอมรับในระดับสากลว่าเป็นภาษาโปรแกรมมิ่งที่เข้าถึงได้ง่าย (User-friendly) มีไวยากรณ์ (Syntax) ที่สะอาดตา และมีไลบรารี (Library) จำนวนมหาศาลที่รองรับงานวิเคราะห์ข้อมูล (Data Analysis) และการประมวลผลสัญญาณ (Signal Processing) ได้อย่างยอดเยี่ยม บทเรียนนี้จึงถูกออกแบบมาเพื่อเป็นรากฐานที่มั่นคงในการทำความเข้าใจหลักการคิดเชิงคำนวณ (Computational Thinking) ซึ่งเป็นทักษะที่สำคัญยิ่งกว่าการจดจำคำสั่งใดๆ

ขอให้ทุกท่านจงทำความเข้าใจหลักการพื้นฐานเหล่านี้อย่างลึกซึ้ง เพราะความเข้าใจในโครงสร้างข้อมูล (Data Structures) และตรรกะควบคุม (Control Flow) คือกุญแจสำคัญในการเปลี่ยนแนวคิดทางวิทยาศาสตร์ให้กลายเป็นระบบอัตโนมัติ (Automated System) ที่ใช้งานได้จริง

8.1.2 1.2.1 สภาพแวดล้อมการเขียนโปรแกรมไพธอน (Python Environment and Sandbox)

ก่อนที่จะเริ่มเขียนโค้ดใดๆ สิ่งแรกที่นักนวัตกรรมต้องทำความเข้าใจคือ "สภาพแวดล้อม" (Environment) ที่เรากำลังทำงานอยู่

1. การติดตั้งและการทำงานของ Python: ภาษาไพธอนเป็นภาษาโปรแกรมมิ่งระดับสูง (High-level Programming Language) ที่ถูกออกแบบมาให้มีไวยากรณ์ที่ใกล้เคียงกับภาษาอังกฤษ ทำให้มนุษย์สามารถอ่านและทำความเข้าใจโครงสร้างของโค้ดได้ง่าย

2. Python Sandbox (สภาพแวดล้อมจำลอง): สำหรับผู้เริ่มต้น เราจะใช้สิ่งที่เรียกว่า Sandbox ซึ่งคือพื้นที่จำลอง (Simulated Area) ที่ช่วยให้เราสามารถรันโค้ด (Execute Code) และทดสอบผลลัพธ์ (Test Output) ได้ทันทีโดยไม่ต้องติดตั้งระบบปฏิบัติการที่ซับซ้อน การทำงานใน Sandbox นี้จะช่วยให้เราสามารถมุ่งเน้นไปที่ตรรกะ (Logic)

ของโปรแกรมได้อย่างเต็มที่

หลักการทำงาน: เมื่อเราพิมพ์โค้ด Python ลงใน Sandbox แล้วกดปุ่มรัน (Run) ระบบจะทำการแปลภาษา (Interpret) โค้ดเหล่านั้นทีละบรรทัด และส่งผลลัพธ์ (Output) ออกมาให้เราเห็นทันที

8.1.3 1.2.2 ชนิดข้อมูลพื้นฐานและโครงสร้างข้อมูล (Data Types and Data Structures)

ในทางฟิสิกส์หรือเกษตรกรรม ทุกค่าที่เราวัดได้ ไม่ว่าจะเป็นอุณหภูมิ ความชื้น หรือปริมาณแสง ล้วนเป็นข้อมูล (Data) ที่ต้องถูกจัดเก็บในรูปแบบที่คอมพิวเตอร์เข้าใจ ซึ่งเราเรียกสิ่งเหล่านี้ว่า ชนิดข้อมูล (Data Types)

2.2.1 ชนิดข้อมูลพื้นฐาน (Primitive Data Types)

ตัวอย่างการกำหนดค่า (Variable Assignment): การกำหนดค่าใน Python ใช้เครื่องหมายเท่ากับ (=)

```
# ตัวอย่างการกำหนดค่า (Variable Assignment)
soil_moisture_level = 0.35 # float: 0.35 (fraction)
critical_threshold = 0.20 # float: 0.20
is_warning_active = True # bool: True
orchard_name = "สวนส้มเขียวหวาน" # str: สวนส้มเขียวหวาน
```

2.2.2 โครงสร้างข้อมูลแบบลำดับ (Collection Data Structures)

เมื่อเราต้องการเก็บข้อมูลหลายค่าที่เกี่ยวข้องกัน เราจะไม่ใช้ตัวแปรเดี่ยวๆ แต่จะใช้โครงสร้างข้อมูลเหล่านี้:

1. List (รายการ):

- คำอธิบาย: คือกลุ่มของข้อมูลที่สามารถเก็บชนิดข้อมูลที่แตกต่างกันได้หลายค่า และสามารถแก้ไขลำดับ (Ordered) ได้

ชนิดข้อมูล (Type)	คำอธิบาย (Description)	ตัวอย่าง (Example)	การใช้งานทางฟิสิกส์/เกษตรกรรม
Integer (<code>int</code>)	จำนวนเต็มที่ไม่มีส่วนทศนิยม (Whole numbers: ..., -2, -1, 0, 1, 2, ...)	10, -50	จำนวนครั้งของการวัด, จำนวนวัน
Floating Point (<code>float</code>)	จำนวนจริงที่มีส่วนทศนิยม (Real numbers)	25.5, -0.8	ค่าความชื้นดิน (Soil Moisture), อุณหภูมิ (°C), ความเข้มแสง (W/m ²)
String (<code>str</code>)	ข้อความตัวอักษร (Sequence of characters)	"□□□□", "Warning!"	ชื่อพืช, คำเตือน, คำอธิบายผลลัพธ์
Boolean (<code>bool</code>)	ค่าความจริง มีเพียงสองสถานะเท่านั้น (True หรือ False)	True, False	การตรวจสอบเงื่อนไข (เช่น: ความชื้นต่ำกว่าเกณฑ์หรือไม่?)

- การใช้งาน: เหมาะสำหรับการเก็บข้อมูลการวัดที่เกิดขึ้นตามลำดับเวลา (Time Series Data) เช่น ค่าความชื้นดิน 5 ค่าที่วัดได้ใน 5 ชั่วโมง

```
# ตัวอย่าง List: 5 ค่า
soil_moisture_readings = [0.45, 0.38, 0.25, 0.19, 0.15]
```

2. Dictionary (พจนานุกรม):

- คำอธิบาย: คือการจัดเก็บข้อมูลในรูปแบบของคู่ คีย์:ค่า (Key:Value) ซึ่งทำให้การเรียกใช้ข้อมูลมีความรวดเร็วและเข้าใจง่ายกว่าการใช้ Index (ลำดับ)
- การใช้งาน: เหมาะสำหรับการเก็บข้อมูลที่ระบุคุณสมบัติเฉพาะของระบบ เช่น การระบุค่าความชื้นของเซนเซอร์แต่ละตัว

```
# ตัวอย่าง Dictionary: 3 คู่คีย์-ค่า
sensor_data = {
    "moisture_sensor_A": 0.35, # Key: "moisture_sensor_A" Value: 0.35
    "temperature_sensor_B": 28.5, # Key: "temperature_sensor_B" Value: 28.5
    "humidity_sensor_C": 75.0 # Key: "humidity_sensor_C" Value: 75.0
}
```

8.1.4 1.2.3 โครงสร้างควบคุมทิศทางโปรแกรม (Control Flow Structures)

หากตัวแปรคือ "ข้อมูล" (Data) โครงสร้างควบคุมทิศทางโปรแกรมก็คือ "ตรรกะ" (Logic) ที่กำหนดว่าโปรแกรมจะตัดสินใจทำอะไรต่อไป

3.2.1 โครงสร้างเงื่อนไข (Conditional Statements): if-elif-else

โครงสร้างนี้ใช้ในการตัดสินใจ (Decision Making)ว่าจะดำเนินการชุดคำสั่งใดชุดคำสั่งหนึ่งหรือไม่ โดยอ้างอิงจากเงื่อนไขทางตรรกะ (Boolean Condition)

หลักการ:

1. **if:** ตรวจสอบเงื่อนไขแรก หากเป็นจริง (True) ให้ทำชุดคำสั่งนี้

2. **elif** (Else If): หากเงื่อนไขแรกเป็นเท็จ (False) ให้ตรวจสอบเงื่อนไขที่สอง หากเป็นจริง ให้ทำชุดคำสั่งนี้
3. **else**: หากทุกเงื่อนไขข้างต้นเป็นเท็จทั้งหมด ให้ทำชุดคำสั่งสุดท้ายนี้

ตัวอย่างทางคณิตศาสตร์ (Mathematical Analogy): สมมติให้ M คือระดับความชื้นดิน และ T_{crit} คือเกณฑ์วิกฤต

IF $M < T_{crit}$ THEN Warning = High

ELSE IF $M < T_{crit} + 0.1$ THEN Warning = Medium

ELSE Warning = Normal

3.2.2 การวนลูป (Loops): **for** และ **while**

การวนลูปใช้เพื่อทำซ้ำชุดคำสั่งเดิมๆ หลายครั้ง โดยไม่ต้องเขียนโค้ดซ้ำๆ ซึ่งช่วยให้โปรแกรมมีประสิทธิภาพสูง

1. **for** Loop (การวนซ้ำตามจำนวนที่กำหนด):

- หลักการ: ใช้เมื่อเรารู้จำนวนรอบที่ต้องการทำซ้ำ หรือต้องการวนผ่านสมาชิกทั้งหมดในโครงสร้างข้อมูล (เช่น List หรือ Dictionary)
- ไวยากรณ์: `for item in sequence:`

2. **while** Loop (การวนซ้ำตามเงื่อนไข):

- หลักการ: ใช้เมื่อเราต้องการให้โปรแกรมทำงานซ้ำไปเรื่อยๆ ตราบใดที่เงื่อนไขที่กำหนดยังคงเป็นจริง (True)
- ข้อควรระวัง: ต้องแน่ใจว่ามีกลไกบางอย่างที่ทำให้เงื่อนไขกลายเป็นเท็จในที่สุด มิฉะนั้นโปรแกรมจะเกิด Infinite Loop (การวนลูปไม่สิ้นสุด)

8.1.5 1.2.4 การประยุกต์ใช้: การจำลองระบบตรวจจับสถานะวิกฤตความชื้นดิน (Soil Moisture Crisis Detection Simulation)

ในภาคปฏิบัติ เราจะจำลองสถานการณ์ที่ระบบเกษตรอัจฉริยะ (Smart Farming System) ทำการอ่านค่าความชื้นดินของแปลงทุเรียน และตัดสินใจส่งคำเตือนภัย (Warning Alert) หากค่าความชื้นต่ำกว่าเกณฑ์ที่กำหนด

สมมติฐานทางฟิสิกส์และเกษตรกรรม:

1. เกณฑ์วิกฤต (Critical Threshold): ความชื้นดินที่ต่ำกว่า 0.20 (หน่วยเป็นเศษส่วนของปริมาตร) ถือเป็นสถานะวิกฤต (Critical State)
2. การวัดค่า: เราจะจำลองการวัดค่าความชื้นดิน 5 ครั้งติดต่อกัน (5 Readings)
3. การตอบสนอง: ระบบต้องตรวจสอบค่าที่วัดได้แต่ละค่า และแจ้งเตือนสถานะที่เหมาะสม

☐ โค้ด Python สำหรับการจำลองระบบ (Python Source Code)

```
# -*- coding: utf-8 -*-
# -----
# 
# .. [?]
```

```
# -----

# 1. (Defining Constants)
CRITICAL_MOISTURE_THRESHOLD = 0.20 # (float)
MIN_MOISTURE_THRESHOLD = 0.10 # (float)
ORCHARD_NAME = " " # (str)

# 2. (Simulated Data)
# List: 5 (fraction)
soil_moisture_readings = [0.45, 0.38, 0.25, 0.19, 0.15]

def analyze_soil_moisture(readings: list) -> str:
    """
    :param readings: 
    :return: 
    """
    print("=====")
```

```
print(f"   : {ORCHARD_NAME}")
print("=====")

# 3.   (Using 'for' loop)
print("\n---   ---")
for i, moisture_value in enumerate(readings):
    #   (i+1)
    print(f"   {i+1}:   = {moisture_value:.2f} (fraction)")

# 4.   (Using 'if-elif-else')
if moisture_value < CRITICAL_MOISTURE_THRESHOLD:
    #   :
    print(f"   :   !   ")
    print(f"   (Action:   ,   )")

elif moisture_value < MIN_MOISTURE_THRESHOLD:
    #   :
    print(f"   :   ")

else:
    #   :
    print(f"   :   ")

return "   "

#
status_summary = analyze_soil_moisture(soil_moisture_readings)
print("\n=====")
print(f"   : {status_summary}")
print("=====")
```

8.2 1.3 การดึงข้อมูลสภาพอากาศ TMD Open Data API และโครงสร้าง JSON

Chapter 9

1.3 การดึงข้อมูลสภาพอากาศ จาก TMD Open Data API และโครงสร้าง JSON

(บทที่ 1.3: การบูรณาการข้อมูลสภาพอากาศแบบเรียลไทม์ด้วย Web API และการวิเคราะห์โครงสร้างข้อมูล JSON)

9.1 1.3.1 แนวคิดพื้นฐานของคลังข้อมูลเปิด (Open Data Repository) และ Web API

ในยุคของการปฏิวัติอุตสาหกรรมครั้งที่สี่ (Industry 4.0) ข้อมูล (Data) ได้ถูกยกสถานะให้เป็นทรัพยากรที่มีมูลค่าสูงที่สุด (Most Valuable Resource) การเข้าถึงข้อมูลที่ถูกต้อง แม่นยำ และทันเวลาจึงเป็นปัจจัยสำคัญในการขับเคลื่อนระบบเกษตรอัจฉริยะ (Smart Agriculture) และการตัดสินใจทางวิศวกรรม (Engineering Decision-Making)

9.1.1 1.3.1.1 คลังข้อมูลเปิด (Open Data Repository)

คลังข้อมูลเปิด (Open Data Repository) คือ แหล่งรวบรวมข้อมูลที่ถูกเผยแพร่สู่สาธารณะโดยหน่วยงานภาครัฐหรือองค์กรขนาดใหญ่ โดยมีเงื่อนไขที่เปิดให้บุคคลทั่วไปสามารถเข้าถึง นำไปใช้ประโยชน์ และนำไปต่อยอดเชิงพาณิชย์ได้โดยไม่มีข้อจำกัดด้านลิขสิทธิ์ที่เข้มงวด (Non-restrictive Licensing)

ในบริบทของอุตุนิยมวิทยา (Meteorology) การที่กรมอุตุนิยมวิทยา (Thai Meteorological Department: TMD) เปิดเผยข้อมูลผ่านช่องทาง Open Data ถือเป็นการสร้างรากฐานที่สำคัญให้นักวิจัยและผู้ประกอบการสามารถนำข้อมูลสภาพอากาศมาใช้ในการสร้างแบบจำลองทางฟิสิกส์ (Physical Modeling) เช่น การพยากรณ์ผลผลิตทางการเกษตร (Crop Yield Forecasting) หรือการบริหารจัดการทรัพยากรน้ำ (Water Resource Management)

9.1.2 1.3.1.2 สถาปัตยกรรม Web API (Web Application Programming Interface)

การเข้าถึงข้อมูลจากคลังข้อมูลขนาดใหญ่มักไม่ได้ทำโดยการดาวน์โหลดไฟล์ทั้งหมด (เช่น ไฟล์ CSV หรือ XML ขนาดใหญ่) แต่จะใช้กลไกที่เรียกว่า Web API (Web Application Programming Interface)

Web API คือ ชุดของกฎเกณฑ์ (Set of Rules) และโปรโตคอล (Protocol) ที่กำหนดวิธีการที่แอปพลิเคชันซอฟต์แวร์หนึ่งจะสามารถสื่อสารและแลกเปลี่ยนข้อมูลกับแอปพลิเคชันซอฟต์แวร์อื่นได้โดยไม่ต้องทราบรายละเอียดการทำงานภายใน (Internal Mechanism) ของระบบต้นทาง

สำหรับ TMD Open Data API นั้น ทำหน้าที่เป็น ตัวกลางในการแปลงข้อมูล (Data Translator) โดยผู้ใช้งาน (Client) จะส่งคำขอ (Request) ไปยัง API Endpoint และ API จะรับผิดชอบในการประมวลผล

คำขอเหล่านั้น จากนั้นจึงส่งข้อมูลที่ต้องการกลับมาในรูปแบบมาตรฐาน (Standardized Format) ซึ่งโดยทั่วไปคือ JSON (JavaScript Object Notation)

9.2 1.3.2 กลไกการร้องขอข้อมูลผ่าน API (API Request Mechanism)

การสื่อสารระหว่างไคลเอนต์ (Client) และเซิร์ฟเวอร์ (Server) ผ่าน Web API อาศัยโปรโตคอล HTTP (Hypertext Transfer Protocol) เป็นหลัก

9.2.1 1.3.2.1 องค์ประกอบของการร้องขอ (Request Components)

การร้องขอข้อมูล API โดยทั่วไปประกอบด้วยองค์ประกอบสำคัญ 3 ส่วน:

1. API Endpoint URL: คือที่อยู่เฉพาะเจาะจงที่ระบุว่าการขอข้อมูลประเภทใด (เช่น ข้อมูลสภาพอากาศปัจจุบัน, ข้อมูลย้อนหลัง)
2. HTTP Method: คือวิธีการที่ไคลเอนต์ต้องการใช้ในการกระทำกับทรัพยากร (Resource) นั้น ๆ
3. Parameters/Headers: คือข้อมูลเพิ่มเติมที่จำเป็นในการระบุตัวตนและการร้องขอข้อมูล (เช่น API Key, สถานีตรวจวัด, วันที่)

9.2.2 1.3.2.2 การเปรียบเทียบ HTTP GET และ HTTP POST
การเลือกใช้ HTTP Method มีความสำคัญอย่างยิ่งต่อความสมบูรณ์ของข้อมูลและการทำงานของระบบ:

1. HTTP GET Method

- วัตถุประสงค์: ใช้สำหรับ การเรียกดู (Retrieving) ข้อมูลจากเซิร์ฟเวอร์ เท่านั้น (Read-Only Operation)
- การส่งข้อมูล: ข้อมูลที่จำเป็นทั้งหมดจะถูกแนบไปกับ URL ในรูปแบบของ Query Parameters (เช่น ?station_id=KhaoSaming&date=2024-05-15)
- ข้อดี: ง่ายต่อการทำ Caching และเหมาะสำหรับการดึงข้อมูลที่ไม่เปลี่ยนแปลงหรือข้อมูลสถานะปัจจุบัน
- การประยุกต์ใช้ใน TMD API: เหมาะที่สุดสำหรับการดึงข้อมูล สภาพอากาศปัจจุบัน (Current Weather Data)

2. HTTP POST Method

- วัตถุประสงค์: ใช้สำหรับ การส่งข้อมูล (Submitting) หรือการสร้างทรัพยากรใหม่บนเซิร์ฟเวอร์ (Create Operation)
- การส่งข้อมูล: ข้อมูลจะถูกส่งไปในส่วนของ Body ของคำขอ (Request Body) ซึ่งมีความปลอดภัยกว่าการส่งใน URL หากข้อมูลมีความยาวหรือเป็นข้อมูลที่ละเอียดอ่อน
- การประยุกต์ใช้: มักใช้เมื่อผู้ใช้งานต้องการส่งข้อมูลการพยากรณ์ที่ซับซ้อน หรือต้องการบันทึกข้อมูลใหม่เข้าสู่ระบบ

สรุป: สำหรับการดึงข้อมูลสภาพอากาศปัจจุบันจาก TMD Open Data API ซึ่งเป็นเพียงการ "อ่าน" ข้อมูล จึงควรใช้ HTTP GET Method

9.2.3 1.3.2.3 การใช้ API Key (Authentication)

API Key คือรหัสประจำตัวที่ออกให้กับผู้ใช้งานแต่ละราย ทำหน้าที่เป็นกลไกการพิสูจน์ตัวตน (Authentication) และการจำกัดอัตราการร้องขอ (Rate Limiting)

เมื่อมีการร้องขอข้อมูล API Key จะถูกส่งไปในส่วนของ Header หรือเป็น Query Parameter เพื่อให้เซิร์ฟเวอร์สามารถตรวจสอบได้ว่าผู้ร้องขอมีสถานะในการเข้าถึงข้อมูลหรือไม่ และเพื่อป้องกันการใช้งานเกินขีดจำกัดที่กำหนดไว้ (Quota Exceeded Error)

9.3 1.3.3 โครงสร้างข้อมูล JSON (JavaScript Object Notation)

เมื่อ API ประมวลผลคำขอสำเร็จ ข้อมูลที่ส่งกลับมาจะอยู่ในรูปแบบ JSON (JavaScript Object Notation) ซึ่งเป็นรูปแบบการแลกเปลี่ยนข้อมูล (Data Interchange Format) ที่ได้รับความนิยมสูงสุดในปัจจุบัน เนื่องจากมีโครงสร้างที่อ่านง่ายสำหรับมนุษย์ (Human-readable) และง่ายต่อการประมวลผลด้วยภาษาโปรแกรมส่วนใหญ่

9.3.1 1.3.3.1 โครงสร้างพื้นฐานของ JSON

JSON มีโครงสร้างหลักที่เรียบง่าย ประกอบด้วยคู่ของ ชื่อ (Key) และ ค่า (Value) (Key-Value Pair)

1. Object (วัตถุ): คือการรวมกลุ่มของคู่ Key-Value ซึ่งถูกล้อมรอบด้วยวงเล็บปีกกา {}
2. ตัวอย่าง: {"temperature": 30.5, "humidity": 75}
3. Array (อาร์เรย์): คือรายการของค่าที่เรียงลำดับกัน ซึ่งถูกล้อมรอบด้วยวงเล็บเหลี่ยม []
4. ตัวอย่าง: ["2024-05-15", "2024-05-16", "2024-05-17"]

9.3.2 1.3.3.2 การวิเคราะห์พารามิเตอร์สภาพอากาศใน JSON

ข้อมูลสภาพอากาศที่ได้จาก TMD API มักจะถูกจัดโครงสร้างเป็น Object ที่ซับซ้อน ซึ่งภายใน Object นั้นอาจมี Array ของข้อมูลย่อย (Nested Array)

พารามิเตอร์หลักที่มักถูกดึงมาประกอบด้วย:

พารามิเตอร์ (Key)	คำอธิบายทาง ฟิสิกส์	หน่วย SI (SI Unit)	ประเภทข้อมูล (Data Type)
temperature	อุณหภูมิอากาศ ณ เวลาที่วัด	เคลวิน (K) หรือ องศาเซลเซียส (°C)	Floating Point Number
humidity	ความชื้นสัมพัทธ์ ของอากาศ	เปอร์เซ็นต์ (%)	Floating Point Number
wind_speed	ความเร็วลม	เมตรต่อวินาที (m/s)	Floating Point Number
rainfall	ปริมาณน้ำฝน สะสมในช่วง เวลาที่กำหนด	มิลลิเมตร (mm)	Floating Point Number
timestamp	เวลาที่บันทึก ข้อมูล	ISO 8601 Format (String)	String

9.3.3 1.3.3.3 กระบวนการแปลงค่า (JSON Parsing)

JSON Parsing คือกระบวนการที่โปรแกรมคอมพิวเตอร์อ่านข้อความ (String) ที่อยู่ในรูปแบบ JSON และแปลงโครงสร้างนั้นให้กลายเป็นโครงสร้างข้อมูลที่ภาษาโปรแกรมสามารถนำไปใช้งานทางคณิตศาสตร์หรือตรรกะได้ (เช่น การแปลง String เป็น Dictionary หรือ Object ใน Python)

ในทางปฏิบัติ การใช้ไลบรารีเฉพาะทาง (เช่น `json` ใน Python) จะ

ช่วยให้เราสามารถจัดการกับความซับซ้อนของโครงสร้างข้อมูลได้อย่างมีประสิทธิภาพ

9.4 1.3.4 การประยุกต์ใช้เชิงปฏิบัติ: การดึงข้อมูลสภาพอากาศด้วย Python

ในส่วนนี้ เราจะสาธิตการเขียนโค้ด Python ที่มีความเสถียรสูง (Robust Code) เพื่อทำการร้องขอข้อมูลสภาพอากาศปัจจุบันของสถานีอุตุนิยมวิทยาเขาสมิง จังหวัดตราด โดยใช้ไลบรารีมาตรฐานของ Python คือ `urllib.request` สำหรับการสื่อสารเครือข่าย และ `json` สำหรับการจัดการโครงสร้างข้อมูล

สมมติฐาน:

1. เราได้ทำการลงทะเบียนและได้รับ `API_KEY` จาก TMD Open Data Portal แล้ว
2. เราทราบ Endpoint URL สำหรับข้อมูลสภาพอากาศปัจจุบัน

9.4.1 1.3.4.1 โค้ด Python สำหรับการดึงและวิเคราะห์ข้อมูล

```
import urllib.request
import urllib.parse
import json
import datetime

# =====
# 1. Configuration Parameters
# =====

# *** TODO: Replace API Key with your own ***
API_KEY = "YOUR_TMD_API_KEY_HERE"

# Endpoint URL (API)
BASE_URL = "https://api.example.com/weather/current"

# TODO: Replace with your station ID
STATION_ID = "KhaoSaming"
```

```
# =====
# 2. Core Data Retrieval Function
# =====

def fetch_weather_data(api_key: str, station_id: str, base_url: str) -> dict or None:
    """
    TMD Open Data API

    Args:
        api_key (str): Authentication Key.
        station_id (str): Station Identifier.
        base_url (str): URL API Endpoint.

    Returns:
        dict or None: Dictionary Parse None

    """
    print("="*70)
    print(f"[{datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S')}] ")
    print(" ")

    # 2.1 Query Parameters
    # urllib.parse.urlencode Encode URL
    params = {
        "api_key": api_key,
        "station": station_id,
        "data_type": "current" #
    }

    # Query String ( ?station=KhaoSaming&api_key=...)
    query_string = urllib.parse.urlencode(params)

    # 2.2 URL
    full_url = f"{base_url}?{query_string}"
    print(f"-> URL : {full_url}")

    try:
        # 2.3 HTTP GET Request
        # urllib.request.urlopen HTTP GET
        with urllib.request.urlopen(full_url) as response:
            # HTTP Status Code
            status_code = response.getcode()
            if status_code != 200:
                print(f"\n[ERROR] ! : {status_code}")
                print("API Key Endpoint URL ")
                return None

            # (bytes)
            data_bytes = response.read()

            # 2.4 JSON (Decoding and Parsing)
            # 1. bytes string (UTF-8)
            json_string = data_bytes.decode('utf-8')

            # 2. JSON string Python dictionary
            weather_data = json.loads(json_string)
```

```

        print("\n[SUCCESS] ข้อมูลได้รับเรียบร้อยแล้ว!")
        return weather_data

    except urllib.error.URLError as e:
        # ข้อมูลไม่สามารถดาวน์โหลดได้ (Network Error)
        print(f"\n[FATAL ERROR] ข้อมูลไม่สามารถดาวน์โหลดได้: {e.reason}")
        return None
    except json.JSONDecodeError:
        # ข้อมูลที่ดาวน์โหลดได้แต่ไม่สามารถแปลงเป็น JSON ได้
        print("\n[ERROR] ข้อมูลที่ดาวน์โหลดได้แต่ไม่สามารถแปลงเป็น JSON: ข้อมูลที่ดาวน์โหลดได้")
        print("JSON ข้อมูล")

    return None
except Exception as e:
    # ข้อมูลไม่สามารถโหลดได้
    print(f"\n[CRITICAL ERROR] ข้อมูลไม่สามารถโหลดได้: {e}")
    return None

# =====
# 3. ข้อมูลที่ได้รับมาวิเคราะห์และแสดงผล (Data Analysis and Display)
# =====

def display_weather_report(data: dict):
    """
    3. ข้อมูลที่ได้รับมาวิเคราะห์และแสดงผล (Data Analysis and Display)

    Args:
        data (dict): Dictionary ข้อมูลที่ได้รับมา

    """
    if not data:
        print("\nข้อมูลที่ได้รับมาไม่ถูกต้อง")
        return

    print("\n" + "="*70)
    print("  ข้อมูลที่ได้รับมา (Current Weather Report)  ")
    print("="*70)

    # ข้อมูลที่ได้รับมา dictionary
    main = data.get("main", {})
    wind = data.get("wind", {})
    weather = data.get("weather", [{}])[0]

    print(f"  ชื่อสถานที่: {data.get('name', '')}")
    print(f"  สภาพอากาศ: {weather.get('description', '')}")
    print(f"  อุณหภูมิ: {main.get('temp', 0) - 273.15:.2f} °C")
    print(f"  รู้สึกเหมือน: {main.get('feels_like', 0) - 273.15:.2f} °C")
    print(f"  ความชื้นสัมพัทธ์: {main.get('humidity', 0)}%")
    print(f"  ความเร็วลม: {wind.get('speed', 0)} m/s")
    print("="*70)

    # ข้อมูลที่ได้รับมา dictionary
    mock_data = {
        "name": "Chanthaburi",
        "main": {
            "temp": 305.15,      # 32 °C
            "feels_like": 308.15, # 35 °C
            "humidity": 75
        },
    },

```



```
"wind": {  
    "speed": 3.6  
},  
"weather": [  
    {"description": "☀️☀️☀️☀️ (Clear sky)"}  
]  
}  
  
display_weather_report(mock_data)
```

9.5 1.4 การวิเคราะห์อนุกรมเวลาสภาพอากาศด้วย Pandas (Weather Time Series Analysis with Pandas & Seaborn)

Chapter 10

ภาควิชาวิทยาศาสตร์
การเกษตรดิจิทัลและฟิสิกส์
ประยุกต์

Chapter 11

หัวข้อ 1.4 การวิเคราะห์ อนุกรมเวลาสภาพอากาศด้วย Pandas (Weather Time Series Analysis with Pandas & Seaborn)

—

11.1 บทนำ: ความสำคัญของการวิเคราะห์อนุกรมเวลา ในอุตุนิยมวิทยาการเกษตร

(Introduction: Importance of Time Series Analysis in Agro-Meteorology)

ในบริบทของเกษตรกรรมยุคดิจิทัล (Digital Agriculture) ข้อมูลสภาพอากาศถือเป็นตัวแปรนำเข้า (Input Variable) ที่มีความสำคัญสูงสุดต่อการวางแผนการเพาะปลูก การจัดการน้ำ และการคาดการณ์ผลผลิต การทำความเข้าใจถึงการเปลี่ยนแปลงของสภาพอากาศในมิติของเวลาจึงเป็นหัวใจสำคัญของการเกษตรแม่นยำ (Precision Agriculture)

อนุกรมเวลา (Time Series Data) คือชุดของข้อมูลที่ถูกเก็บรวบรวมตามลำดับเวลาอย่างต่อเนื่อง (Sequential Data over Time) ในทาง

อุตุนิยมวิทยาการเกษตร (Agro-Meteorology) ข้อมูลอนุกรมเวลาอาจประกอบด้วยตัวแปรต่างๆ เช่น อุณหภูมิเฉลี่ยรายวัน (Daily Mean Temperature), ปริมาณน้ำฝนสะสม (Cumulative Rainfall), ความชื้นสัมพัทธ์ (Relative Humidity), หรือความเร็วลม (Wind Speed) ซึ่งข้อมูลเหล่านี้ไม่ได้เป็นเพียงค่า ณ จุดเวลาใดเวลาหนึ่ง แต่เป็นแนวโน้ม (Trend) และความผันผวน (Variability) ที่สะท้อนถึงสภาวะแวดล้อมที่เปลี่ยนแปลงไปตามกาลเวลา

การวิเคราะห์อนุกรมเวลาจึงมิใช่เพียงการดูค่าเฉลี่ย แต่คือการค้นหา รูปแบบ (Pattern) ฤดูกาล (Seasonality) แนวโน้มระยะยาว (Long-term Trend) และความสัมพันธ์เชิงสาเหตุ (Causality) ระหว่างตัวแปร สภาพอากาศที่ส่งผลกระทบต่อวงจรชีวิตของพืช (Crop Life Cycle) อย่างเป็นระบบ

11.2 1. องค์ประกอบทางทฤษฎี: การจัดการข้อมูลอนุกรมเวลาด้วย Pandas

(Theoretical Component: Handling Time Series Data with Pandas)

ไลบรารี Pandas (Python Data Analysis Library) เป็นเครื่องมือหลักในการจัดการข้อมูลที่มีโครงสร้างแบบตาราง (Tabular Data) ในภาษา Python โดยเฉพาะอย่างยิ่งเมื่อต้องจัดการกับข้อมูลอนุกรมเวลา Pandas ได้ถูกออกแบบมาเพื่อรองรับโครงสร้างข้อมูลที่เรียกว่า DataFrame ซึ่งมีคุณสมบัติที่สำคัญอย่างยิ่งคือการรองรับ Datetime Indexing

11.2.1 1.1 การโหลดและการจัดการโครงสร้างข้อมูล (Data Loading and Structure Management)

เมื่อเราโหลดข้อมูลสภาพอากาศจากไฟล์สถิติ (เช่น CSV) สิ่งแรกที่ต้องดำเนินการคือการตรวจสอบให้แน่ใจว่าคอลัมน์ที่ระบุเวลา (Timestamp) ถูกระบุว่าเป็นรูปแบบวันที่และเวลา (Datetime Object) อย่างถูกต้อง

หลักการทางเทคนิค: การกำหนดให้คอลัมน์ใดคอลัมน์หนึ่งเป็น Index ที่เป็น Datetime Index จะช่วยให้ Pandas สามารถดำเนินการทางเวลา (Time-based Operations) ได้อย่างมีประสิทธิภาพ เช่น การเลือกข้อมูลในช่วงเวลาที่กำหนด (Slicing by Date Range) หรือการ Resampling (การปรับความถี่ของข้อมูล)

11.2.2 1.2 การจัดการข้อมูลสูญหาย (Handling Missing Values: NaN)

ในข้อมูลสภาพอากาศจริง มักเกิดข้อมูลสูญหาย (Missing Values) เนื่องจากเซ็นเซอร์ขัดข้อง หรือการบันทึกข้อมูลที่ไม่สมบูรณ์ การจัดการข้อมูลสูญหายเป็นขั้นตอนที่สำคัญยิ่งก่อนการวิเคราะห์ใดๆ

วิธีการทางสถิติที่ใช้:

1. การลบทิ้ง (Dropping): หากข้อมูลสูญหายมีปริมาณมากเกินไป หรืออยู่ในช่วงเวลาที่สำคัญจนไม่สามารถประมาณค่าได้ อาจพิจารณาลบแถวนั้นทิ้ง (Row Deletion)
2. การแทนที่ด้วยค่าก่อนหน้า/ถัดไป (Forward/Backward Filling): สำหรับข้อมูลที่คาดว่าค่าจะไม่เปลี่ยนแปลงอย่างฉับพลัน (เช่น อุณหภูมิที่คงที่ตลอดคืน) สามารถใช้เทคนิค Forward Fill (ffill) หรือ Backward Fill (bfill) เพื่อประมาณค่าที่หายไปโดยใช้ค่าที่อยู่ใกล้เคียงที่สุด

11.3 2. การวิเคราะห์แนวโน้มด้วยค่าเฉลี่ยเคลื่อนที่ (Rolling Moving Average)

(Trend Analysis using Rolling Moving Average)

เมื่อข้อมูลอนุกรมเวลาสภาพอากาศถูกบันทึกมาอย่างต่อเนื่อง ข้อมูลที่ได้มักมีความผันผวนสูง (High Volatility) เนื่องจากได้รับผลกระทบจากสภาพอากาศเฉพาะวัน (Daily Weather Fluctuation) การวิเคราะห์ค่าดิบ (Raw Data) โดยตรงอาจทำให้เกิดสัญญาณรบกวน (Noise) มากเกินไป จนทำให้การระบุแนวโน้มระยะยาว (Long-term Trend) เป็นไปได้ยาก

ค่าเฉลี่ยเคลื่อนที่ (Moving Average, MA) คือเทคนิคทางสถิติที่ใช้ในการ "ทำให้ข้อมูลเรียบ" (Smoothing) โดยการคำนวณค่าเฉลี่ยของข้อมูลในช่วงเวลาที่กำหนด (Window Size) ค่าที่ได้จากการคำนวณนี้จะถูกเลื่อนไปตามลำดับเวลา ทำให้ได้ค่าประมาณของแนวโน้มที่แท้จริงของตัวแปรนั้นๆ

11.3.1 2.1 หลักการทางคณิตศาสตร์ (Mathematical Principle)

ให้ x_t เป็นค่าของตัวแปรสภาพอากาศ ณ เวลา t และให้ w เป็นขนาดของหน้าต่าง (Window Size) ในการคำนวณค่าเฉลี่ยเคลื่อนที่ ณ เวลา t ($\bar{x}_t$) เราจะใช้ค่าเฉลี่ยของ w จุดข้อมูลที่อยู่รอบๆ เวลา t

$$\bar{x}_t = \frac{1}{w} \sum_{i=0}^{w-1} x_{t-i}$$

โดยที่:

- $\bar{x}_t$: คือค่าเฉลี่ยเคลื่อนที่ ณ เวลา t
- w : คือขนาดหน้าต่าง (Window Size) เช่น $w = 7$ หมายถึงการใช้ข้อมูล 7 วัน
- x_{t-i} : คือค่าของตัวแปร ณ เวลา $t-i$

การประยุกต์ใช้ในเกษตรกรรม: การใช้ค่าเฉลี่ยเคลื่อนที่ 7 วัน (7-day Rolling Average) สำหรับปริมาณน้ำฝน (Rainfall) หรืออุณหภูมิ (Temperature) จะช่วยให้เราสามารถลดผลกระทบจากความผันผวนรายวัน (เช่น วันที่ฝนตกหนักผิดปกติ หรือวันที่แดดจัดผิดปกติ) และสามารถมองเห็น "แนวโน้มสะสม" (Accumulated Trend) ของสภาวะแวดล้อมในช่วงสัปดาห์ที่ผ่านมา ซึ่งเป็นข้อมูลที่มีประโยชน์อย่างยิ่งในการพยากรณ์การเจริญเติบโตของพืช

11.4 3. การปฏิบัติการ: การวิเคราะห์อนุกรมเวลาด้วย Python

(Practical Implementation: Time Series Analysis with Python)

ในส่วนนี้ เราจะทำการจำลองการวิเคราะห์ข้อมูลสภาพอากาศย้อนหลัง โดยใช้ไลบรารี Pandas และ Seaborn เพื่อคำนวณและแสดงผลค่าเฉลี่ยเคลื่อนที่ 7 วัน สำหรับปริมาณน้ำฝนและอุณหภูมิ

11.4.1 3.1 การเตรียมสภาพแวดล้อมและข้อมูลจำลอง

สมมติว่าเรามีไฟล์ CSV ชื่อ `weather_data.csv` ซึ่งมีคอลัมน์หลักดังนี้:

1. `Date`: วันที่และเวลา (Timestamp)
2. `Rainfall_mm`: ปริมาณน้ำฝน (หน่วย: มิลลิเมตร, mm)
3. `Temperature_C`: อุณหภูมิเฉลี่ย (หน่วย: องศาเซลเซียส, °C)

11.4.2 3.2 โค้ด Python สำหรับการวิเคราะห์ (Python Code for Analysis)

```
# -----
#   (Importing Necessary Libraries)
# -----
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

#   (Setting the style to whitegrid)
sns.set_style("whitegrid")

# -----
#   1:   (Data Loading and Preparation)
# -----
#   (Simulating the loading of a CSV file)
#   : df = pd.read_csv('weather_data.csv')
data = {
    'Date': pd.date_range(start='2023-01-01', periods=60, freq='D'), #   60
    'Rainfall_mm': np.random.rand(60) * 10 + np.sin(np.arange(60) / 10) * 5, #   (0-10 mm)
    'Temperature_C': 25 + np.sin(np.arange(60) / 15) * 5 + np.random.randn(60) * 1.5 #   (20-30 °C)
}
df = pd.DataFrame(data)

#   Datetime Index:   'Date'   Index   DataFrame
#   : df.set_index('Date')

print("---   5   (Original Data Head) ---")
print(df.head())

# -----
#   2:   (Handling Missing Values)
# -----
#   30   31
df.loc['2023-01-30':'2023-01-31', 'Rainfall_mm'] = np.nan
df.loc['2023-02-05', 'Temperature_C'] = np.nan

print("\n---   NaN ---")
print(df.loc['2023-01-28':'2023-02-02'])

#   NaN:   Forward Fill (ffill)
#   : df_cleaned = df.copy()
#   : df_cleaned['Rainfall_mm'] = df_cleaned['Rainfall_mm'].fillna(method='ffill')
#   : df_cleaned['Temperature_C'] = df_cleaned['Temperature_C'].fillna(method='ffill')

print("\n---   Forward Fill (Cleaned Data) ---")
print(df_cleaned.loc['2023-01-28':'2023-02-02'])

# -----
#   3:   (Calculating Rolling Average)
```



```
# -----
WINDOW_SIZE = 7 # 7
# 7
# .rolling(window=W)
# .mean()
df_cleaned['Rainfall_MA_7D'] = df_cleaned['Rainfall_mm'].rolling(window=
    WINDOW_SIZE).mean()

# 7
df_cleaned['Temperature_MA_7D'] = df_cleaned['Temperature_C'].rolling(window=
    WINDOW_SIZE).mean()

print(f"\n--- Rolling Average ({WINDOW_SIZE}) ---")
#
print(df_cleaned[['Rainfall_mm', 'Rainfall_MA_7D', 'Temperature_C', '
    Temperature_MA_7D']].head(10))

# -----
# 4: (Visualization using Seaborn/Matplotlib)
# -----
# (Rainfall)
plt.figure(figsize=(14, 6))

# 1. (Raw Data)
plt.plot(df_cleaned.index, df_cleaned['Rainfall_mm'], label='(Raw Data)', color='skyblue', alpha=0.6)

# 2. (Trend Line)
# Rolling Average NaN 6 ( 7 )
# .dropna()
plt.plot(df_cleaned.index, df_cleaned['Rainfall_MA_7D'], label=f'(MA-{WINDOW_SIZE}D)', color='darkblue', linewidth=2.5)

plt.title('Weekly Rainfall Trend', fontsize=16, pad=20)
plt.xlabel('(Date)', fontsize=14)
plt.ylabel('(mm)', fontsize=14)
plt.legend(loc='upper right')
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

# (Temperature)
plt.figure(figsize=(14, 6))

# 1. (Raw Data)
plt.plot(df_cleaned.index, df_cleaned['Temperature_C'], label='(Raw Data)', color='salmon', alpha=0.6)

# 2. (Trend Line)
plt.plot(df_cleaned.index, df_cleaned['Temperature_MA_7D'], label=f'(MA-{WINDOW_SIZE}D)', color='red', linewidth=2.5)

plt.title('Weekly Temperature Trend', fontsize=16, pad=20)
```

```
plt.xlabel('□□□□(Date)', fontsize=14)
plt.ylabel('□□□□□□□□ (°C)', fontsize=14)
plt.legend(loc='upper right')
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

11.5 4. การวิเคราะห์เชิงลึกและข้อสรุปทางวิชาการ

*(In-Depth Analysis and

11.6 1.5 แบบจำลอง Drought Risk Index (DRI) Dashboard และระบบสนับสนุนการตัดสินใจ (Decision Support System)

Chapter 12

บทที่ 1.5 แบบจำลอง Drought Risk Index (DRI) Dashboard และระบบ สนับสนุนการตัดสินใจ (Decision Support System)

12.1 บทนำเชิงทฤษฎี: ระบบสนับสนุนการตัดสินใจ ในการเกษตร (Farm Decision Support Sys- tem: DSS)

ในบริบทของการเกษตรสมัยใหม่ที่ต้องเผชิญกับความผันผวนของสภาพภูมิอากาศ (Climate Variability) และความไม่แน่นอนของทรัพยากรน้ำ การตัดสินใจที่อาศัยประสบการณ์หรือการคาดเดาเพียงอย่างเดียวถือว่ามีความเสี่ยงสูง (High Risk) ดังนั้น การนำระบบสนับสนุนการตัดสินใจ (Decision Support System: DSS) มาประยุกต์ใช้จึงเป็นสิ่งจำเป็นอย่างยิ่ง

ระบบสนับสนุนการตัดสินใจในการทำฟาร์ม (Farm Decision Support System: DSS) คือ กรอบแนวคิดและชุดเครื่องมือทางคอมพิวเตอร์

ที่ออกแบบมาเพื่อช่วยให้เกษตรกรหรือผู้จัดการฟาร์มสามารถรวบรวม วิเคราะห์ และตีความข้อมูลที่ซับซ้อนจากแหล่งข้อมูลที่หลากหลาย (Heterogeneous Data Sources) เพื่อให้สามารถตัดสินใจเลือกแนวทางการปฏิบัติที่เหมาะสมที่สุด (Optimal Management Practices) ภายใต้ข้อจำกัดของทรัพยากรและสภาพแวดล้อมที่เปลี่ยนแปลงไป

12.1.1 1.5.1 องค์ประกอบหลักของ DSS ในการเกษตร

DSS ที่มีประสิทธิภาพสูงในการจัดการภัยแล้งจะต้องประกอบด้วยองค์ประกอบหลัก 3 ส่วนที่ทำงานร่วมกันอย่างเป็นระบบ:

1. ฐานข้อมูล (Database): เป็นแหล่งรวบรวมข้อมูลดิบ (Raw Data) ซึ่งรวมถึงข้อมูลอนุกรมเวลา (Time Series Data) ของสภาพอากาศ (เช่น อุณหภูมิ, ความชื้น, ปริมาณน้ำฝน), ข้อมูลดิน (Soil Moisture Content), ข้อมูลพืชผล (Crop Growth Stage), และข้อมูลทางเศรษฐศาสตร์ (Market Prices)
2. แบบจำลอง (Model): เป็นหัวใจสำคัญของระบบ ซึ่งประกอบด้วยแบบจำลองทางคณิตศาสตร์และฟิสิกส์ (Mathematical and Physical Models) ที่ใช้ในการประมวลผลข้อมูลดิบให้กลายเป็นตัวชี้วัดเชิงปริมาณ (Quantitative Indices) เช่น ดัชนีความเครียดน้ำ (Water Stress Index) หรือ ดัชนีความเสี่ยงภัยแล้ง (Drought Risk Index: DRI)
3. ส่วนติดต่อผู้ใช้ (User Interface/Dashboard): เป็นส่วนที่นำเสนอผลลัพธ์ที่ผ่านการวิเคราะห์แล้วในรูปแบบที่เข้าใจง่าย (Intuitive Visualization) พร้อมทั้งให้คำแนะนำเชิงปฏิบัติ (Actionable Recommendations) แก่ผู้ใช้งานโดยตรง

การบูรณาการองค์ประกอบเหล่านี้ทำให้ DSS ไม่ได้เป็นเพียงแค่เครื่องมือวิเคราะห์ แต่เป็นระบบที่ช่วยยกระดับการตัดสินใจจากระดับ "การ

คาดเดา” ไปสู่ระดับ ”การจัดการเชิงวิทยาศาสตร์” (Scientific Management)

12.2 1.5.2 การพัฒนาแบบจำลอง Drought Risk Index (DRI)

ภัยแล้ง (Drought) ไม่ได้หมายถึงเพียงแค่การขาดแคลนน้ำฝนเท่านั้น แต่เป็นสภาวะที่เกิดจากความไม่สมดุลระหว่างปริมาณน้ำที่ได้รับ (Precipitation Input) กับอัตราการระเหยและการคายน้ำ (Evapotranspiration Output) ซึ่งการคำนวณความเสี่ยงต้องพิจารณาปัจจัยทางกายภาพหลายมิติ

เราจึงได้พัฒนาแบบจำลอง Drought Risk Index (DRI) ซึ่งเป็นดัชนีที่คำนวณแบบถ่วงน้ำหนัก (Weighted Composite Index) โดยพิจารณาจากปัจจัยหลัก 3 ประการที่ส่งผลต่อความเครียดของพืช (Plant Stress) ในสวนทุเรียน:

1. ความเครียดจากความร้อน (Heat Stress): วัดจากความแตกต่างระหว่างอุณหภูมิสูงสุดและอุณหภูมิลดลง
2. ความเครียดจากความชื้นในอากาศ (Atmospheric Humidity Stress): วัดจากความชื้นสัมพัทธ์ในอากาศ (Relative Humidity: RH)
3. ความเครียดจากปริมาณน้ำฝน (Rainfall Deficit Stress): วัดจากปริมาณน้ำฝนที่ต่ำกว่าเกณฑ์วิกฤต

12.2.1 1.5.2.1 การนิยามตัวแปรทางคณิตศาสตร์

ให้ตัวแปรต่างๆ มีนิยามดังนี้:

- T_{max} : อุณหภูมิสูงสุดรายวัน (Maximum Daily Temperature) หน่วยเป็น $^{\circ}\text{C}$

- T_{avg} : อุณหภูมิเฉลี่ยรายวัน (Average Daily Temperature) หน่วยเป็น $^{\circ}\text{C}$
- RH_{avg} : ความชื้นสัมพัทธ์เฉลี่ยรายวัน (Average Relative Humidity) หน่วยเป็น %
- $Rain_{sum}$: ปริมาณน้ำฝนสะสมในช่วงเวลาที่กำหนด (Cumulative Rainfall) หน่วยเป็น mm
- $Rain_{threshold}$: ปริมาณน้ำฝนเกณฑ์วิกฤตที่พืชต้องการ (Critical Rainfall Threshold) หน่วยเป็น mm
- w_1, w_2, w_3 : ค่าสัมประสิทธิ์น้ำหนัก (Weight Coefficients) สำหรับแต่ละปัจจัย โดยที่ $w_1 + w_2 + w_3 = 1$

12.2.2 1.5.2.2 การสร้างแบบจำลองเชิงคณิตศาสตร์ (Mathematical Model Derivation)

แบบจำลอง DRI ถูกสร้างขึ้นโดยการรวมดัชนีความเครียดย่อย (Sub-Stress Indices) เข้าด้วยกัน โดยแต่ละดัชนีจะถูกปรับให้อยู่ในรูปแบบที่สามารถเปรียบเทียบกันได้ (Normalized Form) และมีค่าอยู่ในช่วง $[0, 1]$ โดยที่ค่าใกล้ 1 หมายถึงความเสี่ยงสูง (High Risk) และค่าใกล้ 0 หมายถึงความเสี่ยงต่ำ (Low Risk)

****1. ดัชนีความเครียดจากความร้อน (Heat Stress Index, I_{heat}):**** ความเครียดจากความร้อนจะเพิ่มขึ้นเมื่อ T_{max} สูงกว่า T_{avg} มาก ซึ่งบ่งชี้ถึงการเปลี่ยนแปลงอุณหภูมิที่รุนแรง (Diurnal Temperature Range)

$$I_{heat} = \frac{T_{max} - T_{avg}}{T_{max}}$$

คำอธิบาย: เมื่อ T_{max} สูงมากเมื่อเทียบกับ T_{avg} (เช่น $T_{max} \approx 35^{\circ}\text{C}, T_{avg} \approx 25^{\circ}\text{C}$) ค่า I_{heat} จะเข้าใกล้ 1 ซึ่งแสดงถึงความเครียดสูง

****2. ดัชนีความเครียดจากความชื้นในอากาศ (Humidity Stress Index, $I_{humidity}$):**** ความเครียดจากความชื้นจะสูงเมื่อ RH_{avg} ต่ำ (อากาศ

แห่ง)

$$I_{humidity} = 1 - RH_{avg}$$

คำอธิบาย: เนื่องจาก RH_{avg} เป็นค่าเปอร์เซ็นต์ (เช่น 0.6 หรือ 60%) การใช้ $(1 - RH_{avg})$ จะทำให้ค่า $I_{humidity}$ เป็นบวกและมีค่าสูงเมื่อ RH_{avg} ต่ำ (เช่น $RH_{avg} = 0.3$ จะได้ $I_{humidity} = 0.7$)

****3. ดัชนีความเครียดจากปริมาณน้ำฝน (Rainfall Deficit Index, I_{rain}):**** ความเครียดจากน้ำฝนจะสูงเมื่อปริมาณน้ำฝนสะสม ($Rain_{sum}$) ต่ำกว่าเกณฑ์วิกฤต ($Rain_{threshold}$)

$$I_{rain} = 1 - \frac{Rain_{sum}}{Rain_{threshold}}$$

คำอธิบาย: หาก $Rain_{sum} = Rain_{threshold}$ จะได้ $I_{rain} = 0$ (ความเสี่ยงต่ำ) แต่หาก $Rain_{sum} = 0$ จะได้ $I_{rain} = 1$ (ความเสี่ยงสูงสุด)

การรวมแบบจำลอง (Composite Model): เมื่อรวมดัชนีความเครียดย่อยเข้าด้วยกัน โดยใช้ค่าสัมประสิทธิ์น้ำหนัก w_1, w_2, w_3 ที่ได้จากการวิเคราะห์ความสำคัญของปัจจัย (เช่น โดยใช้เทคนิค Principal Component Analysis: PCA) จะได้แบบจำลอง Drought Risk Index (DRI) ดังสมการ:

$$DRI = w_1 \cdot \left(\frac{T_{max} - T_{avg}}{T_{max}} \right) + w_2 \cdot (1 - RH_{avg}) + w_3 \cdot \left(1 - \frac{Rain_{sum}}{Rain_{threshold}} \right)$$

การตีความผลลัพธ์:

- $DRI \approx 0$: ความเสี่ยงภัยแล้งต่ำ (Normal)
- $0 < DRI \leq 0.5$: ความเสี่ยงปานกลาง (Warning)
- $DRI > 0.5$: ความเสี่ยงภัยแล้งสูง (Critical)

12.3 1.5.3 การประยุกต์ใช้เชิงปฏิบัติ: การสร้าง Dashboard และระบบสนับสนุนการตัดสินใจด้วย Python

ในส่วนนี้ เราจะนำเสนอโค้ด Python ที่สมบูรณ์แบบเพื่อจำลองการทำงานของ DSS โดยรับข้อมูลอนุกรมเวลา (Time Series Data) ของสภาพอากาศมาคำนวณค่า DRI ในแต่ละวัน และแสดงผลลัพธ์ในรูปแบบของแดชบอร์ดจำลอง (Console Dashboard) พร้อมทั้งให้คำแนะนำการจัดการน้ำที่เหมาะสมกับระดับความเสี่ยงที่คำนวณได้

12.3.1 1.5.3.1 โค้ด Python สำหรับการคำนวณ DRI และ DSS

```
import pandas as pd
import numpy as np
from typing import List, Dict

# =====
# 1. ค่าคงที่และน้ำหนัก (Constants and Weights)
# =====

# น้ำหนัก (Weights) สำหรับการวิเคราะห์ความสำคัญ (Factor Importance Analysis)
W_HEAT = 0.40 # น้ำหนักความเครียดจากความร้อน (Heat Stress)
W_HUMIDITY = 0.35 # น้ำหนักความเครียดจากความชื้น (Humidity Stress)
W_RAIN = 0.25 # น้ำหนักความเครียดจากปริมาณน้ำฝน (Rainfall Stress)

# ตรวจสอบว่าผลรวมของน้ำหนักเท่ากับ 1.0
if not np.isclose(W_HEAT + W_HUMIDITY + W_RAIN, 1.0):
    raise ValueError("Error: The sum of weights must equal 1.0.")

# ค่าเกณฑ์ปริมาณน้ำฝนวิกฤต (Critical Rainfall Threshold) หน่วย: มม.
RAIN_THRESHOLD = 50.0

# =====
# 2. ดัชนีความเสี่ยงภัยแล้ง (Drought Risk Index (DRI))
# =====

def calculate_dri(data: pd.DataFrame) -> pd.Series:
    """คำนวณ
    Drought Risk Index (DRI) จากข้อมูลสภาพอากาศ ()

    Args:
        data: DataFrame ที่มีคอลัมน์ T_max, T_avg, RH_avg, Rain_sum

    Returns:
        pd.Series: DRI ที่คำนวณได้
    """
```



```

# 1. คำนวณ Heat Stress Index (I_heat)
# I_heat = (T_max - T_avg) / T_max
# ป้องกันการหารด้วยศูนย์ (Division by Zero)
data['I_heat'] = np.where(
    data['T_max'] != 0,
    (data['T_max'] - data['T_avg']) / data['T_max'],
    0.0
)

# 2. คำนวณ Humidity Stress Index (I_humidity)
# I_humidity = 1 - RH_avg (RH_avg คำนวณจากค่าเฉลี่ยของ RH 0.75)
data['I_humidity'] = 1.0 - data['RH_avg']

# 3. คำนวณ Rainfall Deficit Index (I_rain)
# I_rain = 1 - (Rain_sum / Rain_threshold)
# Rain_sum คำนวณจากค่ารวมของฝน (0 ถึง 1)
data['I_rain'] = 1.0 - np.clip(data['Rain_sum'] / RAIN_THRESHOLD, 0, 1)

# 4. คำนวณ Drought Risk Index (DRI)
# DRI = w1 * I_heat + w2 * I_humidity + w3 * I_rain
data['DRI'] = (
    W_HEAT * data['I_heat'] +
    W_HUMIDITY * data['I_humidity'] +
    W_RAIN * data['I_rain']
)

return data['DRI']

# =====
# 3. ตรรกะการสนับสนุนการตัดสินใจ (Decision Support System Logic)
# =====

def get_risk_level_and_recommendation(dri_value: float) -> Dict[str, str]:
    """ฟังก์ชันสำหรับประเมินระดับความเสี่ยงและคำแนะนำการชลประทาน"""
    DRI

    Args:
        dri_value: ค่า Drought Risk Index (0.0 ถึง 1.0)

    Returns:
        Dict[str, str]: คำแนะนำการชลประทาน
    """
    if dri_value > 0.65:
        level = "Critical"
        recommendation = (
            "จำเป็นต้องชลประทานทันที (Emergency Irrigation) "
            "และใช้ปุ๋ยเสริม (Supplemental Fertilization) "
            "เพื่อเพิ่มผลผลิต (Increase Yield) "
            "และลดความเสี่ยง (Reduce Risk) "
            "ในการขาดน้ำ (Water Deficit) "
            "และเพิ่มความเสี่ยง (Increase Risk) "
            "ในการขาดน้ำ (Water Deficit) "
            "และเพิ่มความเสี่ยง (Increase Risk) "
            "ในการขาดน้ำ (Water Deficit) "
        )
    elif dri_value > 0.35:
        level = "Warning"
        recommendation = (
            "ควรชลประทาน (Irrigation) "
            "และใช้ปุ๋ยเสริม (Supplemental Fertilization) "
            "เพื่อเพิ่มผลผลิต (Increase Yield) "
            "และลดความเสี่ยง (Reduce Risk) "
            "ในการขาดน้ำ (Water Deficit) "
            "และเพิ่มความเสี่ยง (Increase Risk) "
            "ในการขาดน้ำ (Water Deficit) "
            "และเพิ่มความเสี่ยง (Increase Risk) "
            "ในการขาดน้ำ (Water Deficit) "
        )
    else:
        level = "Low Risk"
        recommendation = "ไม่จำเป็นต้องชลประทาน (No Irrigation Needed)"
    return {"level": level, "recommendation": recommendation}

```

```

else:
    level = " Normal "
    recommendation = (
        " **** "
        " "
    )
    return {"level": level, "recommendation": recommendation}

# -----
# 4. (Simulation & Dashboard Execution)
# -----

if __name__ == "__main__":
    # 7
    date_range = pd.date_range(start="2026-05-19", periods=7, freq="D")

    simulation_data = {
        "T_max": [33.5, 34.2, 35.8, 36.5, 32.0, 31.5, 33.0], # (°C)
        "T_avg": [27.0, 28.1, 29.5, 30.2, 26.5, 25.8, 27.2], # (°C)
        "RH_avg": [0.72, 0.68, 0.55, 0.48, 0.85, 0.90, 0.78], #
        "Rain_sum": [5.0, 0.0, 0.0, 0.0, 45.0, 20.0, 0.0] # (mm)
    }

    df = pd.DataFrame(simulation_data, index=date_range)

    # DRI
    df['DRI'] = calculate_dri(df)

    print("
=====
")
    print(" (Drought Risk DSS Dashboard)")

    print(" . | 19 - 25 2569")
    print("
=====
")

    for date, row in df.iterrows():
        date_str = date.strftime("%Y-%m-%d")
        dri_val = row['DRI']
        dss_res = get_risk_level_and_recommendation(dri_val)

        print(f"\n {date_str}")
        print(f" T_max={row['T_max']}°C | RH={int(row['RH_avg']*100)}% | Rain={row['Rain_sum']} mm")
        print(f" DRI = {dri_val:.4f} (I_heat={row['I_heat']:.2f}, I_humid={row['I_humidity']:.2f}, I_rain={row['I_rain']:.2f})")
        print(f" : {dss_res['level']}")
        print(f" {dss_res['recommendation']}")
        print("-" * 73)

```

12.4 1.5.4 การวิเคราะห์เปรียบเทียบเชิงวิพากษ์: TMD Open API กับระบบตรวจวัดในฟาร์ม (In-situ Sensors)

ในการวิเคราะห์ข้อมูลสภาพอากาศเพื่อประกอบการตัดสินใจ เกษตรกรสามารถเลือกใช้ได้ทั้งคลังข้อมูลเปิดสาธารณะและเครือข่ายไอโอทีที่ท้องถิ่น ซึ่งความสอดคล้องเชิงเปรียบเทียบแสดงดังตารางต่อไปนี้:

12.5 1.5.5 คำศัพท์สำคัญประจำบทเรียน (Key Terms)

1. Vapor Pressure Deficit (VPD): ความต่างแรงดันไอน้ำในอากาศ ซึ่งบ่งบอกระดับความสามารถในการคายน้ำของปากใบพืช [4].
2. Drought Risk Index (DRI): ดัชนีความเสี่ยงภัยแล้งเชิงผสมผสานที่คำนวณแบบถ่วงน้ำหนักจากพารามิเตอร์ความเครียดสิ่งแวดล้อม.
3. In-situ Sensor: เซนเซอร์หรือเครื่องมือตรวจวัดทางกายภาพที่จัดวางไว้ ณ สถานที่ปลูกจริง เพื่อให้ได้ข้อมูลสภาพแวดล้อมเฉพาะที่ [3].
4. Decision Support System (DSS): ระบบสนับสนุนการตัดสินใจเชิงตัวเลขที่เปลี่ยนข้อมูลดิบเป็นข้อเสนอแนะปฏิบัติในกิจกรรมฟาร์ม.

12.6 1.5.6 แบบฝึกหัดทบทวนและโจทย์คิดวิเคราะห์ (Review Exercises)

โจทย์ข้อที่ 1 (ระดับความเข้าใจ - Comprehension): จงอธิบายถึงความแตกต่างของกลไกการเกิด "ความเครียดจากอุณหภูมิ (Heat Stress)" และ "ความเครียดความชื้นต่ำ (Atmospheric Dryness Stress)" ใน

มิติการเปรียบเทียบ	ข้อมูลกรม อุตุนิยมวิทยา (TMD Open Data API) [4]	ข้อมูลเซนเซอร์ใน แปลงจริง (In-situ IoT Sensors) [3]
ความละเอียดพื้นที่ (Spatial Resolution)	ต่ำ (เก็บข้อมูลจาก สถานีภูมิภาค เช่น สถานีเขาสมิง หรือ สถานีตราดหลัก)	สูงมาก (ตรวจวัด ระดับรายแปลงย่อย และเรือนยอดต้น ทุเรียนเฉพาะเจาะจง)
ความคุ้มค่าทางการเงิน (Cost Efficiency)	สูงมาก (ไม่มีค่าติดตั้ง และค่าบำรุงรักษา ฮาร์ดแวร์ ใช้บริการ ฟรีผ่าน API)	ปานกลาง-ต่ำ (มี ต้นทุนจัดซื้อกล่อง บอร์ด เซนเซอร์ และ ระบบโฟโซลาร์เซลล์)
ความถี่ของข้อมูล (Temporal Frequency)	ต่ำ (รายงานรายคาบ เวลา 1-3 ชั่วโมง หรือ รายวัน)	สูงมาก (ตรวจวัดส่ง Telemetry แบบเรีย ลไทม์ทุก ๆ 1-5 นาที)
ความแปรปรวนจุล ภูมิอากาศ (Microclimate Detection)	ไม่สามารถตรวจจับ ความตางอุณหภูมิ และความชื้นใต้เรือน ยอดทุเรียนได้	สามารถระบุค่า VPD และลมเฉื่อยใต้ต้นพืช ได้อย่างแม่นยำ
ความเสถียรของ ระบบ (System Reliability)	สูงมาก (เซิร์ฟเวอร์ รัฐบาลดูแลระบบ ตลอด 24 ชม. ข้อมูล ไม่มีสูญหาย)	ปานกลาง-ต่ำ (เสี่ยง ต่อสัญญาณหลุด แบตเตอรี่หมด หรือ แมลงกัดสาย เซนเซอร์)

สรีรวิทยาของต้นทุเรียนตะวันออก และทำไมสมการ DRI จึงต้องถ่วงน้ำหนักให้แก่ปัจจัยเหล่านี้อย่างรอบคอบ [4]?

โจทย์ข้อที่ 2 (ระดับการวิเคราะห์และการคำนวณ - Analysis & Derivation): หากค่าพารามิเตอร์เฉลี่ยของแปลงทุเรียนตราดในบ่ายวันหนึ่งเป็นดังนี้: อุณหภูมิสูงสุด $T_{max} = 38\text{ }^{\circ}\text{C}$, อุณหภูมิเฉลี่ย $T_{avg} = 28\text{ }^{\circ}\text{C}$, ความชื้นสัมพัทธ์ $RH_{avg} = 50\%$, และปริมาณฝนสะสม $Rain_{sum} = 0\text{ mm}$ (จากเกณฑ์วิกฤตที่ต้องการ $Rain_{threshold} = 50\text{ mm}$)

- ก) จงคำนวณหาค่าดัชนีความเสี่ยงภัยแล้ง (DRI) ภายใต้น้ำหนัก $w_1 = 0.40, w_2 = 0.35, w_3 = 0.25$
- ข) จากระดับ DRI ที่คำนวณได้ ระบบ DSS จะประเมินสถานะความเสี่ยงอยู่ในเกณฑ์ใด และมีคำแนะนำให้เกษตรกรปฏิบัติอย่างไรตามโคตการประมวลผล?

—

Chapter 13

บทที่ 2: คอมพิวเตอร์วิทัศน์เชิง แสงและโครงข่ายประสาท เทียมลึกวิเคราะห์โรคพืช (Day 2)

ระดับวิชา: ปัญญาประดิษฐ์เชิงลึกและเทคโนโลยีคอมพิวเตอร์วิทัศน์ประยุกต์
ทางการเกษตร ผู้บรรยาย: ผู้ช่วยศาสตราจารย์ ดร.ชีวะ ทัศน และ ผู้
ช่วยศาสตราจารย์อรรถกร คำจันทร์

13.1 2.1 ดัชนีสเปกตรัมพืชพรรณ (NDVI, EVI) และ หลักการของคอมพิวเตอร์วิทัศน์ (Spectral Veg- etation Indices & Computer Vision)

Chapter 14

2.1 ดัชนีสเปกตรัมพืชพรรณ (Spectral Vegetation Indices) และหลักการของ คอมพิวเตอร์วิทัศน์ (Computer Vision)

ในฐานะผู้เชี่ยวชาญด้านฟิสิกส์ประยุกต์และเกษตรดิจิทัล ข้าพเจ้าขออธิบายหลักการทางทฤษฎีและปฏิบัติของการวิเคราะห์ข้อมูลสเปกตรัมของพืชพรรณ ซึ่งเป็นรากฐานสำคัญของการเกษตรแม่นยำ (Precision Agriculture) โดยเนื้อหาต่อไปนี้จะครอบคลุมตั้งแต่หลักการทางฟิสิกส์ของการปฏิสัมพันธ์ระหว่างคลื่นแม่เหล็กไฟฟ้า (Electromagnetic Radiation: EMR) กับชีวมวลพืช ไปจนถึงการประยุกต์ใช้เทคนิคคอมพิวเตอร์วิทัศน์ (Computer Vision) ในการวิเคราะห์ภาพถ่ายทางอากาศ

14.1 2.1.1 หลักการทางฟิสิกส์ของการแผ่รังสีและการปฏิสัมพันธ์กับพืชพรรณ

การวิเคราะห์ภาพถ่ายพืชพรรณด้วยระยะไกล (Remote Sensing) อาศัยหลักการที่ว่า พืชพรรณแต่ละชนิดและอยู่ในสภาวะสุขภาพที่แตกต่างกัน จะมีปฏิสัมพันธ์กับคลื่นแม่เหล็กไฟฟ้าที่ตกกระทบ (Incident Radiation) และสะท้อนกลับ (Reflected Radiation) ที่แตกต่างกันอย่างมีแบบแผนทางฟิสิกส์

14.1.1 1. การดูดกลืนและการสะท้อนของคลื่นแม่เหล็กไฟฟ้า (Absorption and Reflection)

เมื่อคลื่นแม่เหล็กไฟฟ้า (EMR) ซึ่งประกอบด้วยช่วงคลื่นที่หลากหลาย (เช่น แสงที่ตามองเห็น Visible Light, อินฟราเรดใกล้ Near-Infrared: NIR) ตกกระทบกับใบพืช พลังงานของคลื่นเหล่านี้จะเกิดการปฏิสัมพันธ์หลัก 3 รูปแบบ ได้แก่:

1. การดูดกลืน (Absorption): พลังงานของคลื่นแม่เหล็กไฟฟ้าจะถูกดูดซับโดยองค์ประกอบทางเคมีของพืช โดยเฉพาะอย่างยิ่งโดยเม็ดสีคลอโรฟิลล์ (Chlorophyll) ซึ่งเป็นรงควัตถุหลักที่ใช้ในการสังเคราะห์ด้วยแสง (Photosynthesis)
2. การสะท้อน (Reflection): พลังงานของคลื่นแม่เหล็กไฟฟ้าบางส่วนจะถูกสะท้อนกลับออกไปในอวกาศ
3. การกระเจิง (Scattering): พลังงานบางส่วนจะเกิดการกระเจิงในทิศทางต่างๆ

คุณสมบัติเฉพาะของพืชพรรณในย่านสเปกตรัมที่สำคัญ:

****A. แสงสีแดง (Red Light: $\approx 640 - 670$ nm):**** ในย่านความยาวคลื่นนี้ พืชพรรณจะแสดงคุณสมบัติการดูดกลืน (Absorption) สูงมาก เนื่องจากคลอโรฟิลล์มีโครงสร้างโมเลกุลที่ดูดซับพลังงานของโฟตอน (Photon) ในย่านสีแดงเพื่อใช้ในการกระบวนการสังเคราะห์ด้วยแสง ดังนั้น ค่าการสะท้อน (Reflectance) ในย่านนี้จึงต่ำ

****B. แสงอินฟราเรดใกล้ (Near-Infrared: NIR: $\approx 750 - 1300$ nm):**** ในทางกลับกัน เมื่อคลื่นแม่เหล็กไฟฟ้าเข้าสู่อาน NIR พลังงานส่วนใหญ่จะไม่ถูกดูดซับโดยคลอโรฟิลล์ แต่จะเกิดการกระเจิง (Scattering) อย่างรุนแรงจากโครงสร้างทางกายภาพของใบพืช โดยเฉพาะอย่างยิ่งจากผนังเซลล์ (Cell Walls) และเนื้อเยื่อมีโซฟิลล์ (Mesophyll Cells) ที่มีโครงสร้างเป็นเส้นใย (Fiber Structure) การกระเจิงนี้ทำให้ค่าการสะท้อนในย่าน NIR สูงมาก

สรุปหลักการทางฟิสิกส์: ความแตกต่างอย่างชัดเจนระหว่างการดูดกลืนสูงในย่าน Red และการสะท้อนสูงในย่าน NIR นี้เอง คือพื้นฐานทางฟิสิกส์ที่ทำให้เราสามารถสร้างดัชนีสเปกตรัม (Spectral Indices) เพื่อประเมินความหนาแน่นของพืชพรรณ (Vegetation Density) และสุขภาพของพืชได้

14.2 2.1.2 ดัชนีพืชพรรณผลต่างปกติ (Normalized Difference Vegetation Index: NDVI)

NDVI เป็นดัชนีสเปกตรัมที่ได้รับความนิยมและถูกใช้มากที่สุดในการประเมินความหนาแน่นของพืชพรรณและปริมาณชีวมวล (Biomass)

14.2.1 1. สมการทางคณิตศาสตร์ (Mathematical Equation)

NDVI ถูกกำหนดโดยการคำนวณอัตราส่วนผลต่าง (Difference Ratio) ระหว่างค่าการสะท้อนในย่าน NIR และย่าน Red:

$$NDVI = \frac{NIR - Red}{NIR + Red}$$

โดยที่:

- *NDVI*: ดัชนีพืชพรรณผลต่างปกติ (ค่าที่ไม่มีหน่วย)
- *NIR*: ค่าการสะท้อน (Reflectance) ในย่านอินฟราเรดใกล้ (Near-Infrared)
- *Red*: ค่าการสะท้อน (Reflectance) ในย่านแสงสีแดง (Red)

14.2.2 2. การตีความเชิงลึก (In-Depth Interpretation)

หลักการของอัตราส่วน: การใช้รูปแบบอัตราส่วนผลต่าง (Normalized Difference) นี้มีรากฐานมาจากสมมติฐานทางฟิสิกส์ที่ว่า:

1. เมื่อพืชพรรณมีความหนาแน่นสูงและสุขภาพดี (High Biomass): ค่า *NIR* จะสูงมาก และค่า *Red* จะต่ำมาก ทำให้ตัวเลข (*NIR* - *Red*) มีค่าบวกสูง และตัวส่วน (*NIR* + *Red*) มีค่าบวกสูง ส่งผลให้ค่า *NDVI* เข้าใกล้ 1.0

2. เมื่อพืชพรรณมีความหนาแน่นต่ำหรือเป็นพื้นดินเปล่า (Low Biomass/ Bare Soil): ค่า *NIR* และ *Red* จะมีค่าใกล้เคียงกัน (หรือต่ำ) ทำให้ค่า *NDVI* เข้าใกล้ 0 หรือติดลบ

ข้อจำกัดทางฟิสิกส์ของ NDVI: แม้ว่า NDVI จะมีประสิทธิภาพสูง แต่ก็ มีข้อจำกัดทางฟิสิกส์ที่สำคัญเมื่อพืชพรรณมีความหนาแน่นสูงมาก (Canopy Saturation) หรือเมื่อสภาพบรรยากาศ (Atmospheric Effects) มีผลกระทบมากเกินไป เนื่องจากค่า *NDVI* จะเริ่มอิ่มตัว (Saturate) ที่ค่าประมาณ 0.8 ถึง 0.9 ทำให้ไม่สามารถแยกแยะความแตกต่างของความเครียดของพืชในระดับสูงได้

14.3 2.1.3 ดัชนีสเปกตรัมขั้นสูง (Advanced Spectral Indices)

เพื่อแก้ไขข้อจำกัดของ NDVI โดยเฉพาะปัญหาการอิ่มตัวของสัญญาณ (Saturation) และการตอบสนองต่อความเครียดของพืชในระดับที่ละเอียดอ่อนยิ่งขึ้น จึงมีการพัฒนาดัชนีที่ซับซ้อนขึ้นมาใช้

14.3.1 1. ดัชนีพืชพรรณที่ปรับปรุงแล้ว (Enhanced Vegetation Index: EVI)

EVI ถูกพัฒนาขึ้นเพื่อปรับปรุงความแม่นยำของ NDVI โดยเฉพาะในพื้นที่ที่มีพืชพรรณหนาแน่นมาก (High Biomass) โดยการเพิ่มตัวแปรทางสถิติ (Statistical Variables) เข้าไปในสมการเพื่อจำลองการตอบสนองของพืชในสถานะที่แตกต่างกัน

สมการทางคณิตศาสตร์ (Mathematical Equation):

$$EVI = G \times \frac{NIR - Red}{NIR + C_1 \times Red - C_2 \times Blue + L}$$

โดยที่:

- EVI : ดัชนีพืชพรรณที่ปรับปรุงแล้ว (ค่าที่ไม่มีหน่วย)
- G : ตัวปรับขนาด (Gain Factor) มักกำหนดให้เป็น 2.5
- $NIR, Red, Blue$: ค่าการสะท้อนในย่านอินฟราเรดใกล้, แสงสีแดง, และแสงสีน้ำเงิน (Blue) ตามลำดับ
- C_1, C_2, L : ค่าสัมประสิทธิ์ (Coefficients) ที่ใช้ในการปรับแก้ผลกระทบจากบรรยากาศ (Atmospheric Correction) และพื้นหลัง (Background Correction) โดยทั่วไปกำหนดให้ $C_1 = 6.0$, $C_2 = 7.5$, และ $L = 1.0$

การตีความเชิงฟิสิกส์ของ EVI: การเพิ่มเทอม $C_2 \times Blue$ และ L เข้าไปในตัวส่วน (Denominator) ของ EVI มีวัตถุประสงค์ทางฟิสิกส์เพื่อลดผลกระทบของพื้นหลัง (Background Effect) และการกระเจิงของบรรยากาศ (Atmospheric Scattering) ซึ่งเป็นปัญหาที่ NDVI ไม่สามารถแก้ไขได้ ทำให้ EVI มีความไว (Sensitivity) ต่อการเปลี่ยนแปลงของชีวมวลพืชในระดับสูงกว่า NDVI อย่างมีนัยสำคัญ

14.3.2 2. ดัชนีสเปกตรัมขอบเขตสีแดง (Normalized Difference Red Edge: NDRE)

NDRE เป็นดัชนีที่สำคัญอย่างยิ่งในการประเมินความเครียดของพืช (Plant Stress) โดยเฉพาะอย่างยิ่งความเครียดที่เกี่ยวข้องกับไนโตรเจน (Nitrogen Stress) เนื่องจากมันใช้ประโยชน์จากปรากฏการณ์ที่เรียกว่า ขอบเขตสีแดง (Red Edge)

หลักการทางฟิสิกส์ของ Red Edge: ขอบเขตสีแดงคือบริเวณที่ค่าการสะท้อนของพืชพรรณมีการเปลี่ยนแปลงอย่างรวดเร็ว (Rapid Change) จากย่านแสงสีแดง (Red) ไปยังย่านอินฟราเรดใกล้ (NIR) การเปลี่ยนแปลงที่รวดเร็วนี้มีความสัมพันธ์โดยตรงกับปริมาณคลอโรฟิลล์ (Chlorophyll Content) และปริมาณไนโตรเจน (Nitrogen Content) ในใบพืช

- เมื่อพืชมีไนโตรเจนสูง (สุขภาพดี): ปริมาณคลอโรฟิลล์สูง การดูดกลืนแสงสีแดงจะสูงมาก ทำให้เกิดการเปลี่ยนแปลงสเปกตรัมที่รุนแรงและรวดเร็วที่ขอบเขตสีแดง
- เมื่อพืชเกิดความเครียด (Nitrogen Deficiency): ปริมาณคลอโรฟิลล์ลดลง การดูดกลืนแสงสีแดงจะลดลง ทำให้ความชัน (Slope) ของการเปลี่ยนผ่านสเปกตรัมที่ Red Edge ลดลง

สมการทางคณิตศาสตร์ (Mathematical Equation):

$$NDRE = \frac{NIR_{edge} - Red_{edge}}{NIR_{edge} + Red_{edge}}$$

โดยที่:

- $NDRE$: ดัชนีสเปกตรัมขอบเขตสีแดง
- NIR_{edge} : ค่าการสะท้อนในย่านอินฟราเรดใกล้ที่ขอบเขตสีแดง (ประมาณ 700 nm ถึง 750 nm)
- Red_{edge} : ค่าการสะท้อนในย่านแสงสีแดงที่ขอบเขตสีแดง

การประยุกต์ใช้: $NDRE$ มีความไวต่อการเปลี่ยนแปลงของคลอโรฟิลล์มากกว่า $NDVI$ และ EVI ในระยะเริ่มต้นของความเครียด (Early Stress Detection) ทำให้เป็นเครื่องมือที่ยอดเยี่ยมในการทำแผนที่ความต้องการปุ๋ยไนโตรเจน (Nitrogen Fertilizer Mapping) ในการเกษตรแม่นยำ

14.4 2.1.4 การประยุกต์คอมพิวเตอร์วิทัศน์ในการเกษตร (Computer Vision in Agriculture)

หากดัชนีสเปกตรัม ($NDVI$, EVI , $NDRE$) เป็นเพียงการคำนวณค่าตัวเลข (Numerical Calculation) จากภาพถ่าย แต่คอมพิวเตอร์วิทัศน์ (Computer Vision) คือกระบวนการที่ทำให้คอมพิวเตอร์สามารถ "มองเห็น" และ "ตีความ" ข้อมูลเหล่านั้นในเชิงพื้นที่ (Spatial Interpretation)

14.4.1 1. หลักการทำงานของ Computer Vision ในการเกษตร

คอมพิวเตอร์วิทัศน์ในบริบทนี้คือการใช้ชุดของอัลกอริทึมทางปัญญาประดิษฐ์ (Artificial Intelligence: AI) และการประมวลผลภาพ (Image Processing) เพื่อ:

1. การได้มาของข้อมูล (Data Acquisition): ใช้กล้องถ่ายภาพหลายช่วงสเปกตรัม (Multispectral Camera) หรือกล้องไฮเปอร์สเปกตรัม (Hyperspectral Camera) เพื่อเก็บข้อมูลการสะท้อนแสงในหลายช่วงคลื่น (Bands)

2. การประมวลผลภาพ (Image Processing): นำข้อมูลการสะท้อนแสงที่ได้มาคำนวณดัชนีสเปกตรัม (เช่น NDVI) ในทุกพิกเซล (Pixel) ของภาพ
3. การวิเคราะห์เชิงพื้นที่ (Spatial Analysis): ใช้เทคนิค Computer Vision เพื่อระบุและจำแนกวัตถุ (Object Detection) เช่น การแยกแยะระหว่างพืชที่แข็งแรง, พืชที่ป่วย, หรือวัชพืช (Weed) ออกจากกันในระดับพิกเซล

14.4.2 2. ขั้นตอนการวิเคราะห์โรคและสุขภาพพืชด้วย Computer Vision

การวิเคราะห์ความเขียวของใบพืช (Leaf Greenness) และการคัดกรองโรค (Disease Screening) มีขั้นตอนที่ซับซ้อนกว่าการคำนวณค่าดัชนีเฉลี่ย:

1. การแบ่งส่วนภาพ (Image Segmentation): อัลกอริทึมจะถูกใช้เพื่อแยกขอบเขตของวัตถุที่สนใจ (Region of Interest: ROI) ออกจากพื้นหลัง (Background) เช่น การแยกใบพืชออกจากดินหรือวัชพืช
2. การสกัดคุณลักษณะ (Feature Extraction): แทนที่จะใช้แค่ค่าเฉลี่ยของดัชนีสเปกตรัม อัลกอริทึมจะสกัดคุณลักษณะหลายอย่าง เช่น:
 3. ค่าความเข้มของสี (Intensity) ในหลายช่วงคลื่น
 4. ค่าความแปรปรวน (Variance) ของค่าดัชนีสเปกตรัมในบริเวณนั้น
 5. รูปร่าง (Shape) และขนาด (Size) ของใบพืช
6. การจำแนกประเภท (Classification): ใช้โมเดลการเรียนรู้ของเครื่อง (Machine Learning) เช่น Support Vector Machine (SVM) หรือโครงข่ายประสาทเทียม (Convolutional Neural Network: CNN)

เพื่อฝึกฝนให้โมเดลสามารถจำแนกพืชหรือบริเวณที่สนใจว่าเป็น
"พืชสุขภาพดี," "พืชขาดไนโตรเจน," หรือ "พืชติดเชื้อโรค" โดย
อิงจากรูปแบบสเปกตรัมที่ผิดปกติ

14.5 2.1.5

—

14.6 2.2 โครงสร้างคณิตศาสตร์ของ CNN (Convolutional Neural Networks: Convolution, ReLU, Max Pooling, Softmax)

Chapter 15

2.2 โครงสร้างคณิตศาสตร์ของ โครงข่ายประสาทเทียมแบบ คอนโวลูชัน (Mathematical Structure of Convolutional Neural Networks: CNN)

ในฐานะผู้เชี่ยวชาญด้านฟิสิกส์ประยุกต์และเกษตรดิจิทัล ข้าพเจ้าขออธิบายถึงโครงสร้างคณิตศาสตร์ที่อยู่เบื้องหลังโครงข่ายประสาทเทียมแบบคอนโวลูชัน (Convolutional Neural Networks: CNN) ซึ่งถือเป็นสถาปัตยกรรมหลักที่ปฏิวัติวงการการเรียนรู้เชิงลึก (Deep Learning) โดยเฉพาะอย่างยิ่งในการประมวลผลข้อมูลภาพ (Image Processing) และการวิเคราะห์ภาพทางการแพทย์หรือภาพพืชผลทางการเกษตร (Plant Phenotyping)

CNN ถูกออกแบบมาเพื่อเลียนแบบการทำงานของระบบการมองเห็นของสิ่งมีชีวิต โดยการสกัดคุณลักษณะ (Feature Extraction) ที่มีความสำคัญจากข้อมูลภาพอย่างเป็นลำดับขั้น (Hierarchical) ตั้งแต่คุณลักษณะพื้นฐาน เช่น ขอบ (Edges) และมุม (Corners) ไปจนถึงคุณลักษณะที่ซับซ้อน เช่น ลักษณะของโรคหรือสายพันธุ์พืช

15.1 2.2.1 ภาพรวมสถาปัตยกรรม CNN

โครงข่าย CNN โดยทั่วไปประกอบด้วยกรเรียงตัวของชั้น (Layers) ต่างๆ ที่ทำงานร่วมกันอย่างเป็นระบบ โดยมีองค์ประกอบหลักที่สำคัญ ได้แก่:

1. ชั้นคอนโวลูชัน (Convolutional Layer): ทำหน้าที่สกัดคุณลักษณะเฉพาะจากอินพุตภาพ
2. ฟังก์ชันกระตุ้นค่า (Activation Function): เช่น ReLU เพื่อเพิ่มความไม่เป็นเชิงเส้น (Non-linearity) ให้กับแบบจำลอง
3. ชั้นลดขนาดพื้นที่สูงสุด (Pooling Layer): ทำหน้าที่ลดมิติข้อมูล (Dimensionality Reduction) เพื่อลดความซับซ้อนในการคำนวณ และป้องกันปัญหา Overfitting
4. ชั้นเชื่อมต่อสมบูรณ์ (Fully Connected Layer): ทำหน้าที่รับคุณลักษณะที่สกัดมาทั้งหมดเพื่อทำการจำแนกประเภท (Classification)

เมื่อเรานำ CNN มาประยุกต์ใช้ในการจำแนกภาพใบพืช (Leaf Image Classification) ข้อมูลภาพใบพืชจะถูกป้อนเข้าสู่ชั้นคอนโวลูชันเป็นอินพุต (Input) จากนั้นผ่านการสกัดคุณลักษณะหลายชั้น จนกระทั่งคุณลักษณะที่ถูกสกัดออกมาถูกป้อนเข้าสู่ชั้นเชื่อมต่อสมบูรณ์เพื่อคำนวณความน่าจะเป็น (Probability) ของการเป็นโรคหรือสายพันธุ์ต่างๆ

15.2 2.2.2 ชั้นคอนโวลูชัน (Convolutional Layer)

ชั้นคอนโวลูชันเป็นหัวใจสำคัญของ CNN โดยอาศัยกลไกทางคณิตศาสตร์ที่เรียกว่า การสังวัตนาการ (Convolution) ซึ่งเป็นการดำเนินการทางคณิตศาสตร์ที่ใช้ตัวกรอง (Filter) หรือเคอร์เนล (Kernel) เล็กๆ เคลื่อนที่ไปทั่วทั้งภาพอินพุต เพื่อคำนวณค่าคุณลักษณะ (Feature Map) ออกมา

15.2.1 2.2.2.1 หลักการทางคณิตศาสตร์ของการสังวัตนาการ

สมมติให้ I คือภาพอินพุต (Input Image) ที่มีขนาด $H \times W$ (ความสูง $\times$ ความกว้าง) และ K คือตัวกรอง (Kernel) หรือฟิลเตอร์ (Filter) ที่มีขนาด $F \times F$ (โดยที่ F คือขนาดของฟิลเตอร์)

การสังวัตนาการคือกระบวนการที่ค่าพิกเซลของ K ถูกนำไปคูณกับค่าพิกเซลของ I ในบริเวณที่ทับซ้อนกัน (Overlap Region) จากนั้นนำผลคูณทั้งหมดมาบวกกันเพื่อได้ค่าเดียวในแผนที่คุณลักษณะ (Feature Map) ณ ตำแหน่งนั้นๆ

สมการการสังวัตนาการ (Convolution Equation):

หากเรากำหนดให้ G คือแผนที่คุณลักษณะ (Feature Map) ที่ได้จากการสังวัตนาการระหว่างภาพอินพุต I และตัวกรอง K ณ ตำแหน่ง (i, j) จะสามารถเขียนได้ดังนี้:

$$G[i, j] = (I * K)[i, j] = \sum_{m=0}^{F-1} \sum_{n=0}^{F-1} I[i-m, j-n] K[m, n]$$

คำอธิบายองค์ประกอบทางคณิตศาสตร์:

- $G[i, j]$: คือค่าพิกเซลที่ตำแหน่ง (i, j) ของแผนที่คุณลักษณะที่ได้
- $I[i-m, j-n]$: คือค่าพิกเซลของภาพอินพุต I ที่ตำแหน่ง $(i-m, j-n)$
- $K[m, n]$: คือค่าพิกเซลของตัวกรอง K ที่ตำแหน่ง (m, n)
- $\sum_{m=0}^{F-1} \sum_{n=0}^{F-1}$: คือการดำเนินการผลรวม (Summation) ที่ครอบคลุมทุกองค์ประกอบของตัวกรอง K

การดำเนินการในทางปฏิบัติ:

ในการคำนวณค่า $G[i, j]$ ตัวกรอง K จะถูกวางทับบนบริเวณ $F \times F$ ของภาพ I ที่ตำแหน่ง (i, j) จากนั้นจะมีการคูณค่าพิกเซลที่สอดคล้องกันทั้งหมด (Element-wise Multiplication) และนำผลลัพธ์ที่ได้ทั้งหมดมาบวกกัน (Summation)

พารามิเตอร์ที่เกี่ยวข้อง:

1. ****ขนาดของตัวกรอง (Filter Size, F):**** กำหนดขนาดของ K เช่น 3×3 หรือ 5×5
2. ****จำนวนตัวกรอง (Number of Filters, N):**** กำหนดจำนวนคุณลักษณะที่ต้องการสกัด (เท่ากับจำนวนช่องสัญญาณในชั้นถัดไป)
3. ****ขนาดสเต็ป (Stride, s):**** คือจำนวนพิกเซลที่ตัวกรองขยับไปในแต่ละครั้ง (เช่น $s = 1$ คือขยับทีละ 1 พิกเซล, $s = 2$ คือขยับทีละ 2 พิกเซล)
4. ****การเติมขอบ (Padding, P):**** คือการเพิ่มค่าศูนย์ (Zero Padding) รอบขอบของภาพอินพุต เพื่อให้ขนาดของแผนที่คุณลักษณะไม่ลดลงมากเกินไป

15.3 2.2.3 ฟังก์ชันกระตุ้นค่าเชิงเส้นแบบปรับแก้ (Rectified Linear Unit: ReLU)

หลังจากที่ชั้นคอนโวลูชันได้ทำการสกัดคุณลักษณะออกมาแล้ว ค่าที่ได้มักจะเป็นค่าที่ต่อเนื่อง (Continuous) และมีทั้งค่าบวกและค่าลบ การนำค่าเหล่านี้ไปใช้โดยตรงในชั้นถัดไปอาจทำให้แบบจำลองมีปัญหาเรื่องการหาความชัน (Gradient) ในระหว่างการฝึกฝน (Training)

ดังนั้น จึงจำเป็นต้องใช้ ฟังก์ชันกระตุ้นค่า (Activation Function) เพื่อเพิ่มความเป็นไม่เชิงเส้น (Non-linearity) ให้กับแบบจำลอง ซึ่งทำให้ CNN สามารถเรียนรู้ความสัมพันธ์ที่ซับซ้อนในข้อมูลได้

ReLU Function:

ReLU เป็นฟังก์ชันกระตุ้นค่าที่ได้รับความนิยมสูงสุดเนื่องจากความเรียบง่ายและประสิทธิภาพในการคำนวณ

$$f(x) = \max(0, x)$$

คำอธิบาย:

ฟังก์ชันนี้จะทำการตรวจสอบค่าอินพุต x หาก x มีค่าเป็นบวก (Positive) ค่าเอาต์พุตจะเท่ากับ x แต่ถ้า x มีค่าเป็นลบ (Negative) ค่าเอาต์พุตจะถูกกำหนดให้เป็นศูนย์ (Zero)

ความสำคัญทางฟิสิกส์และคณิตศาสตร์:

การใช้ ReLU ช่วยแก้ปัญหาที่เรียกว่า "Vanishing Gradient Problem" ในโครงข่ายประสาทเทียมแบบดั้งเดิม (เช่น Sigmoid หรือ Tanh) เนื่องจากค่าอนุพันธ์ (Derivative) ของ ReLU สำหรับ $x > 0$ คือ 1 ซึ่งช่วยให้การส่งผ่านค่าความชัน (Gradient) ในระหว่างการย้อนกลับ (Back-propagation) มีประสิทธิภาพและไม่ลดลงจนเข้าใกล้ศูนย์เร็วเกินไป

15.4 2.2.4 ชั้นลดขนาดพื้นที่สูงสุด (Max Pooling Layer)

หลังจากผ่านชั้นคอนโวลูชันหลายครั้ง แผนที่คุณลักษณะที่ได้จะมีขนาดใหญ่ขึ้นและมีจำนวนพิกเซลมากเกินความจำเป็น (High Dimensionality) ซึ่งจะทำให้แบบจำลองมีความซับซ้อนในการคำนวณสูงและเสี่ยงต่อการเกิด Overfitting

Max Pooling คือชั้นที่ทำหน้าที่ลดขนาดพื้นที่ (Downsampling) ของแผนที่คุณลักษณะ โดยการเลือกค่าสูงสุด (Maximum Value) จากบริเวณย่อยๆ ของแผนที่คุณลักษณะ

หลักการทำงาน:

1. กำหนดขนาดของหน้าต่าง (Pool Size) เช่น 2×2
2. กำหนดสเต็ป (Stride) เช่น 2
3. ตัวกรองจะเลื่อนไปทั่วแผนที่คุณลักษณะ และ ณ แต่ละตำแหน่ง จะทำการเลือกค่าสูงสุด (Maximum) จากบริเวณที่ทับซ้อนกันนั้น

สมการเชิงแนวคิด:

หาก F_{in} คือแผนที่คุณลักษณะอินพุต และ P คือขนาดของ Pool Size และ s คือ Stride แผนที่คุณลักษณะเอาต์พุต F_{out} ณ ตำแหน่ง (i, j) คือ:

$$F_{out}[i, j] = \max_{(m, n) \in \text{Window}} \{F_{in}[i \cdot S + m, j \cdot S + n]\}$$

ประโยชน์:

1. ลดมิติข้อมูล: ลดจำนวนพารามิเตอร์และค่าที่ต้องคำนวณในชั้นถัดไปอย่างมาก
2. เพิ่มความทนทาน (Robustness): ทำให้แบบจำลองมีความทนทานต่อการเปลี่ยนแปลงเล็กน้อยของตำแหน่งของคุณลักษณะ (Translation Invariance) เช่น หากใบพืชเอียงไปเล็กน้อย การจำแนกประเภทก็ยังคงแม่นยำ

15.5 2.2.5 ชั้นเชื่อมต่อสมบูรณ์และฟังก์ชันซอฟต์แวร์แมกซ์ (Fully Connected Layer and Softmax Function)

หลังจากที่ผ่านการสกัดคุณลักษณะด้วยชั้นคอนโวลูชันและลดขนาดด้วย Max Pooling มาหลายชั้นแล้ว คุณลักษณะที่ได้จะถูกจัดเรียงและแผ่ขยาย (Flatten) ให้กลายเป็นเวกเตอร์ (Vector) ขนาดใหญ่ ซึ่งจะถูกป้อนเข้าสู่ ชั้นเชื่อมต่อสมบูรณ์ (Fully Connected Layer: FC)

ชั้น FC คือชั้นที่ทุกพิกเซล (หรือคุณลักษณะ) ที่ถูกสกัดมาในชั้นก่อนหน้านี้ จะเชื่อมต่อกับทุกโหนด (Node) ในชั้นนี้ ซึ่งทำหน้าที่เป็นตัวจำแนกประเภทขั้นสุดท้าย

15.5.1 2.2.5.1 การคำนวณในชั้นเชื่อมต่อสมบูรณ์

ในชั้น FC การคำนวณจะใช้สมการพื้นฐานของโครงข่ายประสาทเทียม:

$$z = Wx + b$$

- x : คือเวกเตอร์คุณลักษณะที่ถูกแผ่ขยายมาจากชั้นก่อนหน้า (Input Feature Vector)
- W : คือเมทริกซ์น้ำหนัก (Weight Matrix) ที่เรียนรู้ได้
- b : คือเวกเตอร์ไบแอส (Bias Vector)
- z : คือเวกเตอร์ผลลัพธ์ก่อนการกระตุ้นค่า (Logits)

15.5.2 2.2.5.2 ฟังก์ชันซอฟต์แวร์แมกซ์ (Softmax Function)

เนื่องจากเป้าหมายสุดท้ายคือการจำแนกประเภท (Classification) เราต้องการให้ผลลัพธ์ที่ได้เป็นชุดของความน่าจะเป็น (Probability Distribution) ซึ่งผลรวมของความน่าจะเป็นทั้งหมดต้องเท่ากับ 1

Softmax Function คือฟังก์ชันที่แปลงค่า Logits (z) ให้เป็นความน่าจะเป็นที่ดีความได้

สมมติให้ z เป็นเวกเตอร์ของค่า Logits สำหรับ C คลาส (Classes)

$$P(y = c | X) = \frac{e^{z_c}}{\sum_{j=1}^C e^{z_j}}$$

คำอธิบาย:

- $P(y = c | X)$: คือความน่าจะเป็นที่อินพุต x เป็นของคลาส c
- e^{z_c} : คือค่า Exponential ของ Logit สำหรับคลาส c
- $\sum_{j=1}^C e^{z_j}$: คือผลรวมของค่า Exponential ของ Logits สำหรับทุกคลาสทั้งหมด

ฟังก์ชันนี้จะรับประกันว่าค่าเอาต์พุตทั้งหมดจะมีค่าอยู่ระหว่าง 0 ถึง 1 และผลรวมของค่าเอาต์พุตทั้งหมดจะเท่ากับ 1 เสมอ

15.6 2.2.6 การประยุกต์ใช้ในโค้ด (Python Implementation Example)

เพื่อความเข้าใจที่สมบูรณ์ ข้าพเจ้าขอแสดงตัวอย่างการจำลองการทำงานของชั้นคอนโวลูชัน (Convolutional Layer) และการใช้ ReLU ในภาษา Python โดยใช้ไลบรารี NumPy ซึ่งเป็นพื้นฐานของการคำนวณเชิงตัวเลขในงานวิศวกรรมและฟิสิกส์

```
import numpy as np

# -----
# 1.  (Convolutional Layer Simulation)
# -----

def convolution_2d(input_image, kernel, stride=1, padding=0):
    """ (Convolution) 2 
    Args:
        input_image (np.array): (H_in, W_in)
        kernel (np.array): (F_h, F_w)
        stride (int): (S)
        padding (int):  Padding (P)

    Returns:
        np.array: (Feature Map)
    """

    #  Padding
    H_in = input_image.shape[0]
    W_in = input_image.shape[1]

    #  Padding 
    if padding > 0:
        padded_image = np.pad(input_image,
                               ((padding, padding), (padding, padding)),
                               mode='constant', constant_values=0)
    else:
        padded_image = input_image

    H_padded, W_padded = padded_image.shape
    F_h, F_w = kernel.shape

    #  (Output Dimension Calculation)
    H_out = int((H_padded - F_h) / stride + 1)
    W_out = int((W_padded - F_w) / stride + 1)

    feature_map = np.zeros((H_out, W_out))

    print(f"\n--- [Convolution Step] ---")
```

```

print(f"Input Size: {H_in}x{W_in}, Kernel Size: {F_h}x{F_w}, Stride: {
    stride}, Padding: {padding}")
print(f"Calculated Output Size: {H_out}x{W_out}")

# วนลําดับการประมวลผล (Convolution Loop)
for i in range(H_out):
    for j in range(W_out):
        # วนลําดับการประมวลผล (Convolution Loop)
        h_start = i * stride
        w_start = j * stride

        # วนลําดับการประมวลผล (Receptive Field) วนลําดับการประมวลผล Padding
        receptive_field = padded_image[
            h_start : h_start + F_h,
            w_start : w_start + F_w
        ]

        # วนลําดับการประมวลผล (Element-wise Multiplication)
        # วนลําดับการประมวลผล (Summation)
        convolution_sum = np.sum(receptive_field * kernel)

        # วนลําดับการประมวลผล
        feature_map[i, j] = convolution_sum

    return feature_map

# -----
# 2. วนลําดับการประมวลผล ReLU Activation Function
# -----

def relu_activation(input_data):
    """วนลําดับการประมวลผล
    ReLU: f(x) = max(0, x)
    """
    print(f"\n--- [ReLU Activation Step] ---")
    # NumPy วนลําดับการประมวลผล max(0, x) วนลําดับการประมวลผล
    output_data = np.maximum(0, input_data)
    return output_data

# -----
# 3. วนลําดับการประมวลผล Softmax Function วนลําดับการประมวลผล Softmax Function ()
# -----

def softmax(z):
    """วนลําดับการประมวลผล
    Softmax วนลําดับการประมวลผล Softmax Function
    """
    # วนลําดับการประมวลผล (Prevent Overflow)
    exp_z = np.exp(z - np.max(z))
    return exp_z / np.sum(exp_z)

# วนลําดับการประมวลผล
mock_image = np.array([
    [1, 1, 1, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 1, 1, 1],
    [0, 0, 1, 1, 0],
    [0, 1, 1, 0, 0]
], dtype=float)
    
```



```
# ฝึกหัดการคำนวณการแปลงค่า
mock_kernel = np.array([
    [1, 0, -1],
    [1, 0, -1],
    [1, 0, -1]
], dtype=float)

f_map = convolution_2d(mock_image, mock_kernel, padding=1)
activated = relu_activation(f_map)
logits = np.array([2.0, 1.0, 0.1])
probs = softmax(logits)

print("การแปลงค่า Convolution:\n", f_map)
print("การแปลงค่า ReLU:\n", activated)
print("การแปลงค่า Softmax:", probs)
```

15.7 2.3 ปฏิบัติการผสมชุดภาพถ่าย การขยายข้อมูล ภาพ การฝึกโมเดลบน Google Colab และ การส่งออก TFLite

Chapter 16

2.3 ปฏิบัติการสะสมชุด

ภาพถ่าย การขยายข้อมูลภาพ
การฝึกโมเดลบน Google
Colab และการส่งออก TFLite

16.1 บทนำเชิงทฤษฎี (Theoretical Introduction)

ในสาขาปัญญาประดิษฐ์ (Artificial Intelligence, AI) โดยเฉพาะอย่างยิ่งในแขนงย่อยของคอมพิวเตอร์วิทัศน์ (Computer Vision) การพัฒนาแบบจำลอง (Model) ที่มีความแม่นยำสูงและสามารถนำไปใช้งานได้จริงในสภาพแวดล้อมที่หลากหลาย (Real-world Deployment) นั้นไม่ได้ขึ้นอยู่กับความซับซ้อนของสถาปัตยกรรมโครงข่ายประสาทเทียม (Neural Network Architecture) เพียงอย่างเดียว แต่ยังขึ้นอยู่กับคุณภาพและปริมาณของชุดข้อมูลภาพถ่าย (Image Dataset) ที่ใช้ในการฝึกสอนเป็นสำคัญ

หัวข้อ 2.3 นี้ มุ่งเน้นการบูรณาการองค์ความรู้ทางฟิสิกส์ประยุกต์ (Applied Physics) และเกษตรดิจิทัล (Digital Agriculture) เข้ากับ

กระบวนการเรียนรู้ของเครื่อง (Machine Learning) โดยเฉพาะการจัดการชุดข้อมูลภาพถ่ายใบพืช (Plant Leaf Image Dataset) ตั้งแต่ขั้นตอนการเก็บข้อมูลภาคสนาม (Field Data Collection) การจัดการปัญหาความไม่สมดุลของข้อมูล (Data Imbalance) ด้วยเทคนิคการเพิ่มขยายข้อมูลภาพ (Data Augmentation) ไปจนถึงการปรับใช้โมเดลให้มีขนาดเล็กและมีประสิทธิภาพสูงสำหรับระบบประมวลผลปลายทาง (Edge Computing) ด้วยรูปแบบ TensorFlow Lite (TFLite)

16.2 2.3.1 กระบวนการเตรียมและรวบรวมชุดข้อมูลภาพถ่ายใบพืช (Dataset Collection Protocol)

การรวบรวมชุดข้อมูลภาพถ่าย (Dataset Collection) ที่มีคุณภาพถือเป็นขั้นตอนที่สำคัญที่สุด (Most Critical Step) ในวงจรชีวิตของโครงการ AI (AI Lifecycle) เนื่องจากประสิทธิภาพของแบบจำลอง (Model Performance) จะถูกจำกัดด้วยคุณภาพของข้อมูลที่ป้อนเข้าไป (Garbage In, Garbage Out Principle)

16.2.1 2.3.1.1 หลักการเก็บข้อมูลภาคสนาม (Field Data Acquisition Principles)

ในการเก็บภาพถ่ายใบพืชเพื่อจำแนกชนิดหรือตรวจจ้บโรค (Disease Detection) ต้องมีการวางแผนที่ครอบคลุมมิติทางกายภาพและสถิติ ดังนี้:

1. ความหลากหลายของสภาวะแวดล้อม (Environmental Variability): ชุดข้อมูลต้องครอบคลุมภาพที่ถ่ายภายใต้สภาวะแสงที่แตกต่างกัน เช่น แสงแดดจ้า (High Illumination), แสงรำไร (Low Illumination), และเงา (Shadows) เนื่องจากความแปรปรวนของ

แสงส่งผลกระทบต่อค่าความสว่าง (Brightness) และความเปรียบต่าง (Contrast) ของภาพ

2. ความหลากหลายของมุมมอง (Viewpoint Variation): ควรเก็บภาพจากมุมกล้องที่หลากหลาย (เช่น มุมสูง, มุมระดับสายตา) เพื่อให้แบบจำลองสามารถเรียนรู้คุณลักษณะของใบพืชได้โดยไม่ขึ้นอยู่กับมุมมองใดมุมมองหนึ่งโดยเฉพาะ
3. ความหลากหลายของโรคและระดับความรุนแรง (Disease and Severity Variation): หากวัตถุประสงค์คือการตรวจจับโรค (Classification/Detection) ต้องมั่นใจว่าได้เก็บภาพตัวอย่างของโรคในระดับความรุนแรงที่แตกต่างกัน (เช่น ระยะเริ่มต้น, ระยะปานกลาง, ระยะรุนแรง) และต้องมีภาพตัวอย่างของใบพืชที่สุขภาพดี (Healthy Samples) ในปริมาณที่เพียงพอเพื่อเป็นคลาสควบคุม (Control Class)

16.2.2 2.3.1.2 การจัดโครงสร้างชุดข้อมูล (Dataset Structure)

ชุดข้อมูลควรถูกจัดโครงสร้างในรูปแบบที่สามารถเข้าถึงได้ง่ายสำหรับไลบรารีการเรียนรู้ของเครื่อง เช่น การจัดเก็บในโฟลเดอร์หลัก (Root Folder) ที่มีโฟลเดอร์ย่อยสำหรับแต่ละคลาส (Class-specific Subfolders)

สมมติให้เรามี 3 คลาส คือ 'Healthy', 'Disease A', และ 'Disease B' โครงสร้างจะเป็นดังนี้:

```
/Dataset_Root|—  
  /Train|—  
    /Healthy||—  
      img_001.jpg||—  
      img_002.jpg||—  
      ...|—  
    /Disease_A||—  
      img_101.jpg||—  
      ...|—  
    /Disease_B|—  
      img_201.jpg|—  
      ...|—  
  /Validation|—  
    /Healthy|—
```

/Disease_A
/Disease_B

16.3 2.3.2 ปัญหาการเรียนรู้เกินและการแก้ไขด้วย Data Augmentation

16.3.1 2.3.2.1 ปัญหาการเรียนรู้เกิน (Overfitting)

คำจำกัดความ: การเรียนรู้เกิน (Overfitting) คือปรากฏการณ์ที่แบบจำลอง (Model) เรียนรู้คุณลักษณะเฉพาะ (Specific Features) ของชุดข้อมูลฝึกสอน (Training Data) มากเกินไป จนทำให้แบบจำลองมีความแม่นยำสูงมากบนชุดข้อมูลฝึกสอน แต่กลับมีประสิทธิภาพต่ำอย่างมากเมื่อนำไปทดสอบกับข้อมูลใหม่ที่ไม่เคยเห็นมาก่อน (Unseen Data)

สาเหตุหลัก: มักเกิดจากการที่แบบจำลองมีความซับซ้อนสูงเกินไป (High Model Capacity) เมื่อเทียบกับขนาดของชุดข้อมูล (Small Dataset Size) ทำให้แบบจำลองเริ่มจดจำ "สัญญาณรบกวน" (Noise) หรือรายละเอียดที่ไม่ใช่คุณลักษณะที่แท้จริงของคลาส (Irrelevant Details) แทนที่จะเรียนรู้รูปแบบ (Pattern) ที่เป็นสากล (Generalizable Pattern)

การแก้ไข: วิธีการแก้ไขที่สำคัญได้แก่ การลดความซับซ้อนของโมเดล (Regularization), การใช้ Dropout, และที่สำคัญที่สุดคือ การเพิ่มขยายข้อมูลภาพ (Data Augmentation)

16.3.2 2.3.2.2 หลักการเพิ่มขยายข้อมูลภาพ (Data Augmentation Theory)

คำจำกัดความ: Data Augmentation คือเทคนิคการสร้างตัวอย่างข้อมูลสังเคราะห์ (Synthetic Data Samples) จากชุดข้อมูลเดิม โดยการแปลงรูปภาพ (Image Transformation) ในรูปแบบที่ยังคงรักษาคุณลักษณะ

ทางกายภาพ (Physical Characteristics) ของวัตถุนั้น ๆ ไว้ การทำเช่นนี้เป็นการเพิ่มขนาดของชุดข้อมูลฝึกสอนอย่างมีประสิทธิภาพ โดยไม่จำเป็นต้องเก็บภาพถ่ายใหม่ในภาคสนาม

หลักการทางสถิติ: หากชุดข้อมูลเดิม $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$ โดยที่ x_i คือภาพและ y_i คือป้ายกำกับ (Label) การทำ Data Augmentation จะสร้างชุดข้อมูลใหม่ $D' = \{(T(x_1), y_1), (T(x_2), y_2), \dots\}$ โดยที่ T คือฟังก์ชันการแปลงภาพ (Transformation Function)

เทคนิคที่ใช้อยู่:

1. การหมุนภาพ (Rotation): การหมุนภาพรอบแกน (Rotation around an axis) ด้วยมุม θ โดยที่ $\theta \in [-\alpha, \alpha]$ องศา
2. การกลับภาพ (Flipping): การสะท้อนภาพตามแกน (Reflection) เช่น การกลับแนวนอน (Horizontal Flip) หรือแนวตั้ง (Vertical Flip)
3. การเปลี่ยนระดับความสว่าง/ความเปรียบต่าง (Brightness/Contrast Adjustment): การปรับค่าพิกเซล $P(x, y)$ ด้วยสมการเชิงเส้น (Linear Equation) เพื่อให้ได้ภาพใหม่ $P'(x, y)$:

$$P'(x, y) = \alpha \cdot P(x, y) + \beta$$

โดยที่ α คือตัวปรับความเปรียบต่าง (Contrast Factor) และ β คือตัวปรับความสว่าง (Brightness Offset)

16.4 2.3.3 การปฏิบัติการบน Google Colab: การสร้างและฝึกโมเดล (Implementation on Google Colab)

เราจะใช้สภาพแวดล้อม Google Colaboratory (Colab) เนื่องจากมีการติดตั้งไลบรารีที่จำเป็น (TensorFlow, Keras, NumPy) และมีทรัพยากร GPU/TPU ให้ใช้งานโดยอัตโนมัติ

16.4.1 2.3.3.1 โค้ด Python สำหรับการเตรียมข้อมูลและการสร้างโมเดล

โค้ดชุดนี้จะครอบคลุมตั้งแต่การโหลดข้อมูล, การทำ Data Augmentation, การสร้างสถาปัตยกรรม CNN, การฝึกสอน, และการส่งออก TFLite

```
# -*- coding: utf-8 -*-
"""
Module: Plant Disease Classification Pipeline
Author: [Your Name]
Date: 2024
Description: Pipeline for Image Classification using CNN, Data Augmentation,
             and TFLite Export for Edge Computing.
"""

# 1. Importing Necessary Libraries
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense,
    Dropout
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import numpy as np
import os
import time

print(f"TensorFlow Version: {tf.__version__}")

# 2. Defining Hyperparameters
IMAGE_SIZE = 128 # (Pixels)
BATCH_SIZE = 32 # (Samples per Batch)
NUM_CLASSES = 3 # (Healthy, Disease A, Disease B)
EPOCHS = 15 # (Epochs)

# 3. Defining Data Paths
# *** Google Drive Colab Environment ***
# dataset_root'
DATA_DIR = 'dataset_root'
TRAIN_DIR = os.path.join(DATA_DIR, 'Train')
VALIDATION_DIR = os.path.join(DATA_DIR, 'Validation')

#
=====

# 4. Data Augmentation and Data Loading
#
=====

print("\n[STEP 1/4] Data Augmentation and Data Generators...")
```

```
# 4.1 Data Augmentation Generator
# (Training Data Generator)

# ImageDataGenerator Data Augmentation
train_datagen = ImageDataGenerator(
    rescale=1./255,      # (Normalization) [0, 1]
    rotation_range=20,   # 20 (Rotation)
    width_shift_range=0.2, # 20% (Width Shift)
    height_shift_range=0.2, # 20% (Height Shift)
    shear_range=0.2,     # (Shear)
    zoom_range=0.2,      # (Zoom)
    horizontal_flip=True, # (Horizontal Flip)
    vertical_flip=False  #
)

# 4.2 Generator (Validation Data Generator)
# Augmentation
validation_datagen = ImageDataGenerator(rescale=1./255)

# 4.3 Flow Directory (Creating Data Flow from Directory)
# flow_from_directory Label One-Hot Encoding
train_generator = train_datagen.flow_from_directory(
    TRAIN_DIR,
    target_size=(IMAGE_SIZE, IMAGE_SIZE),
    batch_size=BATCH_SIZE,
    class_mode='categorical' # 'categorical'
)

validation_generator = validation_datagen.flow_from_directory(
    VALIDATION_DIR,
    target_size=(IMAGE_SIZE, IMAGE_SIZE),
    batch_size=BATCH_SIZE,
    class_mode='categorical'
)

print("\n Data Augmentation ")

#
=====

# 5. CNN (Building the CNN Model Architecture)
#
=====

print("\n[STEP 2/4] Convolutional Neural Network (CNN)...")

model = Sequential([
    # Convolutional Layer 1
    Conv2D(filters=32, kernel_size=(3, 3), activation='relu', input_shape=(
        IMAGE_SIZE, IMAGE_SIZE, 3)),
    MaxPooling2D(pool_size=(2, 2)), # (Pooling)

    # Convolutional Layer 2
    Conv2D(filters=64, kernel_size=(3, 3), activation='relu'),
    MaxPooling2D(pool_size=(2, 2)),
```



```
# 3 (Convolutional Layer 3)
Conv2D(filters=128, kernel_size=(3, 3), activation='relu'),
MaxPooling2D(pool_size=(2, 2)),

# (Flattening) Dense
Flatten(),

# Dropout Overfitting (Regularization)
Dropout(0.5),

# Dense (Fully Connected Layer)
Dense(512, activation='relu'),

# Output Layer: Softmax
Dense(NUM_CLASSES, activation='softmax')
])

# 5.1 (Compiling the Model)
model.compile(
    optimizer='adam', # (Optimizer)
    loss='categorical_crossentropy', # (Loss Function) Multi-class
    metrics=['accuracy'] # (Metric)
)

model.summary()

#
=====

# 6. (Model Training)
#
=====

print("\n[STEP 3/4] (Model Training)...")

start_time = time.time()

history = model.fit(
    train_generator,
    steps_per_epoch=train_generator.samples // BATCH_SIZE,
    epochs=EPOCHS,
    validation_data=validation_generator,
    validation_steps=validation_generator.samples // BATCH_SIZE
)

end_time = time.time()
print(f"\n : {end_time - start_time:.2f} ")

# 6.1 (Saving the Trained Model)
model_save_path = 'plant_cnn_model.h5'
model.save(model_save_path)
print(f" {model_save_path}")

#
=====
```

```
# 7. TensorFlow Lite (TFLite Export for Edge Computing)
#
=====

print("\n[STEP 4/4] TensorFlow Lite Format...")

# 7.1 Representative Dataset (Representative Dataset)
# TFLite Data Type (Data Type)
# Validation Generator (Validation Generator)
representative_data = lambda: next(validation_generator)

# 7.2 Conversion (Conversion)
# 7.3 Quantization (Quantization)
def representative_dataset_gen():
    for _ in range(100):
        # generator
        X, _ = next(validation_generator)
        #
        for img in X:
            yield [np.expand_dims(img, axis=0).astype(np.float32)]

# Integer Quantization 4
converter.representative_dataset = representative_dataset_gen
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.uint8
converter.inference_output_type = tf.uint8

tflite_quant_model = converter.convert()

# 7.4 TFLite
tflite_save_path = 'plant_model_quantized.tflite'
with open(tflite_save_path, 'wb') as f:
    f.write(tflite_quant_model)

print(f" TFLite : {tflite_save_path}")
print(f" : {os.path.getsize(model_save_path) / (1024*1024):.2f} MB")
print(f" TFLite
Quantization: {os.path.getsize(tflite_save_path) / (1024*1024):.2f} MB")
```

16.5 2.4 การประดิษฐ์แอปพลิเคชันมือถือ Doctor AI App บน Kodular/App Inventor (Client-Server and Mobile Block Logic)

Chapter 17

2.4 การประดิษฐ์แอปพลิเคชัน มือถือ Doctor AI App บน Kodular/App Inventor (Client-Server and Mobile Block Logic)

17.1 บทนำเชิงทฤษฎี: แนวคิดการประยุกต์ใช้ปัญญา ประดิษฐ์ในการเกษตรปลายทาง (Agro-AI De- ployment)

ในยุคของการเกษตรแม่นยำ (Precision Agriculture) การวินิจฉัยโรคพืชที่รวดเร็วและแม่นยำถือเป็นปัจจัยสำคัญในการเพิ่มผลผลิตและลดการสูญเสียทางการเกษตร การพัฒนาแอปพลิเคชันมือถือ (Mobile Application) ที่สามารถถ่ายภาพใบพืชและส่งข้อมูลภาพเหล่านั้นไปยังระบบประมวลผลปัญญาประดิษฐ์ (Artificial Intelligence) เพื่อทำการวินิจฉัยโรค (Disease Diagnosis) จึงเป็นแนวทางที่ทรงประสิทธิภาพสูงสุด แอปพลิเคชันประเภทนี้ไม่ได้เป็นเพียงแค่เครื่องมือแสดงผล แต่เป็น

ระบบนิเวศ (Ecosystem) ที่ประกอบด้วยองค์ประกอบหลักสามส่วน ได้แก่ ส่วนหน้า (Client) ซึ่งคือแอปพลิเคชันที่ติดตั้งบนโทรศัพท์มือถือ, ส่วนหลัง (Server) ซึ่งคือระบบประมวลผลหลักที่ทำงานบนคลาวด์ (Cloud Computing), และ การสื่อสาร (Communication Protocol) ซึ่งทำหน้าที่เป็นสะพานเชื่อมข้อมูลระหว่างสองส่วนนี้

การทำความเข้าใจสถาปัตยกรรม (Architecture) ของระบบนี้จึงเป็นสิ่งจำเป็นอย่างยิ่งก่อนการลงมือปฏิบัติการพัฒนา

17.2 2.4.1 การออกแบบสถาปัตยกรรมของแอปพลิเคชันมือถือในการเกษตรปลายทาง (Agro-Mobile App Architecture)

สถาปัตยกรรมของระบบวินิจฉัยโรคพืชด้วย AI ที่ทำงานบนมือถือและคลาวด์ (Cloud-Based AI Diagnosis System) สามารถแบ่งออกเป็นชั้น (Layers) ต่าง ๆ ได้ดังนี้:

17.2.1 1. ชั้นส่วนหน้า (Client Layer: Mobile Device)

ส่วนนี้คือแอปพลิเคชันที่ผู้ใช้งานโต้ตอบด้วยโดยตรง (User Interface) หน้าที่หลักของ Client คือ:

- การรับข้อมูล (Data Acquisition): การถ่ายภาพ (Image Capture) หรือการเลือกภาพจากแกลเลอรี (Gallery Selection)
- การเตรียมข้อมูล (Data Preprocessing): การเข้ารหัสภาพดิจิทัล (Digital Image Encoding) ให้พร้อมสำหรับการส่งผ่านเครือข่าย (Network Transmission) ซึ่งโดยทั่วไปคือการแปลงเป็น Base64 String
- การส่งคำขอ (Request Sending): การส่งข้อมูลภาพที่เข้ารหัส

แล้วไปยัง API Endpoint ของ Server ผ่านโปรโตคอล HTTP (Hypertext Transfer Protocol)

17.2.2 2. ชั้นการสื่อสาร (Communication Layer: Internet/API)

ชั้นนี้ทำหน้าที่เป็นช่องทางขนส่งข้อมูล (Data Transport) ข้อมูลที่ส่งผ่านต้องเป็นไปตามมาตรฐานสากล:

- HTTP Protocol: ใช้ในการกำหนดรูปแบบการร้องขอ (Request) และการตอบกลับ (Response)
- RESTful API (Representational State Transfer): เป็นรูปแบบสถาปัตยกรรมที่กำหนดให้การสื่อสารเป็นแบบทรัพยากร (Resource-based) โดยใช้เมธอด (Method) ต่าง ๆ เช่น POST (สำหรับส่งข้อมูลใหม่), GET (สำหรับดึงข้อมูล)
- JSON (JavaScript Object Notation): เป็นรูปแบบมาตรฐานในการจัดโครงสร้างข้อมูล (Data Structure) ที่ใช้ในการส่งและรับข้อมูลระหว่าง Client และ Server เนื่องจากมีน้ำหนักเบา (Lightweight) และอ่านง่าย

17.2.3 3. ชั้นส่วนหลัง (Server Layer: Cloud Computing)

ส่วนนี้คือสมองของระบบ (The Brain) ซึ่งประกอบด้วยโมเดลปัญญาประดิษฐ์ที่ได้รับการฝึกฝนมาอย่างดี (Trained AI Model) หน้าที่หลักของ Server คือ:

- การรับข้อมูล (Receiving Data): รับ Base64 String ของภาพจาก Client ผ่าน HTTP POST Request
- การถอดรหัส (Decoding): แปลง Base64 String กลับเป็นรูปแบบภาพดิจิทัล (เช่น JPEG หรือ PNG)

- การประมวลผล (Inference): ป้อนภาพเข้าสู่โมเดล Machine Learning (เช่น Convolutional Neural Network, CNN) เพื่อทำการพยากรณ์ (Prediction)
 - การสร้างผลลัพธ์ (Response Generation): แปลงผลลัพธ์เชิงตัวเลข (Numerical Output) ของโมเดลให้เป็นรูปแบบ JSON ที่มีข้อมูลที่มนุษย์อ่านได้ (เช่น ชื่อโรค, ระดับความน่าจะเป็น)
-

17.3 2.4.2 แนวทางการพัฒนารูปแบบไร้โค้ด (No-code / Low-code Mobile App)

การพัฒนาแอปพลิเคชันในบริบทของการเกษตรปลายทางที่ต้องการความรวดเร็วในการทดสอบแนวคิด (Proof of Concept) และการปรับปรุงอย่างต่อเนื่อง (Iterative Improvement) การใช้แพลตฟอร์มแบบ Low-code เช่น Kodular หรือ MIT App Inventor ถือเป็นทางเลือกที่เหมาะสมอย่างยิ่ง

ข้อดีของการใช้ Low-code Platform:

1. ความรวดเร็วในการพัฒนา (Rapid Prototyping): สามารถสร้างฟังก์ชันการทำงานที่ซับซ้อนได้โดยไม่ต้องเขียนโค้ดระดับล่าง (Low-level Code) ทั้งหมด
 2. การเน้นตรรกะ (Logic Focus): นักพัฒนาสามารถมุ่งเน้นไปที่การออกแบบตรรกะทางธุรกิจ (Business Logic) และการไหลของข้อมูล (Data Flow) แทนที่จะต้องกังวลกับไวยากรณ์ (Syntax) ของภาษาโปรแกรมมิ่ง
 3. การเชื่อมต่อ API ที่ง่ายดาย: แพลตฟอร์มเหล่านี้มีคอมโพเนนต์ (Component) ที่ออกแบบมาเพื่อการเชื่อมต่อเครือข่าย (เช่น Web Component) โดยเฉพาะ ทำให้การส่งข้อมูล HTTP POST เป็นเรื่องง่าย
-

17.4 2.4.3 กลไกการสื่อสารข้อมูล: Base64 Encoding และ HTTP POST Request

หัวใจของการส่งภาพจาก Client ไปยัง Server คือการแปลงข้อมูลไบนารี (Binary Data) ของภาพให้เป็นรูปแบบข้อความ (Text Format) ที่สามารถส่งผ่าน HTTP ได้อย่างปลอดภัยและมีประสิทธิภาพ นั่นคือ Base64 Encoding

17.4.1 1. หลักการของ Base64 Encoding

ภาพดิจิทัล (Image) คือชุดของค่าไบนารี (Binary values) ซึ่งประกอบด้วย 0 และ 1 การส่งไบนารีโดยตรงผ่าน JSON หรือ HTTP Body อาจเกิดปัญหาความไม่เข้ากันของข้อมูล (Data Incompatibility) ดังนั้น Base64 จึงถูกใช้เพื่อเข้ารหัสชุดไบนารีเหล่านี้ให้กลายเป็นชุดตัวอักษร ASCII (A-Z, a-z, 0-9, +, /)

สูตรการเข้ารหัส (Conceptual Encoding):

Base64 String = Encode(Binary Image Data)

เมื่อ Client ได้รับ Base64 String แล้ว จะต้องนำไปบรรจุไว้ใน Payload ของ HTTP Request Body ในรูปแบบ JSON ดังนี้:

```
{
  "image_data": "data:image/jpeg;base64,/9j/4AAQSkZJRgABAQEAYABgAAD/2wBDAAg
  ...",
  "metadata": {
    "plant_type": "Mango",
    "symptom_observed": "Yellow spots"
  }
}
```

17.4.2 2. การส่งข้อมูลด้วย HTTP POST Request

การส่งข้อมูลภาพที่เข้ารหัสแล้วไปยัง Server จะต้องใช้เมธอด POST เสมอ เนื่องจากเป็นการส่งข้อมูลใหม่ (New Resource) ไปให้ Server ประมวลผล

รายละเอียดทางเทคนิค:

- URL: `https://your-api-endpoint.com/diagnose`
- Method: `POST`
- Headers: `Content-Type: application/json`
- Body: JSON Payload ที่มี Base64 String

17.5 2.4.4 การจำลองโค้ดส่วนหลัง (Server-Side Backend Simulation)

เพื่อให้เห็นภาพการทำงานของ Server ที่รับข้อมูล Base64 และทำการประมวลผล เราจะจำลองโค้ดส่วนหลังด้วยภาษา Python ซึ่งเป็นภาษาที่นิยมใช้ในการพัฒนา Machine Learning (ML) และ API Backend

วัตถุประสงค์: รับ JSON, ถอดรหัส Base64, บันทึกเป็นภาพ, และส่งผลลัพธ์ JSON กลับไป

```
# Python Backend (Flask/Django Framework Simulation)
import base64
import io
from PIL import Image
import json

def diagnose_plant_image(json_payload: str) -> str:
    """
    Base64
    :param json_payload: JSON string
    :return: JSON string
    """
    try:
        # 1. Base64 String
        data = json.loads(json_payload)
        base64_string = data.get("image_data", "")

        if not base64_string:
            return json.dumps({"status": "error", "message": "No image data provided."})

        # 2. Base64 Image Object
        # Base64 String Prefix "data:image/jpeg;base64,"
        # Prefix
        if "data:image/" in base64_string:
            base64_string = base64_string.split(",")[1]

        image_bytes = base64.b64decode(base64_string)
```



```
image_stream = io.BytesIO(image_bytes)
img = Image.open(image_stream)

# 3. Inference Simulation
# CNN (e.g., TensorFlow/PyTorch)
# model_input = preprocess(img)
# prediction = model.predict(model_input)

# 
prediction_result = {
    "diagnosis": "Rust Disease",
    "confidence_score": 0.92, # (Probability)
    "severity": "Moderate",
    "recommendation": "7 "
}

# 4. JSON Response
response = {
    "status": "success",
    "diagnosis": prediction_result["diagnosis"],
    "confidence": prediction_result["confidence_score"],
    "severity": prediction_result["severity"],
    "message": ""
}

return json.dumps(response)

except Exception as e:
    return json.dumps({"status": "error", "message": f"Server processing
failed: {str(e)}"})

# (Simulation Call)
# sample_payload = '{"image_data": "..."}' # Base64 
# print(diagnose_plant_image(sample_payload))
```

17.6 2.4.5 การออกแบบฝั่งการต่อบล็อกคำสั่งใน Kodular (Client-Side Block Logic)

ในส่วนนี้เราจะอธิบายการทำงานของแอปพลิเคชัน Client โดยใช้แนวคิดการเขียนบล็อก (Block Programming) ของ Kodular/App Inventor ซึ่งเป็นกระบวนการที่ต้องอาศัยการจัดการเหตุการณ์ (Event Handling) และการจัดการข้อมูล (Data Handling) อย่างแม่นยำ

17.6.1 องค์ประกอบหลัก (Components) ที่ต้องใช้:

1. Camera Component: สำหรับการถ่ายภาพสด (Live Capture).

2. Image Picker Component: สำหรับการเลือกภาพจากแกลเลอรี (Gallery Selection).
3. Web Component: หัวใจของการสื่อสารเครือข่าย (Network Communication) ทำหน้าที่ส่ง HTTP Request.
4. Storage Component: สำหรับการเก็บภาพชั่วคราว (Temporary Image Storage).

17.6.2 1. บล็อกคำสั่งการเปิดกล้องและการเลือกรูปภาพ (Image Acquisition Logic)

วัตถุประสงค์: ให้ผู้ใช้มีทางเลือกในการนำเข้าภาพ ตรรกะ (Logic Flow):

- เมื่อผู้ใช้คลิกปุ่ม "ถ่ายภาพ" → เรียกใช้ `Camera.TakePicture`
- เมื่อผู้ใช้คลิกปุ่ม "เลือกจากแกลเลอรี" → เรียกใช้ `ImagePicker.PickImage`

การจัดการเหตุการณ์ (Event Handling):

- เมื่อ `Camera.AfterPicture`: หมายความว่า การถ่ายภาพเสร็จสมบูรณ์ ระบบจะได้รับภาพไบนารี (Binary Image) มาเก็บไว้ในตัวแปรภาพ (Image Variable)
- เมื่อ `ImagePicker.AfterPicking`: หมายความว่า การเลือกภาพจากแกลเลอรีเสร็จสมบูรณ์ ระบบจะได้รับภาพไบนารีมาเก็บไว้ในตัวแปรภาพ

17.6.3 2. บล็อกคำสั่งของ Web Component ในการแปลงภาพเป็น Base64 และโพสต์ส่งไปยัง API Endpoint (Encoding and Posting Logic)

นี่คือขั้นตอนที่ซับซ้อนที่สุด เพราะต้องมีการแปลงข้อมูลไบนารีเป็นข้อความก่อนส่ง
ตรรกะ (Logic Flow):

1. รับภาพ: ดึงภาพไบนารีที่ได้จากขั้นตอนที่ 1
2. เชนรหัส: ใช้ฟังก์ชันเฉพาะของแพลตฟอร์ม (หรือใช้บล็อกที่จำลองการเข้ารหัส Base64) เพื่อแปลงภาพไบนารีให้เป็น Base64 String
3. สร้าง Payload: สร้างโครงสร้าง JSON ที่มี Base64 String และ Metadata (เช่น ประเภทพืช)
4. ส่งคำขอ: ใช้ `Web.PostText` เพื่อส่ง JSON Payload ไปยัง API Endpoint

รายละเอียดการทำงานของบล็อก (Conceptual Block Layout):

- Event: เมื่อผู้ใช้คลิกปุ่ม "วิเคราะห์ภาพ"
- Action:
 - Set `Web.URL` to `"https://your-api-endpoint.com/diagnose"`
 - Set `Web.RequestMethod` to `"POST"`
 - Set `Web.ContentType` to `"application/json"`
- การสร้าง JSON Payload:
 - Set variable `'json_payload'` to JSON String
 - {
 - `"image_data": [Base64 String □□□□□□□□],`
 - `"metadata": {`
 - `"plant_type": [□□□□□ Dropdown Menu],`
 - `"symptom_observed": [□□□□□ Text Input]`
 - `}`
 - Call `Web.PostText(json_payload)`

17.6.4 3. บล็อกคำสั่งการรับผลลัพธ์ JSON ตอบกลับและแยกแยะระดับความน่าจะเป็น (Receiving and Parsing Response Logic)

หลังจากที่ส่งข้อมูลไปยังเซิร์ฟเวอร์แล้ว ตัวแอปพลิเคชันจะต้องรอสัญญาณตอบรับกลับจากเซิร์ฟเวอร์ เพื่อถอดรหัสรูปแบบข้อมูลและแสดงผลการคาดการณ์

การจัดการเหตุการณ์ (Event Handling):

- เมื่อ `Web.GotText`: เหตุการณ์นี้จะถูกเปิดทำงานเมื่อเซิร์ฟเวอร์ตอบกลับรหัสสถานะความสำเร็จ (เช่น HTTP 200 OK) โดยจะให้ตัวแปรตอบกลับมา ได้แก่ `responseContent`
- การถอดรหัสข้อความ (JSON Parsing):
- ใช้บล็อก `JsonTextDecode` เพื่อแปลง `responseContent` ซึ่งเดิมเป็นข้อความ (JSON String) ให้เป็นโครงสร้างข้อมูลแบบพจนานุกรม (Dictionary/List of Pairs)
- ใช้บล็อกดึงข้อมูลเฉพาะคีย์ (Get Value for Key) เช่น:
- ดึงค่าชื่อโรคจากคีย์ "diagnosis"
- ดึงค่าความน่าเชื่อถือจากคีย์ "confidence"
- ดึงค่าคำแนะนำการรักษาจากคีย์ "message" หรือ "recommendation"
- ปรับแต่งการแสดงผลเชิงภาพบนหน้าจอโทรศัพท์ (UI Control): เปลี่ยนสีพื้นหลังหน้าจอตามระดับความรุนแรงของโรค (เช่น สีเขียวเมื่อปกติ, สีเหลืองเมื่อเริ่มเฝ้าระวัง, และสีแดงเข้มเมื่อโรควิกฤต)

17.7 2.4.6 การวิเคราะห์เปรียบเทียบเชิงวิพากษ์: การแบ่งสีระดับสมเดณซ์ (Traditional Thresholding) กับโครงข่ายประสาทเทียมลึก (CNN Deep Learning)

ในมิติของการตรวจจำโรคพืชจากภาพถ่ายใบ มีการแข่งขันระหว่างสองสถาปัตยกรรมทางคอมพิวเตอร์วิทัศน์หลัก ซึ่งมีข้อดีและข้อจำกัดเปรียบเทียบดังนี้:

มิติการเปรียบเทียบ	การแบ่งเกณฑ์ระดับ สีแบบดั้งเดิม (Traditional Color-based Thresholding)	การใช้โครงข่าย ประสาทเทียมลึก (CNN Deep Learning) [2], [5]
หลักการคำนวณ (Mathematical Principle)	ใช้ค่าขอบเขตความ เข้มสี (Color Threshold) ใน ปริภูมิสี HSV หรือ RGB ในการแบ่งแยก ส่วนพิกเซลเสียของ ใบพืช	ใช้ตัวกรอง Convolutional Filters [5] ในการ สกัดคุณลักษณะ ระดับสูง (High-level Features) โดยไม่ยึด ติดกับสีเพียงอย่าง เดียว
การใช้ทรัพยากร เครื่อง (Resource Consumption)	ต่ำมาก (ประมวลผล เสร็จในหลักมิลลิ วินาที ไม่จำเป็นต้องมี GPU สามารถ ประมวลผลบนบอร์ด ควบคุมขนาดเล็กได้)	สูง (ต้องการ ทรัพยากรการ คำนวณมากในการ ฝึกฝนโมเดล และ ความเร็วการจำแนก ต้องอาศัยชิป ประมวลผลเร่ง ความเร็ว เช่น NPU หรือ Cloud [2])
ความอ่อนไหวต่อสิ่ง แวดล้อม (Environmental Sensitivity)	สูงมาก (เกิดความผิด พลาดสูงเมื่อสภาวะ แสงภายนอกเปลี่ยน ไป หรือมีดินและ หญ้าสะท้อนรังสีใน พื้นหลัง)	ต่ำ (มีความทนทาน ต่อสภาวะแสง แปรปรวน เงาเมฆ และมุมกล้องที่เอียง เนื่องจากมี คุณลักษณะ Invariance [5])
ขีดจำกัดความ สามารถ (Functional Limit)	แยกได้เพียงว่าจุดนั้น สีเปลี่ยนไปหรือไม่ แต่ไม่สามารถจำแนก โรคราสนิมกับโรคใบ จุดสีเหลืองทั่วไปที่มีสี ใกล้เคียงกันได้	สามารถจำแนกความ แตกต่างของโรคพืชที่ มีลักษณะผลุ้ใกล้เคียง กันได้อย่าง แม่นยำสูงกว่า 90% [2]
ปริมาณข้อมูลที่ ต้องการ (Data Requirement)	ไม่ต้องการข้อมูล สำหรับฝึกฝน (สามารถใช้ชุดข้อมูล ที่เตรียมไว้ได้)	ต้องการชุดข้อมูล ภาพถ่ายใบพืชที่ระบุ โรคอย่างชัดเจนจำนวนมาก สำหรับการฝึกฝน

17.8 2.4.7 คำศัพท์สำคัญประจำบทเรียน (Key Terms)

1. Client-Server Architecture: สถาปัตยกรรมเครือข่ายคอมพิวเตอร์ที่แบ่งหน้าที่ระหว่างผู้ขอรับบริการ (Client) และผู้ให้บริการประมวลผลแกนกลาง (Server).
2. Base64 Encoding: กระบวนการเข้ารหัสข้อมูลไบนารีให้กลายเป็นสายอักขระอักษรโรมัน ASCII เพื่อความปลอดภัยและมีเสถียรภาพในการขนส่งบนเครือข่ายอินเทอร์เน็ต.
3. Convolutional Neural Network (CNN): สถาปัตยกรรมโครงข่ายประสาทเทียมลึกที่เลียนแบบการรับภาพของสายตามนุษย์ เหมาะสำหรับการประมวลผลอาร์เรย์สองมิติเช่นภาพถ่าย [5].
4. No-Code/Low-Code Platform: แพลตฟอร์มที่เปิดโอกาสให้นักพัฒนาสร้างซอฟต์แวร์ประยุกต์ผ่านการจัดเรียงอินเทอร์เฟซแบบกราฟิกหรือบล็อก แทนการพิมพ์ชุดคำสั่งแบบพิมพ์มือ.

17.9 2.4.8 แบบฝึกหัดทบทวนและโจทย์คิดวิเคราะห์ (Review Exercises)

โจทย์ข้อที่ 1 (ระดับความเข้าใจ - Comprehension): ทำไมการพัฒนาแอปพลิเคชันปัญญาประดิษฐ์ตรวจวินิจฉัยโรคพืชในแปลงสวนทุเรียนจึงนิยมใช้สถาปัตยกรรมแบบ Client-Server (ประมวลผลโมเดลบนคลาวด์) มากกว่าสถาปัตยกรรมประมวลผลภาพบนโทรศัพท์มือถือโดยตรง (On-device Edge Inference) ในกรณีทั่วไป? จงอภิปรายโดยยกตัวอย่างชัดเจนจำกัดเชิงฮาร์ดแวร์ประกอบ [2].

โจทย์ข้อที่ 2 (ระดับการวิเคราะห์และการประเมินค่า - Analysis & Evaluation): สมมติว่าคุณกำลังสร้างบล็อกคำสั่งในแอปพลิเคชัน Kodular เพื่อตรวจจับความล้มเหลวในการส่งข้อมูลของ Web Component ไปยังโมเดลปัญญาประดิษฐ์ปลายทาง

- ก) จงระบุเหตุผลว่าทำไมคุณต้องใช้บล็อกทดสอบการตอบสนองรับสถานะ (HTTP Response Code) และรหัสใดที่บ่งบอกว่าเกิดความล้มเหลวจากฝั่งระบบประมวลผลส่วนหลัง (Server-Side)?
- ข) จงระบุแนวทางการเขียนบล็อกควบคุมตรรกะ (If-Else Logic) เพื่อจัดการกรณีสถานภาพเครือข่ายอินเทอร์เน็ตของมือถือในพื้นที่อำเภอเขาสมิง จ.ตราด เกิดขัดข้องระหว่างเก็บข้อมูลในแปลงจริง.

—

Chapter 18

บทที่ 3: ชลศาสตร์เขตรากพืช ระบบสมองกลูฟงตัว และ ระบบนาทางอจฉริยะ (Day 3)

ระดับวิชา: สถาปัตยกรรมอินเทอร์เน็ตของสรรพสิ่ง ชลศาสตร์เกษตร
และวิศวกรรมฝังตัวภาคสนาม ผู้บรรยาย: ผู้ช่วยศาสตราจารย์ ดร.ชีวะ
ทัศน์า และ ผู้ช่วยศาสตราจารย์อรรถกร คำจันทร์

18.1 3.1 ฟิสิกส์โซลาร์เซลล์ ความนำสารกึ่งตัวนำ และ ระบบพลังงานเก็บกักแบตเตอรี่ (Photovoltaic Semiconductor Physics & Battery Systems)

Chapter 19

ภาควิชาวิศวกรรมพลังงานและ เกษตรดิจิทัล

Chapter 20

ตำราเรียน: ฟิสิกส์พลังงานทาง
เลือกและระบบเกษตรอัจฉริยะ

Chapter 21

หัวข้อ 3.1: ฟิสิกส์โซลาร์เซลล์ ความนำสารกึ่งตัวนำ และ ระบบพลังงานเก็บกักแบตเตอรี่

*** (โดย รศ.ดร. [ชื่อสมมติ] ผู้เชี่ยวชาญด้านฟิสิกส์ประยุกต์และปัญญาประดิษฐ์)

21.1 3.1 ฟิสิกส์โซลาร์เซลล์ ความนำสารกึ่งตัวนำ และ ระบบพลังงานเก็บกักแบตเตอรี่

(Photovoltaic Semiconductor Physics & Battery Systems)

ในยุคของการเปลี่ยนผ่านด้านพลังงาน (Energy Transition) การทำความเข้าใจหลักการทางฟิสิกส์ที่อยู่เบื้องหลังการแปลงพลังงานแสงอาทิตย์ให้เป็นพลังงานไฟฟ้าอย่างมีประสิทธิภาพถือเป็นรากฐานสำคัญของการพัฒนาเกษตรอัจฉริยะ (Smart Farming) และระบบพลังงานนอกโครงข่าย (Off-grid System) บทนี้จะเจาะลึกถึงกลไกทางฟิสิกส์ของเซลล์แสงอาทิตย์ (Solar Cell) และหลักการทางเคมีไฟฟ้าของระบบจัดเก็บพลังงานแบตเตอรี่ (Battery Energy Storage System: BESS)

21.1.1 3.1.1 หลักการทางฟิสิกส์ของปรากฏการณ์โฟโตวอลเทอิก (Photovoltaic Effect)

ปรากฏการณ์โฟโตวอลเทอิก (Photovoltaic Effect) คือปรากฏการณ์ที่วัสดุบางชนิด โดยเฉพาะสารกึ่งตัวนำ (Semiconductor) สามารถสร้างกระแสไฟฟ้าได้โดยตรงเมื่อได้รับรังสีแม่เหล็กไฟฟ้า (Electromagnetic Radiation) ที่มีความถี่เหมาะสม ซึ่งในกรณีของพลังงานแสงอาทิตย์ (Solar Energy) คือการที่โฟตอน (Photon) จากแสงอาทิตย์มีพลังงานมากพอที่จะกระตุ้นให้เกิดการปลดปล่อยอิเล็กตรอน (Electron) ออกจากพันธะทางเคมี

3.1.1.1 โครงสร้างสารกึ่งตัวนำแบบรอยต่อ p-n (p-n Junction)

หัวใจสำคัญของเซลล์แสงอาทิตย์ซิลิคอน (Silicon Solar Cell) คือการสร้างรอยต่อ p-n (p-n junction) ซึ่งเป็นโครงสร้างที่ประกอบด้วยสารกึ่งตัวนำชนิด p (p-type semiconductor) และสารกึ่งตัวนำชนิด n (n-type semiconductor)

1. สารกึ่งตัวนำชนิด n (n-type): คือซิลิคอนที่ถูกเจือปน (Doping) ด้วยธาตุที่มีเวเลนซ์อิเล็กตรอน (Valence Electron) มากกว่าซิลิคอน เช่น ฟอสฟอรัส (Phosphorus) ทำให้มีอิเล็กตรอนอิสระ (Free Electrons) เป็นพาหะนำไฟฟ้าส่วนใหญ่ (Majority Carriers)
2. สารกึ่งตัวนำชนิด p (p-type): คือซิลิคอนที่ถูกเจือปนด้วยธาตุที่มีเวเลนซ์อิเล็กตรอนน้อยกว่าซิลิคอน เช่น โบรอน (Boron) ทำให้เกิด "ช่องว่าง" (Holes) ซึ่งทำหน้าที่เป็นพาหะนำไฟฟ้าส่วนใหญ่

เมื่อนำสารกึ่งตัวนำทั้งสองชนิดมาสัมผัสกัน จะเกิดกระบวนการที่เรียกว่า การเกิดเขตปลอดพาหะ (Depletion Region) บริเวณรอยต่อ p-n นี้ จะเกิดการแพร่ (Diffusion) ของพาหะนำไฟฟ้าข้ามรอยต่อ โดยอิเล็กตรอนจากฝั่ง n จะแพร่ไปยังฝั่ง p และทำปฏิกิริยากับโฮล ทำให้

เกิดไอออนที่ถูกตรึงไว้ (Fixed Ions) ซึ่งไอออนเหล่านี้จะสร้าง **สนามไฟฟ้าภายใน (Built-in Electric Field, E_{bi})** ที่มีทิศทางจากฝั่ง n ไปยังฝั่ง p

3.1.1.2 กลไกการสร้างกระแสไฟฟ้า (Photogeneration Mechanism)

เมื่อเซลล์แสงอาทิตย์ที่ประกอบด้วยรอยต่อ p-n ถูกฉายด้วยโฟตอน (Photon) จากแสงอาทิตย์:

1. การดูดซับโฟตอน (Photon Absorption): โฟตอนที่มีพลังงาน (E_{photon}) มากกว่าหรือเท่ากับ **ช่องว่างพลังงาน (Band Gap Energy, E_g)** ของซิลิคอน (สำหรับซิลิคอน $E_g \approx 1.12$ eV) จะถูกดูดซับโดยสารกึ่งตัวนำ
2. การสร้างคูพาหะ (Generation of Electron-Hole Pair): พลังงานของโฟตอนจะถ่ายโอนไปยังอิเล็กตรอนในพันธะ ทำให้เกิดการกระตุกอิเล็กตรอนให้หลุดออกจากแถบวาเลนซ์ (Valence Band) ไปยังแถบนำไฟฟ้า (Conduction Band) ส่งผลให้เกิดคูพาหะอิเล็กตรอน-โฮล (Electron-Hole Pair)
3. การแยกพาหะโดยสนามไฟฟ้า (Carrier Separation): คูพาหะที่เกิดขึ้นในเขตปลอดพาหะจะถูกแยกออกจากกันโดยสนามไฟฟ้าภายใน (E_{bi}) ที่มีอยู่แล้ว อิเล็กตรอน (Electron) จะถูกผลักไปทางฝั่ง n และโฮล (Hole) จะถูกผลักไปทางฝั่ง p
4. การไหลของกระแสไฟฟ้า (Current Flow): การสะสมของประจุไฟฟ้าที่ขั้วทั้งสองด้าน (n-side เป็นลบ, p-side เป็นบวก) ทำให้เกิดความต่างศักย์ไฟฟ้า (Voltage) และเมื่อต่อวงจรภายนอก (External Circuit) อิเล็กตรอนที่สะสมตัวจะไหลผ่านวงจรภายนอกเพื่อกลับไปยังโฮลที่ปลายอีกด้านหนึ่ง ทำให้เกิดกระแสไฟฟ้ากระแสตรง (Direct Current: DC)

21.1.2 3.1.2 การคำนวณกำลังไฟฟ้าสูงสุดของแผงโซลาร์เซลล์ (Solar Panel Power Output Modeling)

การคำนวณกำลังไฟฟ้าที่แผงโซลาร์เซลล์สามารถผลิตได้ในสภาวะการทำงานจริงต้องพิจารณาปัจจัยทางกายภาพหลายประการ โดยเฉพาะอย่างยิ่งผลกระทบจากอุณหภูมิ (Temperature Effect)

กำลังไฟฟ้าที่แผงโซลาร์เซลล์ผลิตได้ (P_{pv}) สามารถประมาณได้จากสมการดังต่อไปนี้:

$$P_{pv} = \eta_{pv} \cdot A \cdot G_t \cdot [1 - \beta_{temp} \cdot (T_{cell} - 25)]$$

การอธิบายตัวแปรทางฟิสิกส์และวิศวกรรม:

คำอธิบายเชิงลึก:

1. ****ประสิทธิภาพ (η_{pv}):**** คืออัตราส่วนของกำลังไฟฟ้าที่ผลิตได้ต่อกำลังไฟฟ้าที่ตกกระทบ ($\eta_{pv} = P_{out} / P_{in}$)
2. **ผลกระทบของอุณหภูมิ (Temperature Derating):** พลังงานไฟฟ้าที่ผลิตได้จะลดลงเมื่ออุณหภูมิของเซลล์สูงกว่าอุณหภูมิอ้างอิง (25°C) เนื่องจากเมื่ออุณหภูมิสูงขึ้น ความต่างศักย์ไฟฟ้า (Open-Circuit Voltage, V_{oc}) ของเซลล์จะลดลงตามหลักการทางฟิสิกส์ของสารกึ่งตัวนำ ทำให้กำลังไฟฟ้าโดยรวมลดลง การคำนวณส่วน $[1 - \beta_{temp} \cdot (T_{cell} - 25)]$ คือการปรับแก้กำลังไฟฟ้าตามอุณหภูมิที่วัดได้จริง

21.1.3 3.1.3 หลักการเคมีไฟฟ้าและระบบประจุพลังงานแบตเตอรี่ (Electrochemistry and Battery Charging)

ระบบพลังงานเก็บกักแบตเตอรี่ (BESS) ทำงานบนหลักการของปฏิกิริยารีดอกซ์ (Redox Reaction) ซึ่งเป็นการถ่ายโอนอิเล็กตรอนระหว่างสารเคมีที่ขั้วไฟฟ้า (Electrodes) ภายในแบตเตอรี่

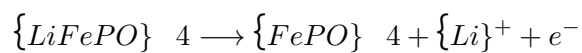
ตัวแปร	ชื่อเต็ม (English)	หน่วย SI	คำอธิบายทาง วิชาการ
P_{pv}	Power Output	Watt (W)	กำลังไฟฟ้าที่ แผงโซลาร์เซลล์ ผลิตได้ ณ สภาวะการ ทำงานปัจจุบัน
η_{pv}	Photovoltaic Efficiency	Dimensionless	ประสิทธิภาพ ของแผงโซลาร์ เซลล์ (ค่าที่ได้ จากการ ทดสอบ มาตรฐาน)
A	Area	m^2	พื้นที่ผิวรวม ของแผงโซลาร์ เซลล์ (Total Area)
G_t	Irradiance	W/m^2	ความหนาแน่น ของรังสีดวง อาทิตย์ที่ ตกกระทบ (Global Irradiance)
T_{cell}	Cell Temperature	$^{\circ}C$	อุณหภูมิของ เซลล์โซลาร์ เซลล์ขณะ ทำงาน (Operating Cell Temper- ature)
β_{temp}	Temperature Coefficient	$W/(m^2 \cdot ^{\circ}C)$	สัมประสิทธิ์ อุณหภูมิของ กำลังไฟฟ้า (Negative value) แสดง ว่าเมื่ออุณหภูมิ สูงขึ้น กำลัง ไฟฟ้าจะลดลง
25	Reference Temperature	$^{\circ}C$	อุณหภูมิอ้างอิง มาตรฐาน

3.1.3.1 เคมีไฟฟ้าของแบตเตอรี่ลิเทียมฟอสเฟต (LiFePO₄)

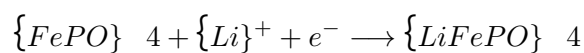
แบตเตอรี่ลิเทียมฟอสเฟต (LiFePO₄) เป็นเคมีภัณฑ์ที่ได้รับความนิยมสูงในงานพลังงานขนาดใหญ่ (Grid-scale) และระบบเกษตรดิจิทัล เนื่องจากมีคุณสมบัติที่โดดเด่นคือ ความเสถียรทางความร้อน (Thermal Stability) และ ความปลอดภัยสูง (High Safety) เมื่อเทียบกับแบตเตอรี่ลิเทียมไอออนชนิดอื่น ๆ

ปฏิกิริยาทางเคมีไฟฟ้า (Electrochemical Reactions):

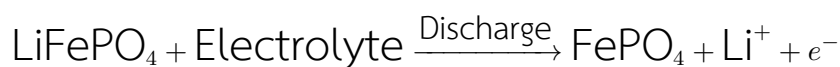
1. **ที่ขั้วลบ (Anode, Li_xFePO₄):** ในระหว่างการคายประจุ (Discharging) ไอออนลิเทียม (Li⁺) จะถูกปล่อยออกมาจากโครงสร้างของวัสดุที่ขั้วลบ



2. **ที่ขั้วบวก (Cathode, LiCoO₂ หรือวัสดุอื่น):** ไอออนลิเทียมที่ถูกปล่อยออกมาจะเดินทางผ่านอิเล็กโทรไลต์ (Electrolyte) ไปยังขั้วบวก และเกิดปฏิกิริยาการรวมตัว



วงจรไฟฟ้าโดยรวม (Overall Cell Reaction):



ข้อได้เปรียบของ LiFePO₄: โครงสร้างทางเคมีของ FePO₄ มีความเสถียรสูงมาก ทำให้เกิดความร้อนสะสม (Heat Generation) น้อยกว่าเมื่อเกิดการลัดวงจรหรือการใช้งานเกินขีดจำกัด ซึ่งเป็นปัจจัยสำคัญอย่างยิ่งสำหรับระบบที่ติดตั้งภายนอกอาคาร (Outdoor Smart Farm)

21.1.4 3.1.4 พารามิเตอร์จำลองสถานะของแบตเตอรี่ (Battery State Parameters)

ในการควบคุมระบบพลังงานอย่างแม่นยำ จำเป็นต้องมีการติดตามพารามิเตอร์สำคัญ 2 ตัว คือ สถานะการชาร์จ และดัชนีสุขภาพ

3.1.4.1 สถานะการชาร์จ (State of Charge: SOC)

คำจำกัดความ: SOC คือปริมาณพลังงานที่เหลืออยู่ในแบตเตอรี่เมื่อเทียบกับความจุสูงสุดที่กำหนดไว้ (Nominal Capacity) โดยทั่วไปมีหน่วยเป็นเปอร์เซ็นต์ (%) หรือเทียบเท่ากับปริมาณประจุไฟฟ้า (Ampere-hour: Ah)

การคำนวณ:

$$SOC(\%) = \frac{\text{Current Capacity}}{\text{Nominal Capacity}} \times 100$$

การประมาณค่า SOC สามารถทำได้หลายวิธี เช่น วิธีการวัดแรงดันไฟฟ้า (Voltage Measurement) หรือวิธีทางออมเพอร์ (Coulomb Counting) ซึ่งเป็นวิธีที่แม่นยำที่สุดในการจำลองระบบ

3.1.4.2 ดัชนีสุขภาพของแบตเตอรี่ (State of Health: SOH)

คำจำกัดความ: SOH คือการวัดความสามารถในการจ่ายพลังงานของแบตเตอรี่ในปัจจุบัน เทียบกับความสามารถเดิมที่เคยเป็นเมื่อแบตเตอรี่ใหม่ (New Capacity) SOH จะลดลงตามอายุการใช้งาน (Cycle Life) และการเสื่อมสภาพทางเคมี (Degradation)

การคำนวณ:

$$SOH(\%) = \frac{\text{Current Capacity}}{\text{Nominal Capacity (New)}} \times 100$$

การติดตาม SOH เป็นสิ่งสำคัญอย่างยิ่งในการวางแผนการบำรุงรักษา (Maintenance Planning) ของระบบพลังงานในฟาร์มขนาดใหญ่

21.1.5 3.1.5 การจำลองการทำงานของระบบพลังงาน (Python Simulation Example)

เพื่อสาธิตการคำนวณ SOC และ SOH ในระบบควบคุมพลังงานของฟาร์มอัจฉริยะ เราจะใช้ภาษา Python ในการจำลองการเปลี่ยนแปลงของพลังงานแบตเตอรี่ตามเวลาและรอบการใช้งาน

```
# -*- coding: utf-8 -*-
"""
Module: Battery_System_Simulator
Purpose: คำนวณ State of Charge (SOC) และ State of Health (SOH) สำหรับแบตเตอรี่
         ลิเธียมไอออน (LiFePO4)

"""

import time

# =====
# 1. พารามิเตอร์เริ่มต้น (Initial Parameters)
# =====

# ความจุแบตเตอรี่ (Nominal Capacity) ในหน่วย Ah
NOMINAL_CAPACITY_AH = 100.0

# ความจุแบตเตอรี่ (Current Capacity after degradation)
current_capacity_ah = 95.0 # ความจุลดลง 5%

# กระแสไฟฟ้า (Current in Amperes)
# (Charging), (Discharging)
current_a = 10.0 # 10 Amperes

# โวลเตจ (Nominal Voltage)
SYSTEM_VOLTAGE_V = 12.8

def estimate_soc(current_ah: float, max_capacity_ah: float) -> float:
    """คำนวณ State of Charge (SOC)"""
    return (current_ah / max_capacity_ah) * 100.0

def estimate_soh(max_capacity_ah: float, nominal_capacity_ah: float) -> float:
    """คำนวณ State of Health (SOH)"""
    return (max_capacity_ah / nominal_capacity_ah) * 100.0

# เวลาจำลอง 5 ชั่วโมง
dt_hours = 1.0 # เวลาจำลอง 1 ชั่วโมง
current_energy_ah = 50.0 # พลังงาน 50 Ah (SOC 50% ของความจุ)

print("--- ระบบจำลองพลังงาน Smart Farm ---")
print(f"พารามิเตอร์เริ่มต้น: {NOMINAL_CAPACITY_AH} Ah")
print(f"พารามิเตอร์เริ่มต้น: {current_capacity_ah} Ah")
print(f"พารามิเตอร์เริ่มต้น: {current_a} A และ {SYSTEM_VOLTAGE_V} V\n")

for hour in range(1, 6):
```

```
#  (Ampere-hours change)
delta_ah = current_a * dt_hours
current_energy_ah = min(current_energy_ah + delta_ah, current_capacity_ah)

soc = estimate_soc(current_energy_ah, current_capacity_ah)
energy_wh = current_energy_ah * SYSTEM_VOLTAGE_V

print(f" : ")
    = {current_energy_ah:.2f} Ah | SOC = {soc:.2f}% | 
    = {energy_wh:.2f} Wh")
```

21.2 3.2 บอร์ดฝังตัว ESP32, ไดรเวอร์ CH340 และ สิ่งแวดล้อมโปรแกรม Arduino IDE

Chapter 22

3.2 บอร์ดฝังตัว ESP32, ไดรเวอร์ CH340 และสิ่ง แวดล้อมโปรแกรม Arduino IDE

ในฐานะผู้เชี่ยวชาญด้านฟิสิกส์ประยุกต์และระบบเกษตรดิจิทัล (Digital Agriculture) การทำความเข้าใจกับสถาปัตยกรรมของบอร์ดฝังตัว (Embedded Board) เป็นรากฐานสำคัญในการพัฒนาระบบ Internet of Things (IoT) และระบบอัตโนมัติ (Automation System) บทที่ 3.2 นี้จะเจาะลึกถึงองค์ประกอบหลักของบอร์ด ESP32, ความสำคัญของตัวแปลงสัญญาณ (Converter Chip) เช่น CH340, และการตั้งค่าสภาพแวดล้อมการพัฒนา (Development Environment) ด้วย Arduino IDE อย่างละเอียดถี่ถ้วน

22.1 3.2.1 สถาปัตยกรรมของไมโครคอนโทรลเลอร์ ESP32

ESP32 เป็นไมโครคอนโทรลเลอร์ (Microcontroller Unit: MCU) ที่พัฒนาโดย Espressif Systems ซึ่งถูกออกแบบมาเพื่อตอบสนองต่อการประมวลผลในระบบ IoT โดยเฉพาะ จุดเด่นที่สำคัญที่สุดของ ESP32

คือการรวมเอาคุณสมบัติการเชื่อมต่อไร้สาย (Wireless Connectivity) เข้ากับหน่วยประมวลผลหลัก (Central Processing Unit: CPU) ในชิปเดียว

22.1.1 3.2.1.1 โครงสร้างทางสถาปัตยกรรม (Architectural Structure)

ESP32 ถูกสร้างขึ้นบนสถาปัตยกรรมแบบ Tensilica Xtensa LX6 ซึ่งเป็นสถาปัตยกรรมแบบ 32 บิต (32-bit architecture) การใช้สถาปัตยกรรม 32 บิตหมายความว่า MCU สามารถจัดการกับข้อมูล (Data) ได้ทีละ 32 บิต ซึ่งช่วยเพิ่มประสิทธิภาพในการประมวลผล (Processing Efficiency) อย่างมาก เมื่อเทียบกับสถาปัตยกรรม 8 บิต หรือ 16 บิต

องค์ประกอบหลักทางสถาปัตยกรรมของ ESP32 ประกอบด้วย:

1. หน่วยประมวลผลกลาง (CPU Core): ESP32 มักจะมีแกนประมวลผลหลัก (Core) อย่างน้อยหนึ่งแกน (Core) และสามารถมีแกนที่สองได้ (Dual-core) โดยแต่ละแกนสามารถทำงานแบบอิสระ (Independent Operation) ซึ่งช่วยให้สามารถรันงานหลายอย่างพร้อมกัน (Multitasking) ได้ เช่น แกนหนึ่งจัดการการเชื่อมต่อ Wi-Fi ขณะที่อีกแกนจัดการการอ่านค่าเซนเซอร์ (Sensor Reading)
2. โมดูลการสื่อสารไร้สาย (Wireless Modules): ESP32 มีโมดูล Wi-Fi (IEEE 802.11 b/g/n) และ Bluetooth (Classic Bluetooth และ Bluetooth Low Energy: BLE) ในตัว ซึ่งช่วยลดความซับซ้อนในการออกแบบวงจรภายนอก (External Circuitry) และทำให้บอร์ดมีขนาดเล็กลง
3. หน่วยความจำ (Memory):
4. RAM (Random Access Memory): ใช้สำหรับเก็บข้อมูลที่กำลังถูกประมวลผลชั่วคราว (Temporary Data)

5. Flash Memory: ใช้สำหรับเก็บโค้ดโปรแกรม (Firmware) และข้อมูลถาวร (Persistent Data)
6. ส่วนต่อประสาน (Peripherals): ประกอบด้วย GPIO, ADC, DAC, I²C, SPI, UART และ Timer ซึ่งเป็นส่วนที่ใช้ในการติดต่อสื่อสารกับอุปกรณ์ภายนอก (External Devices)

22.1.2 3.2.1.2 หลักการทำงานของ Wi-Fi และ Bluetooth

การรวมโมดูลไร้สายเข้ากับ MCU ทำให้ ESP32 สามารถทำหน้าที่เป็นโหนด (Node) ในเครือข่าย IoT ได้ทันที

- Wi-Fi: ทำงานตามมาตรฐาน IEEE 802.11 โดยใช้คลื่นความถี่ 2.4 GHz มีอัตราการส่งข้อมูล (Data Rate) ที่เหมาะสมสำหรับการส่งข้อมูลขนาดเล็กถึงปานกลาง เช่น การส่งค่าเซนเซอร์ไปยังคลาวด์ (Cloud)
- Bluetooth: ใช้สำหรับการสื่อสารระยะใกล้ (Short-range communication) เหมาะสำหรับการเชื่อมต่อกับอุปกรณ์ปลายทาง (End Devices) เช่น โทรศัพท์มือถือ หรือเซนเซอร์เฉพาะจุด

22.2 3.2.2 การทำความเข้าใจแผนผังขาพิน (Pinout Map) ของ ESP32

การทำความเข้าใจว่าขาพินใดทำหน้าที่อะไรเป็นสิ่งสำคัญอย่างยิ่งในการเชื่อมต่อวงจรภายนอก (External Circuit Interfacing) ESP32 มีขาพินที่หลากหลายประเภท ซึ่งแต่ละประเภทมีข้อจำกัดในการใช้งานที่แตกต่างกัน

ประเภทขาพิน	คำย่อ	หน้าที่หลัก	คำอธิบายทางเทคนิค
General Purpose Input/Output	GPIO	ขาอินพุต/ เอาต์พุตดิจิทัล	สามารถตั้งค่าให้เป็นเอาต์พุต (Output) เพื่อขับสัญญาณดิจิทัล (Digital Signal) หรือเป็นอินพุต (Input) เพื่อรับสัญญาณดิจิทัล
Analog-to-Digital Converter	ADC	การแปลงสัญญาณอนาล็อกเป็นดิจิทัล	ใช้สำหรับวัดค่าแรงดันไฟฟ้า (Voltage) หรือสัญญาณต่อเนื่อง (Continuous Signal) จากเซนเซอร์ เช่น อุณหภูมิ ความชื้น
Power Supply	V_{CC} , GND	การจ่ายกำลังไฟฟ้า	ขาสำหรับจ่ายแรงดันไฟฟ้าหลัก (Power Rail) และกราวด์ (Ground Reference)

22.2.1 3.2.2.1 ประเภทของขาพิน (Pin Types)

22.2.2 3.2.2.2 การวิเคราะห์ขา GPIO และ ADC

1. ขา GPIO (General Purpose Input/Output): ขาเหล่านี้สามารถกำหนดค่าได้ทั้งเป็นอินพุตและเอาต์พุต โดยทั่วไปแล้วสัญญาณดิจิทัลที่ส่งออกจะมีระดับแรงดันไฟฟ้า (Voltage Level) ที่ 3.3 V (โวลต์) ซึ่งเป็นแรงดันไฟฟ้ามาตรฐานของ ESP32 การขับโหลด (Load) ที่ใช้แรงดันไฟฟ้า 5 V จำเป็นต้องใช้ระดับตัวแปลงสัญญาณ (Level Shifter) เพื่อป้องกันความเสียหายต่อ MCU

2. ขา ADC (Analog-to-Digital Converter): ESP32 มี ADC ที่มีความละเอียด (Resolution) มักอยู่ที่ 12 บิต (bits) ซึ่งหมายความว่าสามารถวัดค่าแรงดันไฟฟ้าในช่วงที่กำหนด (เช่น 0 V ถึง 3.3 V) และแปลงเป็นค่าดิจิทัลจำนวนเต็ม (Integer Value) ได้ $2^{12} = 4096$ ระดับ

หลักการคำนวณค่าดิจิทัลจาก ADC: หากกำหนดให้ V_{ref} คือแรงดันอ้างอิง (Reference Voltage) และ V_{in} คือแรงดันไฟฟ้าที่วัดได้ สูตรการแปลงค่าดิจิทัล D คือ:

$$D = \text{round} \left(\frac{V_{in}}{V_{ref}} \times 2^N - 1 \right)$$

โดยที่ N คือความละเอียดของ ADC (ในที่นี้คือ 12 บิต)

ข้อควรระวัง: การใช้งาน ADC ต้องคำนึงถึงข้อจำกัดของแรงดันไฟฟ้าที่ป้อนเข้า (Input Voltage Range) และความแม่นยำ (Accuracy) ของ ADC ในแต่ละรุ่นของบอร์ด

22.3 3.2.3 ความสำคัญของชิปแปลงสัญญาณ USB-to-Serial (CH340 Driver)

แม้ว่า ESP32 จะเป็นไมโครคอนโทรลเลอร์ที่ทรงพลัง แต่การสื่อสารกับคอมพิวเตอร์ (Host Computer) เพื่อการเขียนโปรแกรม (Programming) และการดีบั๊ก (Debugging) จำเป็นต้องมีส่วนประกอบที่ทำหน้าที่

เป็น ตัวแปลงสัญญาณ USB-to-Serial

22.3.1 3.2.3.1 บทบาทของ USB-to-Serial Converter

คอมพิวเตอร์ส่วนใหญ่สื่อสารกับอุปกรณ์ภายนอกผ่านพอร์ต USB ซึ่งใช้โปรโตคอล (Protocol) ที่ซับซ้อนกว่าการสื่อสารแบบ Serial (เช่น UART) โดยตรง

- Serial Communication (UART): เป็นการส่งข้อมูลแบบลำดับ (Sequential Data) ผ่านสายสัญญาณสองเส้น (TX/RX) โดยมีอัตราการส่งข้อมูล (Baud Rate) ที่กำหนด
- USB-to-Serial Chip (เช่น CH340, FTDI, CP2102): ชิปเหล่านี้ทำหน้าที่เป็นสะพาน (Bridge) โดยรับสัญญาณดิจิทัลความเร็วสูงจาก USB (USB Protocol) และแปลงให้เป็นสัญญาณระดับแรงดันไฟฟ้า (Voltage Level) และรูปแบบสัญญาณ (Signal Format) ที่ MCU สามารถเข้าใจได้ในรูปแบบ Serial (UART)

CH340: เป็นชิปที่ได้รับความนิยมอย่างสูงเนื่องจากมีราคาถูกและมีประสิทธิภาพในการแปลงสัญญาณ USB เป็น UART ทำให้บอร์ดพัฒนาจำนวนมากใช้ชิปนี้เป็นตัวเชื่อมต่อระหว่าง ESP32 กับคอมพิวเตอร์

22.3.2 3.2.3.2 ขั้นตอนการติดตั้งไดรเวอร์ (Driver Installation)

เพื่อให้ระบบปฏิบัติการ (Operating System: OS) เช่น Windows หรือ macOS สามารถ "มองเห็น" และสื่อสารกับบอร์ด ESP32 ได้อย่างถูกต้อง จำเป็นต้องติดตั้งไดรเวอร์ (Driver) ของชิปแปลงสัญญาณ CH340 ก่อน

ขั้นตอนโดยสังเขป (สำหรับ Windows OS):

1. การเชื่อมต่อ: เสียบสาย USB เข้ากับบอร์ด ESP32 และเสียบปลายอีกด้านเข้ากับพอร์ต USB ของคอมพิวเตอร์

2. การตรวจจับ: ระบบปฏิบัติการจะตรวจพบอุปกรณ์ใหม่ แต่เนื่องจากไม่มีไดรเวอร์ที่เหมาะสม จึงอาจแสดงข้อความแจ้งเตือนว่า "Unknown Device"
3. การติดตั้งไดรเวอร์: ผู้ใช้จะต้องดาวน์โหลดไฟล์ไดรเวอร์ CH340 (มักเป็นไฟล์ .zip หรือ .exe) จากเว็บไซต์ผู้ผลิตที่เชื่อถือได้
4. การดำเนินการ: ดับเบิลคลิกที่ไฟล์ติดตั้งและทำตามขั้นตอนบนหน้าจอ (Next → Finish)
5. การตรวจสอบ: หลังจากติดตั้งเสร็จสิ้น ให้ตรวจสอบใน Device Manager (ตัวจัดการอุปกรณ์) ของ Windows ภายใต้หมวดหมู่ "Ports (COM & LPT)" ควรปรากฏชื่ออุปกรณ์ที่ระบุว่า "USB-SERIAL CH340" หรือคล้ายกัน พร้อมหมายเลขพอร์ต (เช่น COM3) ซึ่งหมายความว่า การสื่อสารทางกายภาพ (Physical Communication) พร้อมใช้งานแล้ว

22.4 3.2.4 การตั้งค่าสภาพแวดล้อมการพัฒนา Arduino IDE

Arduino IDE (Integrated Development Environment) เป็นเครื่องมือที่ช่วยให้ผู้ใช้สามารถเขียนโค้ด (Sketch Code) อัปโหลด (Upload) และตรวจสอบ (Monitor) การทำงานของไมโครคอนโทรลเลอร์ได้อย่างง่ายดาย

22.4.1 3.2.4.1 การเพิ่มการรองรับบอร์ด ESP32 ผ่าน Boards Manager

โดยค่าเริ่มต้น Arduino IDE อาจรองรับเฉพาะบอร์ด Arduino รุ่นเก่าเท่านั้น การจะให้ IDE สามารถคอมไพล์ (Compile) และอัปโหลดโค้ดไปยัง ESP32 ได้ ผู้ใช้ต้องเพิ่มการสนับสนุนบอร์ด (Board Support Package: BSP) ของ ESP32 เข้าไปใน IDE

ขั้นตอนการเพิ่ม BSP:

1. การเพิ่ม URL Repository: เข้าไปที่เมนู `File → Preferences` (หรือ `Arduino → Preferences` ใน macOS) และเพิ่ม URL ของ Board Manager สำหรับ ESP32 (เช่น `https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json`) ในช่อง "Additional Boards Manager URLs"
2. การติดตั้ง Board Package: เข้าไปที่เมนู `Tools → Board → Boards Manager`
3. การค้นหาและติดตั้ง: ในช่องค้นหา ให้พิมพ์ "esp32" และคลิกที่ปุ่ม "Install" เพื่อดาวน์โหลดและติดตั้งไลบรารีและคอมไพเลอร์ที่จำเป็นสำหรับ ESP32

22.4.2 3.2.4.2 โครงสร้างหลักของโค้ด (The Sketch Structure)

โค้ดโปรแกรมที่เขียนสำหรับไมโครคอนโทรลเลอร์ส่วนใหญ่จะถูกจัดโครงสร้างตามหลักการทำงานที่กำหนดไว้ในฟังก์ชันหลักสองฟังก์ชัน คือ `setup()` และ `loop()`

1. ฟังก์ชัน `setup()`:

- วัตถุประสงค์: ฟังก์ชันนี้จะถูกเรียกใช้เพียง ครั้งเดียว (Once) เมื่อบอร์ดได้รับพลังงาน (Power Up) หรือถูกรีเซ็ต (Reset)
- การใช้งาน: ใช้สำหรับกำหนดค่าเริ่มต้น (Initialization) ของระบบทั้งหมด เช่น การกำหนดโหมดของขาพิน (Pin Mode: `INPUT` หรือ `OUTPUT`), การเริ่มต้นการสื่อสาร (Serial Communication), และการเชื่อมต่อเครือข่าย (Wi-Fi Connection)

2. ฟังก์ชัน `loop()`:

- วัตถุประสงค์: ฟังก์ชันนี้คือหัวใจของการทำงานของโปรแกรม มันจะถูกเรียกใช้ซ้ำๆ (Repeatedly) อย่างต่อเนื่องด้วยความถี่สูงสุดเท่าที่ MCU จะทำได้ (High Frequency) ตราบใดที่บอร์ดยังมีพลังงาน

- การใช้งาน: ใช้สำหรับใส่ตรรกะการทำงานหลัก (Main Logic) ของระบบ เช่น การอ่านค่าเซนเซอร์, การประมวลผลข้อมูล, และการส่งข้อมูลผ่านเครือข่าย

22.4.3 3.2.4.3 ตัวอย่างโค้ด C++ สำหรับการทดสอบการทำงานพื้นฐาน (Blink Test)

ตัวอย่างโค้ดนี้แสดงการกระพริบไฟ LED ที่ต่อกับขา GPIO 2 (ซึ่งเป็นขาที่มักจะเชื่อมต่อกับ LED บนบอร์ด ESP32 DevKit) เพื่อยืนยันว่าการตั้งค่าบอร์ดและการอัปโหลดโค้ดทำงานได้อย่างสมบูรณ์

```
// =====  
// ไฟล์ esp32_blink_test.ino  
// วัตถุประสงค์: ทดสอบการทำงานของ LED ที่เชื่อมต่อกับขา GPIO 2 ของ ESP32  
// ภาษา: C++ (Arduino Framework)  
// =====  
  
// กำหนด GPIO ที่เชื่อมต่อกับ LED (ESP32 DevKit ใช้ GPIO 2)  
const int ledPin = 2;  
  
// ฟังก์ชัน setup(): เริ่มต้นระบบ (Initialization)  
void setup() {  
    // 1. เริ่มต้นการสื่อสารอนุกรม (Serial Communication)  
    // Baud Rate 115200 bps: อัตราการส่งข้อมูล 115,200 บิตต่อวินาที  
    // เริ่มต้นการสื่อสารอนุกรม  
    Serial.begin(115200);  
    Serial.println("--- ESP32 Initializing System ---");  
  
    // 2. กำหนดโหมดการทำงาน (Pin Mode Configuration)  
    // กำหนดขา ledPin ให้เป็นโหมดเอาต์พุต (Output) เพื่อควบคุม LED  
    pinMode(ledPin, OUTPUT);  
    Serial.println("GPIO Pin " + String(ledPin) + " configured as OUTPUT.");  
}  
  
// ฟังก์ชัน loop(): วนซ้ำการทำงานหลัก (Main Program Loop)  
void loop() {  
    // 1. ตั้งสถานะ HIGH (High Voltage State)  
    // ส่งสัญญาณ HIGH (3.3 V) ไปยัง LED เพื่อเปิดไฟ  
    digitalWrite(ledPin, HIGH);  
    Serial.println("LED ON. Time: " + String(millis() / 1000) + " s");  
  
    // 2. ฟังก์ชันหน่วงเวลา (Delay Function)  
    // หน่วงเวลา 500 มิลลิวินาที (ms) หรือ 0.5 วินาที  
    delay(500);  
  
    // 3. ตั้งสถานะ LOW (Low Voltage State)  
    // ส่งสัญญาณ LOW (0 V) ไปยัง LED เพื่อปิดไฟ  
    digitalWrite(ledPin, LOW);  
    Serial.println("LED OFF. Time: " + String(millis() / 1000) + " s");  
}
```


Chapter 23

3.3 การเขียนโค้ดอ่านเซนเซอร์ วิเคราะห์กราฟคาลิเบรตดิน เหนียวตราด (Soil Moisture Sensor Calibration for Trat Clay)

ในฐานะนักวิชาการผู้เชี่ยวชาญด้านฟิสิกส์ประยุกต์และเกษตรดิจิทัล การทำความเข้าใจหลักการทางฟิสิกส์ที่อยู่เบื้องหลังการวัดค่าความชื้นในดิน (Soil Moisture Measurement) เป็นสิ่งสำคัญยิ่งก่อนการเขียนโค้ดใด ๆ การวัดค่าความชื้นในดินที่แม่นยำและเชื่อถือได้นั้น ไม่ได้อาศัยเพียงแค่การอ่านค่าดิจิทัล (Digital Reading) เท่านั้น แต่ต้องอาศัยการทำความเข้าใจปฏิสัมพันธ์ระหว่างสนามไฟฟ้า (Electric Field) กับคุณสมบัติทางไฟฟ้าของตัวกลาง (Dielectric Properties) อย่างลึกซึ้ง

บทนี้จะนำเสนอการวิเคราะห์เชิงทฤษฎีและปฏิบัติอย่างละเอียด ตั้งแต่หลักการการทำงานของเซนเซอร์วัดความชื้นแบบเก็บประจุ (Capacitive Soil Moisture Sensor) ไปจนถึงการสร้างแบบจำลองทางคณิตศาสตร์ (Mathematical Model) สำหรับการสอบเทียบ (Calibration) เพื่อให้ได้ค่าความชื้นดินเชิงปริมาตร (Volumetric Water Content: VWC) ที่มีความแม่นยำสูงสำหรับสภาพดินเหนียวเฉพาะถิ่นอย่างดินเหนียวตราด

23.1 3.3.1 หลักการทำงานทางฟิสิกส์ของเซนเซอร์วัดความชื้นแบบเก็บประจุ (Capacitive Principle)

23.1.1 3.3.1.1 ความแตกต่างระหว่างเซนเซอร์แบบต้านทานและแบบเก็บประจุ

เซนเซอร์วัดความชื้นในดินในตลาดมีหลายประเภท แต่สำหรับงานวิจัยและงานภาคสนามที่ต้องการความทนทานสูงและค่าที่เสถียรในระยะยาว เซนเซอร์แบบเก็บประจุ (Capacitive Sensor) ถือเป็นทางเลือกที่เหนือกว่าเซนเซอร์แบบต้านทาน (Resistive Sensor) อย่างชัดเจน

1. เซนเซอร์แบบต้านทาน (Resistive Sensor): เซนเซอร์เหล่านี้ทำงานโดยการวัดความต้านทานไฟฟ้า (Electrical Resistance) ระหว่างขั้วไฟฟ้าสองขั้ว เมื่อดินเปียก ความต้านทานจะลดลง เนื่องจากน้ำเป็นตัวนำไฟฟ้าที่ดีกว่าอากาศ อย่างไรก็ตาม ข้อเสียหลักคือการเกิดการกัดกร่อน (Corrosion) ของขั้วไฟฟ้าอย่างรวดเร็วเมื่อสัมผัสกับอิเล็กโทรไลต์ (Electrolyte) ในดินเป็นเวลานาน ทำให้ค่าที่วัดได้ผิดเพี้ยนและอายุการใช้งานสั้น
2. เซนเซอร์แบบเก็บประจุ (Capacitive Sensor): เซนเซอร์เหล่านี้ไม่ได้วัดความต้านทาน แต่จะวัดค่าความจุไฟฟ้า (Capacitance, C) ซึ่งเป็นคุณสมบัติทางไฟฟ้าที่เกิดจากการกักเก็บประจุไฟฟ้า (Electric Charge) ระหว่างแผ่นตัวนำ (Conducting Plates) สองแผ่นที่ถูกแยกด้วยไดอิเล็กทริก (Dielectric) ซึ่งในกรณีนี้คือดินรอบ ๆ เซนเซอร์

23.1.2 3.3.1.2 กลไกทางฟิสิกส์ของการวัดค่าความจุไฟฟ้า

เซนเซอร์แบบเก็บประจุถูกออกแบบให้มีโครงสร้างเป็นตัวเก็บประจุ (Capacitor) โดยมีแผ่นตัวนำไฟฟ้า (Electrodes) สองแผ่นที่ขนานกัน และมีตัวกลาง (Medium) คือดินที่อยู่ระหว่างแผ่นตัวนำ

ค่าความจุไฟฟ้า (C) ของตัวเก็บประจุแบบขนาน (Parallel Plate Capacitor) ถูกกำหนดโดยสมการพื้นฐานทางไฟฟ้าสถิต (Electrostatics):

$$C = \frac{\epsilon_r \epsilon_0 A}{d}$$

เมื่อ:

- C คือ ค่าความจุไฟฟ้า (Capacitance) มีหน่วยเป็น ฟารัด (Farad, F)
- ϵ_0 คือ ค่าสภาพยอมทางไฟฟ้าของสุญญากาศ (Permittivity of Free Space) มีค่าคงที่เท่ากับ 8.854×10^{-12} F/m
- A คือ พื้นที่ผิวของแผ่นตัวนำ (Area) มีหน่วยเป็น ตารางเมตร (m^2)
- d คือ ระยะห่างระหว่างแผ่นตัวนำ (Distance) มีหน่วยเป็น เมตร (m)
- ϵ_r คือ ค่าคงที่ไดอิเล็กทริก (Relative Permittivity) หรือค่าคงที่ไดอิเล็กทริกสัมพัทธ์ (Dielectric Constant) ของตัวกลางที่อยู่ระหว่างแผ่นตัวนำ

การวิเคราะห์ความสัมพันธ์กับความชื้น:

ในบริบทของเซนเซอร์วัดความชื้นในดิน ตัวกลางที่อยู่ระหว่างแผ่นตัวนำคือดิน (Soil) ดังนั้น ค่า ϵ_r จึงเป็นค่าคงที่ไดอิเล็กทริกของดิน ($\epsilon_{r,soil}$) ซึ่งเป็นคุณสมบัติที่ขึ้นอยู่กับองค์ประกอบของดินและปริมาณน้ำในดิน

เมื่อปริมาณน้ำในดินเพิ่มขึ้น (ดินเปียกขึ้น) น้ำจะเข้าไปแทนที่อากาศในช่องว่างของดิน (Pore Space) เนื่องจากน้ำมีค่าคงที่ไดอิเล็กทริก ($\epsilon_{r,water} \approx 80$) สูงกว่าอากาศ ($\epsilon_{r,air} \approx 1$) อย่างมาก การเพิ่มขึ้นของน้ำจึงส่งผลให้ค่า $\epsilon_{r,soil}$ เพิ่มขึ้นตามไปด้วย

ดังนั้น:

$$\text{Soil Moisture} \uparrow \Rightarrow \varepsilon_{r,\text{soil}} \uparrow \Rightarrow C \uparrow$$

เซนเซอร์จึงทำงานโดยการวัดค่าความจุไฟฟ้าที่แปรผันตามค่าคงที่ได้
อิเล็กทริกของดิน ซึ่งเป็นตัวบ่งชี้ถึงปริมาณน้ำในดิน

23.2 3.3.2 ทฤษฎีการสอบเทียบเซนเซอร์ (Sensor Calibration Theory)

23.2.1 3.3.2.1 การแปลงค่าความจุไฟฟ้าเป็นความชื้นเชิงปริมาตร (VWC)

เซนเซอร์สมัยใหม่มักจะแปลงค่าความจุไฟฟ้า (C) ให้เป็นสัญญาณแรงดันไฟฟ้าอนาล็อก (Analog Voltage Output) (V_{out}) ซึ่งเป็นค่าที่ไม่ไครคอนโทรลเลอร์ (Microcontroller) เช่น ESP32 หรือ Arduino สามารถอ่านได้

เนื่องจากค่า C มีความสัมพันธ์เชิงเส้น (Linear Relationship) กับค่า $\varepsilon_{r,\text{soil}}$ และค่า V_{out} ก็มีความสัมพันธ์เชิงเส้นกับ C ดังนั้น เราจึงสามารถสร้างสมการเชิงเส้น (Linear Equation) เพื่อแปลงค่าแรงดันไฟฟ้าที่อ่านได้ (V_{raw}) ให้เป็นเปอร์เซ็นต์ความชื้นดินเชิงปริมาตร (Volumetric Water Content: VWC %) ได้

$$VWC(\%) = f(V_{raw})$$

โดยทั่วไปแล้ว ความสัมพันธ์นี้สามารถประมาณได้ด้วยสมการเชิงเส้นรูปแบบ:

$$VWC = m \cdot ADC_{raw} + c$$

เมื่อ:

- vwc คือ ปริมาณน้ำในดินเชิงปริมาตร (Volumetric Water Content) มีหน่วยเป็นเปอร์เซ็นต์ (%)

- ADC_{raw} คือ ค่าดิจิทัลดิบ (Raw Digital Value) ที่อ่านได้จากตัวแปลงอนาล็อกเป็นดิจิทัล (Analog-to-Digital Converter, ADC) ซึ่งมีช่วงตั้งแต่ 0 ถึง $2^N - 1$ (เช่น 0 ถึง 4095 สำหรับ ADC 12-bit)
- m คือ ความชัน (Slope) ของกราฟการสอบเทียบ (Calibration Slope)
- c คือ จุดตัดแกน Y (Y-intercept) หรือค่าความชันเมื่อ $ADC_{raw} = 0$

23.2.2 3.3.2.2 การกำหนดจุดสอบเทียบ (Defining Calibration Points)

การหาค่า m และ c อย่างแม่นยำต้องอาศัยการวัดค่าในสถานะที่ทราบค่าความชื้นอย่างแน่นอน ซึ่งเราเรียกว่า จุดสอบเทียบ (Calibration Points) โดยเฉพาะสำหรับดินเหนียวตราด (Trat Clay) ซึ่งมีคุณสมบัติทางความชื้นที่เฉพาะเจาะจง เราต้องกำหนดจุดสองจุดหลัก:

1. จุดความชื้นต่ำสุด (Dry Soil Point): คือการวัดค่าเมื่อเซนเซอร์ถูกฝังในดินที่แห้งสนิท (Air-dry state) ณ จุดนี้ vwc ควรมีค่าใกล้เคียง 0%
2. ให้ค่า $ADC_{raw,dry}$ เป็นค่าที่อ่านได้
3. กำหนดให้ $vwc_{dry} = 0\%$
4. จุดความชื้นสูงสุด (Wet Soil Point): คือการวัดค่าเมื่อเซนเซอร์ถูกฝังในดินที่อิ่มตัวด้วยน้ำ (Saturated state) ณ จุดนี้ vwc ควรมีค่าใกล้เคียง 100%
5. ให้ค่า $ADC_{raw,wet}$ เป็นค่าที่อ่านได้
6. กำหนดให้ $vwc_{wet} = 100\%$

23.2.3 3.3.2.3 การคำนวณค่าความชื้นและจุดตัดแกน Y

เราใช้ข้อมูลจากสองจุด ($ADC_{raw,dry}, 0\%$) และ ($ADC_{raw,wet}, 100\%$) เพื่อหาค่า m และ c โดยใช้สูตรความชัน (Slope-Intercept Form) ของสมการเส้นตรง:

****1. การหาค่าความชัน (m):****

$$m = \frac{VWC_{wet} - VWC_{dry}}{ADC_{raw,wet} - ADC_{raw,dry}} = \frac{100\% - 0\%}{ADC_{raw,wet} - ADC_{raw,dry}}$$

****2. การหาจุดตัดแกน Y (c):**** เนื่องจากเราทราบว่าเมื่อ $VWC = 0\%$ ค่า ADC_{raw} คือ $ADC_{raw,dry}$ เราสามารถแทนค่านี้ในสมการ:

$$0 = m \cdot ADC_{raw,dry} + c$$

$$c = -m \cdot ADC_{raw,dry}$$

ดังนั้น สมการการสอบเทียบที่สมบูรณ์สำหรับดินเหนียวตราดคือ:

$$VWC(\%) = \left(\frac{100}{ADC_{raw,wet} - ADC_{raw,dry}} \right) \cdot ADC_{raw} - \left(\frac{100}{ADC_{raw,wet} - ADC_{raw,dry}} \right) \cdot ADC_{raw,dry}$$

23.3 3.3.3 การประยุกต์ใช้โค้ดและการกรองสัญญาณรบกวน (Implementation and Noise Filtering)

ในการอ่านค่าจากเซนเซอร์ในสภาพแวดล้อมจริง ค่าที่อ่านได้จาก ADC มักจะมีสัญญาณรบกวน (Noise) จากการเปลี่ยนแปลงอุณหภูมิ (Temperature Fluctuation) การรบกวนทางแม่เหล็กไฟฟ้า (Electromagnetic Interference) และการเปลี่ยนแปลงของสภาพแวดล้อมอื่น ๆ การใช้เทคนิคการกรองสัญญาณ (Signal Filtering) จึงเป็นสิ่งจำเป็นอย่างยิ่ง

23.3.1 3.3.3.1 การกรองค่าเฉลี่ยเคลื่อนที่ (Moving Average Filter)

Moving Average Filter (MAF) เป็นวิธีการกรองสัญญาณที่ง่ายและมีประสิทธิภาพในการลดสัญญาณรบกวนแบบสุ่ม (Random Noise) โดยการคำนวณค่าเฉลี่ยของชุดข้อมูลที่อ่านได้ในช่วงเวลาสั้น ๆ (Window Size)

ถ้า s_i คือค่าที่อ่านได้ในรอบที่ i และ N คือขนาดของหน้าต่าง (Window Size) ค่าที่กรองแล้ว $\hat{s}_i$ จะคำนวณดังนี้:

$$\hat{s}_i = \frac{1}{N} \sum_{j=0}^{N-1} s_{i-j}$$

การใช้ MAF จะช่วยให้ค่า ADC_{raw} ที่นำไปคำนวณ VWC มีความเสถียรและใกล้เคียงกับค่าจริงทางฟิสิกส์มากขึ้น

23.3.2 3.3.3.2 โค้ดตัวอย่าง C++ สำหรับ Arduino (ESP32)

โค้ดต่อไปนี้เป็นตัวอย่างการเขียนโปรแกรมในภาษา C++ สำหรับแพลตฟอร์ม Arduino (ที่ใช้ไมโครคอนโทรลเลอร์ เช่น ESP32) เพื่อดำเนินการอ่านค่าเซนเซอร์ การกรองสัญญาณ และการแปลงค่าเป็น VWC อย่างสมบูรณ์

```
//  
=====
```

```
// [1]  (Constants and Variables Definition)  
//  
=====
```

```
// Analog Input  (Soil Moisture Sensor)  
// ESP32  ADC  
const int SENSOR_PIN = 34;
```

```
// Moving Average Filter ()  
// N  5  15  
const int FILTER_SIZE = 10;
```

```
// ADC  (Raw ADC Values)  
int raw_adc_buffer[FILTER_SIZE];  
int buffer_index = 0; //
```

```
//  
=====
```

```
// [2] Calibration Parameters
//
// =====

// *** Calibration Parameters (Dry and Wet Soil Points) ***

// Dry Soil Point (0-4095)
const int ADC_RAW_DRY = 1000;

// Wet Soil Point (0-4095)
const int ADC_RAW_WET = 3500;

//
// =====

// [3] Setup and Loop Functions
//
// =====

void setup() {
    // Initialize Serial
    Serial.begin(115200);
    Serial.println("--- Soil Moisture Sensor Calibration System Initialized ---");
    ;
    Serial.println("System ready to measure VWC for Trat Clay.");
}

void loop() {
    // 1. Raw Reading
    int raw_reading = read_sensor_value();

    // 2. Filtering (Moving Average)
    int filtered_adc = calculate_moving_average(raw_reading);

    // 3. VWC Calculation
    float vwc_percentage = calculate_vwc(filtered_adc);

    // 4. Output Display
    Serial.print("-----");
    Serial.println();
    Serial.print("Raw ADC Reading: ");
    Serial.print(raw_reading);
    Serial.print(" | Filtered ADC: ");
    Serial.print(filtered_adc);
    Serial.print(" | VWC (Trat Clay): ");
    Serial.print(vwc_percentage, 2); // Print 2 decimal places
    Serial.println(" %");

    // Delay between readings
    delay(2000);
}

/**
```

```

* @brief  ADC
* @return  ADC
*/
int read_sensor_value() {
    //
    int raw_value = analogRead(SENSOR_PIN);

    //
    raw_adc_buffer[buffer_index] = raw_value;

    // ( 0 )
    buffer_index = (buffer_index + 1) % FILTER_SIZE;

    return raw_value;
}

/**
* @brief  (Moving Average Filter)
* @param latest_reading
* @return  ADC
*/
int calculate_moving_average(int latest_reading) {
    long sum = 0;
    for (int i = 0; i < FILTER_SIZE; i++) {
        sum += raw_adc_buffer[i];
    }
    return sum / FILTER_SIZE;
}

/**
* @brief  VWC ADC
* @param adc_value  ADC
* @return  (% VWC)
*/
float calculate_vwc(int adc_value) {
    // (Out of bounds constraint)
    if (adc_value <= ADC_RAW_DRY) return 0.0;
    if (adc_value >= ADC_RAW_WET) return 100.0;

    //
    float vwc = ((float)(adc_value - ADC_RAW_DRY) / (float)(ADC_RAW_WET -
        ADC_RAW_DRY)) * 100.0;
    return vwc;
}

```

23.4 3.4 ตรรกะระบบควบคุมป้อนกลับ (Feedback Control Loop) สั่งงานวงจรไฟฟ้าปั้มน้ำด้วยรีเลย์

Chapter 24

3.4 ตรรกะระบบควบคุมป้อนกลับ (Feedback Control Loop) สำหรับการสั่งงานวงจรไฟฟ้าปั้มน้ำด้วยรีเลย์

(ระดับความลึก: การวิเคราะห์ระบบควบคุมเชิงวิศวกรรม, การประยุกต์ใช้ไฟฟ้ากำลัง, การเขียนโปรแกรมไมโครคอนโทรลเลอร์)

24.1 บทนำเชิงทฤษฎี: หลักการควบคุมระบบอัตโนมัติ

ในสาขาวิศวกรรมเกษตรดิจิทัล (Digital Agriculture Engineering) การควบคุมสภาพแวดล้อมของพืชผลให้เหมาะสมที่สุดเป็นหัวใจสำคัญของการเพิ่มผลผลิตอย่างยั่งยืน (Sustainable Yield Enhancement) หนึ่งในตัวแปรทางกายภาพที่สำคัญที่สุดที่ต้องได้รับการควบคุมอย่างแม่นยำคือความชื้นในดิน (Soil Moisture)

ระบบควบคุมแบบป้อนกลับ (Feedback Control Loop) คือรากฐานทางทฤษฎีที่ใช้ในการรักษาสภาวะของตัวแปรทางกายภาพ (Process Variable, PV) ให้คงที่ใกล้เคียงกับค่าที่กำหนดไว้ล่วงหน้า (Set Point,

SP) โดยอาศัยการวัดค่าที่เกิดขึ้นจริงและนำความแตกต่างนั้นมาปรับปรุงการทำงานของอุปกรณ์ขับเคลื่อน (Actuator) อย่างต่อเนื่อง

24.1.1 3.4.1 หลักการควบคุมป้อนกลับ (Feedback Control Loop) และระบบ On-Off Controller

3.4.1.1 แนวคิดพื้นฐานของวงจรควบคุมป้อนกลับ

ระบบควบคุมป้อนกลับเป็นวงจรที่ประกอบด้วยองค์ประกอบหลัก 3 ส่วน ได้แก่:

1. ตัวแปรกระบวนการ (Process Variable, PV): คือค่าที่วัดได้จริง ณ จุดปฏิบัติการ เช่น ความชื้นในดิน (Soil Moisture Content) ที่วัดได้จากเซนเซอร์
2. ค่าตั้งต้น (Set Point, SP): คือค่าเป้าหมายที่ต้องการให้ระบบรักษาไว้ เช่น ค่าความชื้นที่เหมาะสมที่สุด (Optimal WVC)
3. ตัวควบคุม (Controller): คือหน่วยประมวลผล (เช่น ไมโครคอนโทรลเลอร์) ที่คำนวณหาความผิดพลาด (Error Signal, $e(t)$) และส่งสัญญาณคำสั่งไปยังตัวขับเคลื่อน

ความผิดพลาด (Error Signal) ถูกนิยามทางคณิตศาสตร์ดังนี้:

$$e(t) = SP - PV$$

เมื่อ $e(t)$ มีค่าเป็นบวก หมายความว่า $PV < SP$ (เช่น ความชื้นต่ำกว่าที่กำหนด) และเมื่อ $e(t)$ มีค่าเป็นลบ หมายความว่า $PV > SP$ (เช่น ความชื้นสูงกว่าที่กำหนด)

3.4.1.2 การควบคุมแบบสองตำแหน่ง (On-Off Control) และปรากฏการณ์ฮิสเทรีซิส (Hysteresis)

ในการควบคุมความชื้นในดินด้วยปั้มน้ำ เรามักใช้กลไกควบคุมแบบ On-Off Controller เนื่องจากมีความเรียบง่ายในการประยุกต์ใช้และมีประสิทธิภาพเพียงพอสำหรับงานเกษตรกรรมส่วนใหญ่

On-Off Controller คือตัวควบคุมที่ให้เอาต์พุตเพียงสองสถานะ คือ "เปิด (ON)" หรือ "ปิด (OFF)" โดยไม่มีการปรับค่าเอาต์พุตอย่างต่อเนื่อง (เช่น PID Controller ที่จะปรับกำลังปั้มน้ำตามความผิดพลาด)

อย่างไรก็ตาม การใช้ On-Off Controller โดยไม่มีการกำหนดขอบเขตที่ชัดเจนจะทำให้เกิดปรากฏการณ์ที่เรียกว่า การสวิง (Cycling) อย่างรวดเร็ว ซึ่งอาจส่งผลเสียต่ออายุการใช้งานของปั้มน้ำและรีเลย์

เพื่อแก้ไขปัญหานี้ เราจึงต้องใช้หลักการ ฮิสเทรีซิส (Hysteresis) ซึ่งคือการกำหนดช่วงค่า (Bandwidth) ของความผิดพลาดที่ยอมให้ระบบทำงานก่อนที่จะสลับสถานะเอาต์พุต

เราจะกำหนดขีดจำกัดสองค่า:

1. ****ขีดเริ่มต่ำสุด (Lower Threshold, T_{low}):**** เมื่อ $PV < T_{low}$ ระบบต้องสั่งเปิดปั้มน้ำ (Pump ON)
2. ****ขีดเริ่มสูงสุด (Upper Threshold, T_{high}):**** เมื่อ $PV > T_{high}$ ระบบต้องสั่งปิดปั้มน้ำ (Pump OFF)

โดยที่ $T_{high} > T_{low}$ เพื่อให้เกิดช่วงความคลาดเคลื่อน (Deadband) ที่เหมาะสม
สมการตรรกะการควบคุม (Control Logic):

IF ($PV < T_{low}$) THEN Output = ON

ELSE IF ($PV > T_{high}$) THEN Output = OFF

ELSE Output = Maintain Current State

24.1.2 3.4.2 การเชื่อมต่อทางไฟฟ้า: รีเลย์และหลักการแยกสัญญาณ (Isolation)

ในทางปฏิบัติ ระบบควบคุมที่ใช้ไมโครคอนโทรลเลอร์ (Microcontroller) ซึ่งทำงานด้วยแรงดันไฟฟ้าต่ำ (Low Voltage, เช่น 3.3 V หรือ 5 V) ไม่สามารถจ่ายกำลังไฟฟ้าที่สูงพอที่จะขับเคลื่อนปั้มน้ำ (ซึ่งอาจใช้แรงดันไฟฟ้ากระแสสลับ AC 220 V หรือ DC 12 V) ได้โดยตรง

ดังนั้น เราจึงต้องใช้ โมดูลรีเลย์ (Relay Module) ซึ่งทำหน้าที่เป็น สวิตช์ทางไฟฟ้า (Electromechanical Switch) ที่แยกวงจรควบคุมออกจากวงจรโหลด (Load Circuit) อย่างสมบูรณ์

3.4.2.1 หลักการทำงานของรีเลย์ (Relay Operation)

รีเลย์ประกอบด้วยขดลวดแม่เหล็กไฟฟ้า (Electromagnet Coil) และ หน้าสัมผัส (Contacts) เมื่อจ่ายกระแสไฟฟ้ากระแสตรง (DC) ไปยังขดลวด จะเกิดสนามแม่เหล็ก (Magnetic Field) ดึงดูดหน้าสัมผัสให้ปิดวงจร ทำให้กระแสไฟฟ้าแรงสูงสามารถไหลผ่านหน้าสัมผัสไปยังโหลดได้

3.4.2.2 การแยกสัญญาณด้วยออปโตคัปเปิลอร์ (Optocoupler Isolation)

ในระบบควบคุมที่ซับซ้อน การเชื่อมต่อวงจรควบคุม (Microcontroller) เข้ากับวงจรขับเคลื่อน (Relay Coil) โดยตรงอาจทำให้เกิดปัญหาทางไฟฟ้าหลายประการ เช่น:

1. Ground Loop: ความต่างศักย์ของกราวด์ที่อาจทำให้เกิดสัญญาณรบกวน (Noise)
2. Voltage Spike: ไฟกระชากกลับ (Back EMF) เมื่อขดลวดแม่เหล็กไฟฟ้าถูกตัดกระแส

เพื่อป้องกันปัญหานี้ เราจึงต้องใช้ ออปโตคัปเปิลอร์ (Optocoupler) หรือ ออปโตไอโซเลเตอร์ (Optoisolator) ซึ่งเป็นอุปกรณ์ที่ใช้หลักการส่งสัญญาณด้วยแสง (Optical Signal Transmission)

หลักการ: ออปโตคัปเปิลอร์จะแยกวงจรไฟฟ้าควบคุม (Input Side) ออกจากวงจรขับเคลื่อน (Output Side) อย่างสมบูรณ์ด้วยการใช้แสงเป็นตัวกลางในการส่งสัญญาณ ทำให้เกิด การแยกทางไฟฟ้า (Galvanic Isolation) ซึ่งเป็นมาตรการด้านความปลอดภัยทางไฟฟ้าที่สำคัญยิ่ง

3.4.2.3 ปรากฏการณ์ Active Low

โมดูลรีเลย์ส่วนใหญ่ที่ใช้กับไมโครคอนโทรลเลอร์ถูกออกแบบมาให้ทำงานในโหมด Active Low หมายความว่า:

- สถานะ OFF (ปกติ): ขาควบคุม (Control Pin) ของรีเลย์จะถูกดึงให้เป็นระดับแรงดันไฟฟ้าสูง (High Voltage, V_{CC}) หรือถูกปล่อยให้ลอย (Floating)
- สถานะ ON (ทำงาน): ต้องจ่ายสัญญาณแรงดันไฟฟ้าต่ำ (Low Voltage, GND) หรือระดับแรงดันไฟฟ้าที่ต่ำกว่า V_{CC} เข้าไปที่ขาควบคุม เพื่อให้เกิดการทำงานของรีเลย์

ดังนั้น ในการเขียนโค้ด เราจะต้องกำหนดให้สถานะ "เปิด" (Activate) คือการส่งสัญญาณ LOW ไปยังขาควบคุมของรีเลย์

24.1.3 3.4.3 การออกแบบตรรกะระบบควบคุมเชิงปฏิบัติ (Algorithmic Design)

การเขียนโปรแกรมควบคุมต้องคำนึงถึงประสิทธิภาพและความเสถียรของระบบเป็นหลัก โดยเฉพาะอย่างยิ่งการหลีกเลี่ยงการหยุดการทำงานของโปรแกรม (Blocking) ซึ่งจะทำให้ระบบไม่สามารถอ่านค่าเซนเซอร์อื่น ๆ หรือตอบสนองต่อเหตุการณ์อื่น ๆ ได้

เราจะใช้เทคนิค Non-blocking delay โดยอาศัยฟังก์ชัน `millis()` เพื่อติดตามเวลาที่ผ่านไปแทนการใช้ฟังก์ชัน `delay()` แบบดั้งเดิม

3.4.3.1 โครงสร้างการทำงานของโปรแกรม (Program Flow)

1. Initialization: กำหนดค่าขีดจำกัด (T_{low}, T_{high}) และกำหนดสถานะเริ่มต้นของรีเลย์ (OFF)
2. Loop Cycle: ทำซ้ำอย่างต่อเนื่อง a. Read Sensor: อ่านค่าความชื้นในดินเชิงปริมาตร (Volumetric Water Content, VWC) จากเซนเซอร์ b. Check Threshold: เปรียบเทียบค่า VWC ที่อ่านได้กับ T_{low} และ T_{high} c. Control Logic Execution:
3. ถ้า $VWC < T_{low}$: สั่งเปิดรีเลย์ (Activate Pump)
4. ถ้า $VWC > T_{high}$: สั่งปิดรีเลย์ (Deactivate Pump)
5. ถ้าอยู่ในช่วง $T_{low} \leq VWC \leq T_{high}$: คงสถานะเดิม (Maintain State) d. Non-blocking Delay: หน่วงเวลาการวนรอบ (Cycle Time) โดยไม่บล็อกการทำงานของไมโครคอนโทรลเลอร์

24.1.4 3.4.4 การประยุกต์ใช้โค้ด (C++ Arduino Implementation)

เราจะใช้ภาษา C++ สำหรับแพลตฟอร์ม Arduino ซึ่งเป็นมาตรฐานในการพัฒนาโปรเจกต์ IoT และระบบควบคุมขนาดเล็ก

สมมติฐานของฮาร์ดแวร์:

- Sensor Pin: ขาที่เชื่อมต่อกับเซนเซอร์วัดความชื้น (Analog Input)
- Relay Pin: ขาควบคุมที่เชื่อมต่อกับโมดูลรีเลย์ (Digital Output)

```
//
// =====

// [.. Academic Code Listing]
// 3.4 Feedback Control Loop (Feedback Control Loop)
// C++ (Arduino Sketch)
// Hysteresis Non-blocking delay
//
// =====

// --- 1. Constants Definition ---
// GPIO Pin Definitions
const int SENSOR_PIN = A0; // Analog Pin (Soil Moisture Sensor)
const int RELAY_PIN = 7; // Digital Pin (Relay Module)

// Thresholds % VWC (Volumetric Water Content)
const float LOWER_THRESHOLD = 35.0; // T_low: ( )
const float UPPER_THRESHOLD = 65.0; // T_high: ( )

// Cycle Time (ms)
const long CONTROL_CYCLE_INTERVAL = 5000; // 5 seconds

// --- 2. State Variables ---
// Non-blocking delay mechanism
unsigned long previousMillis = 0;

// Pump State
// State Machine Logic
bool isPumpActive = false;

//
// =====

// Setup: Initialization
//
// =====

void setup() {
    // Serial Monitor (Debugging/Monitoring)
    Serial.begin(9600);
    Serial.println("=====");
    Serial.println("Feedback Control System");
    Serial.println("=====");

    // OUTPUT
    pinMode(RELAY_PIN, OUTPUT);

    // OFF (Deactivate)
    // Active Low, HIGH pin is OFF
    digitalWrite(RELAY_PIN, HIGH);

    Serial.println("System Initialized. Pump Status: OFF.");
}
```

```
//
=====

// ควบคุม Loop: ควบคุมการทำงาน (Main Control Loop)
//
=====

void loop() {
    // 1. ตรวจสอบเวลา (Timing Check)
    // unsigned long millis() ให้อ่านค่าเวลาที่ผ่านไปนับจากเริ่มทำงาน
    unsigned long currentMillis = millis();

    if (currentMillis - previousMillis >= CONTROL_CYCLE_INTERVAL) {
        // ควบคุมการทำงานตามเวลาที่กำหนด
        previousMillis = currentMillis;

        // 2. อ่านค่าจากเซ็นเซอร์ (Sensor Reading)
        // อ่านค่าความชื้นดิน (VWC) จากเซ็นเซอร์
        // อ่านค่าจาก readSoilMoistureVWC() ให้อ่านค่า ADC และแปลง
        // ค่าเป็น VWC (0-100%)
        float currentVWC = readSoilMoistureVWC();

        // 3. ตรรกะการควบคุมแบบมีฮิสเทรีซิส (Feedback Control Logic with Hysteresis)
        if (currentVWC < LOWER_THRESHOLD) {
            // ความชื้นดินต่ำเกินไป -> เปิดปั๊ม
            isPumpActive = true;
            // ควบคุมการทำงาน (Active Low: LOW หมายความว่าเปิดปั๊ม)
            digitalWrite(RELAY_PIN, LOW);
            Serial.print("[PUMP ALERT] Soil is too dry! VWC: ");
            Serial.print(currentVWC);
            Serial.println("% | Pump Status: ON");
        }
        else if (currentVWC > UPPER_THRESHOLD) {
            // ความชื้นดินสูงเกินไป -> ปิดปั๊ม
            isPumpActive = false;
            // ควบคุมการทำงาน (Active Low: HIGH หมายความว่าปิดปั๊ม)
            digitalWrite(RELAY_PIN, HIGH);
            Serial.print("[PUMP ALERT] Soil is saturated. VWC: ");
            Serial.print(currentVWC);
            Serial.println("% | Pump Status: OFF");
        }
        else {
            // ความชื้นดินอยู่ในช่วงที่เหมาะสม (LOWER_THRESHOLD <= VWC <= UPPER_THRESHOLD)
            // ควบคุมการทำงาน (Hysteresis Deadband)
            Serial.print("[SYSTEM OK] Soil moisture is stable. VWC: ");
            Serial.print(currentVWC);
            Serial.print("% | Pump Status: ");
            Serial.println(isPumpActive ? "ON" : "OFF");
        }
    }
}

/**
 * @brief อ่านค่าความชื้นดินจากเซ็นเซอร์ (VWC)
 * @return ค่าความชื้นดินเป็นเปอร์เซ็นต์
 */
float readSoilMoistureVWC() {
```

```
int rawADC = analogRead(SENSOR_PIN);  
// ADC 10-bit (0 - 1023)  
// 300 (Wet), 800 (Dry)  
if (rawADC <= 300) return 100.0;  
if (rawADC >= 800) return 0.0;  
return ((800.0 - (float)rawADC) / 500.0) * 100.0;  
}
```

24.2 3.5 แล็บวิจัยเกษตรขั้นสูง: 1D Richards Infiltration Solver (Python) และระบบนำทาง Extended Kalman Filter (EKF)

Chapter 25

3.5 แล็บวิจัยเกษตรขั้นสูง: การจำลองการซึมผ่านน้ำในดิน 1 มิติด้วยสมการริชาร์ดส์ และ ระบบนำทางด้วย Extended Kalman Filter

(Advanced Agricultural Research Laboratory: 1D Richards Infiltration Solver and Extended Kalman Filter Navigation System)

25.1 บทนำ: หลักการบูรณาการทางวิศวกรรมและ วิทยาศาสตร์ดิน (Introduction: Principles of Engineering and Soil Science Integration)

ในยุคของการเกษตรดิจิทัล (Digital Agriculture) การทำความเข้าใจกระบวนการทางกายภาพที่ซับซ้อนในระบบนิเวศการเพาะปลูกเป็นสิ่งจำเป็นอย่างยิ่ง การจัดการน้ำในดิน (Soil Water Management) ถือเป็นหัวใจสำคัญของการเพิ่มผลผลิตและลดผลกระทบต่อสิ่งแวดล้อม การ

จำลองการเคลื่อนที่ของน้ำในดิน (Soil Water Movement Simulation) จึงเป็นเครื่องมือทางวิศวกรรมที่ทรงพลังที่ช่วยให้เราสามารถทำนายพฤติกรรมของความชื้นในดิน (Soil Moisture) ภายใต้สภาวะการให้น้ำที่แตกต่างกัน

หัวข้อนี้จะนำเสนอการศึกษาเชิงลึกในสองส่วนหลัก: ส่วนแรกคือการจำลองการซึมผ่านน้ำแนวตั้งในดิน (Vertical Water Infiltration) โดยใช้สมการริชาร์ดส (Richards Equation) ซึ่งเป็นสมการเชิงอนุพันธ์ย่อย (Partial Differential Equation: PDE) ที่อธิบายการไหลของน้ำในช่องว่างของดิน (Pore Water Flow) ส่วนที่สองคือการประยุกต์ใช้ตัวกรองคาลมานขยาย (Extended Kalman Filter: EKF) เพื่อการหลอมรวมข้อมูลเซ็นเซอร์นำทาง (Navigation Sensor Data Fusion) สำหรับการทำแผนที่ฟาร์ม (Farm Mapping) อย่างแม่นยำ

25.2 ส่วนที่ 1: การจำลองการซึมผ่านน้ำในดิน 1 มิติด้วยสมการริชาร์ดส (1D Richards Infiltration Solver)

25.2.1 1.1 ทฤษฎีพื้นฐาน: สมการริชาร์ดส (Richards Equation Theory)

การไหลของน้ำในดินไม่ได้เป็นเพียงแค่การไหลของมวล (Mass Flow) เท่านั้น แต่ยังเกี่ยวข้องกับแรงดันน้ำในช่องว่าง (Pore Water Pressure) และแรงตึงผิว (Capillary Forces) ซึ่งทั้งหมดนี้ถูกรวมอยู่ในสมการริชาร์ดส (Richards Equation) ซึ่งเป็นสมการหลักที่ใช้อธิบายการเปลี่ยนแปลงของปริมาณความชื้นในดิน (θ) ตามเวลา (t) และความลึก (z)

สมการริชาร์ดสในรูปแบบหนึ่งมิติ (1D) สำหรับการเปลี่ยนแปลงความชื้นในดิน (θ) คือ:

$$\frac{\partial \theta}{\partial t} = \frac{\partial}{\partial z} \left[K(\theta) \left(\frac{\partial \psi}{\partial z} + 1 \right) \right]$$

คำอธิบายตัวแปร:

- θ : ความชื้นในดิน (Soil Water Content) [L/L หรือ m^3/m^3]
- t : เวลา (Time) [s]
- z : ความลึกในแนวตั้ง (Depth) [m]
- $K(\theta)$: ค่าการนำความร้อนของน้ำในดิน (Hydraulic Conductivity) [L/T หรือ m/s] ซึ่งขึ้นอยู่กับความชื้นในดิน
- ψ : ศักย์น้ำในช่องว่าง (Pore Water Potential) [L หรือ m]
- $\frac{\partial \psi}{\partial z}$: การไล่ระดับของศักย์น้ำตามความลึก (Gradient of Pore Water Potential)

25.2.2 1.2 ความสัมพันธ์ระหว่างตัวแปร (Constitutive Relationships)

เพื่อให้สามารถแก้สมการนี้ได้ เราจำเป็นต้องนิยามความสัมพันธ์ทางกายภาพที่ควบคุมตัวแปรหลัก:

1.2.1 ความสัมพันธ์ระหว่างศักย์น้ำและความชื้น ($\psi - \theta$ Relationship)

ความสัมพันธ์นี้มักถูกประมาณค่าโดยใช้ฟังก์ชันของ Van Genuchten (Van Genuchten Model) ซึ่งมีความแม่นยำสูงในการจำลองการดูดซับน้ำในดิน:

$$\frac{\theta - \theta_r}{\theta_s - \theta_r} = [1 + (\alpha|\psi|)^n]^{-1}$$

โดยที่:

- θ_r : ความชื้นที่จุดอิ่มตัว (Residual Water Content) [L/L]
- θ_s : ความชื้นที่จุดอิ่มตัวสูงสุด (Saturated Water Content) [L/L]
- α : ค่าสัมประสิทธิ์การดูดซับ (Parameter α)
- n : ค่าดัชนีการดูดซับ (Parameter n)

1.2.2 ความสัมพันธ์ระหว่างการนำความร้อนและความชื้น ($K - \theta$ Relationship)

ค่าการนำความร้อนของน้ำในดิน $K(\theta)$ ก็ขึ้นอยู่กับความชื้นในดินเช่นกัน โดยทั่วไปสามารถใช้ฟังก์ชันที่เชื่อมโยงกับ Van Genuchten ได้:

$$K(\theta) = K_s \left[\frac{1 - (1 - (\frac{\theta - \theta_r}{\theta_s - \theta_r})^m)^n}{1 - (1 - (\frac{\theta - \theta_r}{\theta_s - \theta_r})^m)} \right]^2$$

โดยที่ K_s คือค่าการนำความร้อนเมื่อดินอิ่มตัว (Saturated Hydraulic Conductivity) [L/T] และ m คือค่าที่เกี่ยวข้องกับ n

25.2.3 1.3 การประยุกต์ใช้กรณีศึกษา: ดินเหนียวจันทบุรี (Chan-thaburi Clay)

สำหรับกรณีศึกษาการจำลองการดูดซับน้ำในดินชั้นดินของสวนทุเรียน ในจังหวัดจันทบุรี เราจะใช้พารามิเตอร์จำลองของดินเหนียว (Clay Soil) ดังนี้:

- $\theta_r = 0.098$
- $\theta_s = 0.485$
- $K_s = 1.15 \times 10^{-4} \text{ cm/s} = 1.15 \times 10^{-6} \text{ m/s}$

25.2.4 1.4 การแก้สมการด้วยระเบียบวิธีผลต่างอันดับสิ้นสุด (Finite Difference Method: FDM)

เนื่องจากสมการรีชาร์ดสเป็น PDE ที่ซับซ้อน การแก้ปัญหาเชิงตัวเลข (Numerical Solution) จึงเป็นสิ่งจำเป็น เราจะใช้ระเบียบวิธีผลต่างอันดับสิ้นสุด (FDM) โดยการแบ่งโดเมนเวลาและพื้นที่ออกเป็นขั้นตอนย่อยๆ (Discretization)

การแปลง PDE เป็นรูปแบบที่สามารถคำนวณได้:

เราจะใช้การประมาณค่าแบบ Explicit หรือ Implicit (ในที่นี้จะใช้ Implicit Scheme เพื่อความเสถียร) โดยการประมาณค่าอนุพันธ์ย่อย ($\frac{\partial \theta}{\partial t}$ และ $\frac{\partial \theta}{\partial z}$) ด้วยผลต่างจำกัด (Finite Difference).

ขั้นตอนการคำนวณ:

1. กำหนดโดเมน: กำหนดความลึก Z_{max} (เช่น 1.0 m) และจำนวนจุด N_z
2. กำหนดเวลา: กำหนดช่วงเวลา T_{max} และขนาดขั้นตอนเวลา Δt
3. การวนซ้ำ (Iteration): ในแต่ละขั้นตอนเวลา t^{k+1} เราจะคำนวณ θ ที่ตำแหน่ง z_i โดยใช้ค่า θ จากเวลา t^k และการไหลที่ขอบเขต (Boundary Conditions)

เงื่อนไขขอบเขต (Boundary Conditions):

- ****ขอบเขตด้านบน ($z = 0$):**** เป็นแหล่งกำเนิดน้ำ (Infiltration Flux, q_{in}) ซึ่งเป็นอัตราการซึมผ่านน้ำที่ผิวหน้า
- ****ขอบเขตด้านล่าง ($z = Z_{max}$):**** เป็นการไหลออกสู่ชั้นน้ำใต้ดิน (Drainage Flux, q_{out}) ซึ่งมักจะกำหนดให้เป็นศูนย์หรือเป็นอัตราการไหลคงที่

25.2.5 1.5 ซอร์สโค้ด Python: ChanthaburiClayDiffusion-Solver

โค้ดนี้ใช้หลักการของ FDM เพื่อจำลองการแพร่กระจายความชื้นในดิน 1 มิติ

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import interp1d

# =====
# ChanthaburiClayDiffusionSolver
# (FDM) 1 มิติ
# =====
class ChanthaburiClayDiffusionSolver:
    """
    Solver for 1D Richards Equation using Finite Difference Method (FDM).
    Simulates water infiltration in Chanthaburi Clay soil profile.
    """
```

```
def __init__(self, depth_m=1.0, num_points=100, initial_theta=0.3):
    """
    Initializes the solver parameters.
    :param depth_m:  (m)
    :param num_points:  (Number of nodes)
    :param initial_theta:  (L/L)
    """
    self.L = depth_m # (Length)
    self.N = num_points # (Number of nodes)
    self.dz = self.L / (self.N - 1) # (Delta z)
    self.theta = np.full(self.N, initial_theta, dtype=float) #

# (Chanthaburi Clay Parameters)
self.theta_r = 0.098 # Residual Water Content (L/L)
self.theta_s = 0.485 # Saturated Water Content (L/L)
self.K_s = 1.15e-6 # Saturated Hydraulic Conductivity (m/s)
self.alpha = 1.5 # Van Genuchten alpha (Hypothetical value)
self.n_van = 1.5 # Van Genuchten n (Hypothetical value)
self.m_van = 0.5 # Van Genuchten m (Hypothetical value)

# -----
# 1. (Constitutive Functions)
# -----

def _van_genuchten_theta_psi(self, theta_val):
    """
    (psi) (theta) Van Genuchten Model.
    psi = - ( (theta - theta_r) / (theta_s - theta_r) ) ^ (1/m)
    """
    #
    ratio = (theta_val - self.theta_r) / (self.theta_s - self.theta_r)
    if ratio < 0:
        return -1e6 # psi ( ) theta < theta_r

    # psi
    psi = - (ratio ** (1 / self.m_van))
    return psi

def calculate_k(self, theta_val):
    """
    K(theta) (m/s).
    Van Genuchten K(theta).
    """
    if theta_val < self.theta_r:
        return 0.0 # K = 0

    #
    ratio = (theta_val - self.theta_r) / (self.theta_s - self.theta_r)

    # K(theta)
    term1 = 1 - (1 - ratio**self.m_van)**self.n_van
    term2 = 1 - (1 - ratio**self.m_van)

    if term2 == 0:
        return self.K_s #

    K_theta = self.K_s * (term1 / term2)**2
```

```

return K_theta

# -----
# 2.  (Main Simulation Function)
# -----

def solve_richards_1d(self, infiltration_rate_q_in, total_time_s, dt_s):
    """
    1 (1D Richards Infiltration).
    :param infiltration_rate_q_in:  (m/s).
    :param total_time_s:  ().
    :param dt_s:  ().
    :return: numpy array  (History of theta).
    """
    time_steps = int(total_time_s / dt_s)
    theta_history = [self.theta.copy()]

    #  (Time stepping loop)
    for k in range(time_steps):
        theta_new = self.theta.copy()

        #  (Boundary Fluxes)
        # 1.  (z=0):  (q_in)
        q_in_k = infiltration_rate_q_in

        # 2.  (z=L):  (Drainage condition)
        q_out_k = 0.0

        #  (Depth loop)
        for i in range(1, self.N - 1):
            # 1.  K  psi  (z_i)
            K_i = self.calculate_k(self.theta[i])
            psi_i = self._van_genuchten_theta_psi(self.theta[i])

            # 2.  (Approximation of d(psi)/dz)
            #  Central Difference Scheme
            d_psi_dz_i = (self._van_genuchten_theta_psi(self.theta[i+1]) -
                          self._van_genuchten_theta_psi(self.theta[i-1]))
                          / (2 * self.dz)

            # 3.  i (Flux, q)
            #  $q = -K(\theta) * (d(\psi)/dz + 1)$ 
            #  +1  Richards
            #  (Pressure Head)
            q_i = -K_i * (d_psi_dz_i + 1.0)

            # 4.  (d(theta)/dt)
            #  $d(\theta)/dt = -d^2(q)/dz / (\rho * C_r)$  (Simplified form for
            #  water only)

            #
            #  i
            q_left = -self.calculate_k(self.theta[i]) * (self._van_genuchten_theta_psi(self.theta[i]) - self._van_genuchten_theta_psi(self.theta[i-1])) / self.dz
            q_right = -self.calculate_k(self.theta[i]) * (self._van_genuchten_theta_psi(self.theta[i+1]) - self._van_genuchten_theta_psi(self.theta[i])) / self.dz

```

```

        d_theta_dt = (q_right - q_left) / self.dz

        # 5. Explicit Euler Method
        theta_new[i] = self.theta[i] + d_theta_dt * dt_s

        # 
        theta_new[0] = self.theta[0] + (infiltration_rate_q_in - self.
            calculate_k(self.theta[0])) * (dt_s / self.dz)
        theta_new[-1] = self.theta[-1] + (self.calculate_k(self.theta[-2])
            - q_out_k) * (dt_s / self.dz)

        # 
        (Physical Constraints: theta_r <= theta <= theta_s)
        theta_new = np.clip(theta_new, self.theta_r, self.theta_s)

        # 
        self.theta = theta_new.copy()
        theta_history.append(self.theta.copy())

    return np.array(theta_history)

# 
solver = ChanthaburiClayDiffusionSolver()
# 0.05 / = 0.0005 
history = solver.solve_richards_1d(infiltration_rate_q_in=0.0005, total_time_s
    =600, dt_s=1.0)
print("--- (Richards 1D) ---")
print(f" : {len(history)} ")
print(f" : {history[0][0]:.4f} m^3/m^3")
print(f" 10 : {history[-1][0]:.4f} m^3/m^3")

```

25.3 3.6 การประกอบกล่องภายนอกอาคารมาตรฐาน IP65 และ Telemetry ส่งพารามิเตอร์ขึ้น Cloud (MQTT)

Chapter 26

3.6 การประกอบกล่อง ภายนอกอาคารมาตรฐาน

IP65 และ Telemetry ส่ง พารามิเตอร์ขึ้น Cloud (MQTT)

26.1 บทนำเชิงทฤษฎี: การออกแบบระบบ IoT สำหรับ สภาพแวดล้อมภายนอกอาคาร (Outdoor IoT Deployments)

ในบริบทของการเกษตรดิจิทัล (Digital Agriculture) และการเกษตรแม่นยำ (Precision Agriculture) การเก็บรวบรวมข้อมูลสภาพแวดล้อม (Environmental Data) จากแปลงปลูกภายนอกอาคารถือเป็นหัวใจสำคัญของการตัดสินใจที่ขับเคลื่อนด้วยข้อมูล (Data-Driven Decision Making) อย่างไรก็ตาม การติดตั้งอุปกรณ์ตรวจวัด (Sensor Nodes) ในสภาพแวดล้อมภายนอกอาคารนั้นมีความท้าทายอย่างยิ่ง เนื่องจากต้องเผชิญกับปัจจัยทางธรรมชาติที่รุนแรงและแปรปรวน เช่น ความชื้นสูง (High Humidity), ฝุ่นละออง (Dust Particulates), การเปลี่ยนแปลงของอุณหภูมิอย่างรวดเร็ว (Rapid Temperature Fluctuations), และการสัมผัสกับ

น้ำฝนโดยตรง (Direct Rainfall Exposure)

ดังนั้น การออกแบบระบบจึงต้องคำนึงถึงความคงทนทางกายภาพ (Physical Robustness) และความน่าเชื่อถือในการทำงาน (Operational Reliability) เป็นสำคัญยิ่งยวด กล่องหุ้มอุปกรณ์ (Enclosure) ที่ใช้ในการติดตั้งจึงไม่สามารถเป็นเพียงกล่องพลาสติกธรรมดาได้ แต่ต้องเป็นกล่องที่ไ้มาตรฐานการป้องกันระดับสูง เพื่อให้มั่นใจว่าวงจรไฟฟ้า (Electronic Circuits) และเซนเซอร์ต่างๆ จะได้รับการปกป้องจากปัจจัยภายนอกได้อย่างสมบูรณ์

26.1.1 3.6.1 หลักการป้องกันการแทรกซึม (Ingress Protection: IP Rating)

มาตรฐานการป้องกันการแทรกซึม (Ingress Protection Rating) หรือที่เรียกว่า IP rating เป็นระบบมาตรฐานสากลที่กำหนดระดับการป้องกันของอุปกรณ์ไฟฟ้าและอิเล็กทรอนิกส์จากสิ่งแปลกปลอม (Foreign Objects) และการแทรกซึมของน้ำ (Water Ingress) โดยมีรหัสตัวเลขสองหลัก (Two-digit Code)

หลักการของ IP Rating:

1. หลักแรก (First Digit): ระดับการป้องกันจากสิ่งแปลกปลอมที่เป็นของแข็ง (Solid Objects) โดยมีช่วงตั้งแต่ 0 ถึง 10 (เช่น 0 = ไม่มีป้องกัน, 1 = ป้องกันวัตถุที่ใหญ่กว่า 52 มม., 6 = ป้องกันฝุ่นได้อย่างสมบูรณ์แบบ)
2. หลักที่สอง (Second Digit): ระดับการป้องกันจากน้ำ (Water Immersion) โดยมีช่วงตั้งแต่ 0 ถึง 9 (เช่น 0 = ไม่มีป้องกัน, 1 = ป้องกันการสัมผัสจากของแข็งที่พุ่งลงมา, 6 = ป้องกันการแช่น้ำชั่วคราว, 7 = ป้องกันการแช่น้ำชั่วคราวที่ความลึก 1 เมตร, 8 = ป้องกันการแช่น้ำต่อเนื่องที่ความลึก 1.5 เมตร)

การเนนมาตรฐาน IP65:

สำหรับงานเกษตรดิจิทัลที่ต้องติดตั้งในพื้นที่เปิดโล่งและเผชิญกับมรสุม (Monsoon Climate) การเลือกใช้กล่องที่มีมาตรฐาน IP65 ถือเป็นแนวทางปฏิบัติที่ดีที่สุด (Best Practice)

- IP6: หมายถึง การป้องกันฝุ่นได้อย่างสมบูรณ์แบบ (Dust Dust-tight) ซึ่งหมายความว่าไม่มีฝุ่นละอองสามารถเข้าสู่ภายในกล่องได้เลย
- IP65: หมายถึง การป้องกันน้ำที่พ่นหรือฉีดจากทุกทิศทาง (Water Jets) ได้อย่างมีประสิทธิภาพ

การบรรลุมาตรฐาน IP65 ไม่ได้ขึ้นอยู่กับตัวกล่องเพียงอย่างเดียว แต่ยังรวมถึงรายละเอียดของการติดตั้งด้วย เช่น การซีลรอยต่อ (Sealing Gaskets), การเดินสายไฟที่ต้องผ่านท่อร้อยสาย (Conduit) ที่มีการซีลที่จุดผ่าน (Cable Gland) อย่างเหมาะสม

26.1.2 3.6.2 แนวทางการประกอบกล่องภายนอกอาคารตามมาตรฐาน IP65

การประกอบกล่องภายนอกอาคารให้ได้มาตรฐาน IP65 ต้องอาศัยความพิถีพิถันในทุกขั้นตอน โดยเฉพาะอย่างยิ่งในส่วนของการจัดการรอยต่อ (Joint Management) และการเดินสายไฟ

1. การเลือกวัสดุและโครงสร้าง (Material Selection)

ควรเลือกใช้กล่องพลาสติกอะคริลิก (Acrylic) หรือโพลีคาร์บอเนต (Polycarbonate) ที่มีคุณสมบัติทน UV (UV Resistance) และทนต่อการเปลี่ยนแปลงอุณหภูมิ (Thermal Stability)

2. การจัดการรอยต่อและการเดินสายไฟ (Cable Management and Sealing)

จุดที่น้ำและฝุ่นมีโอกาสแทรกซึมมากที่สุดคือบริเวณที่สายไฟเข้าและออกจากกล่อง (Penetrations)

- การใช้ท่อร้อยสาย (Conduit): ต้องใช้ท่อร้อยสายกระดุกงู (Flexible Conduit) หรือท่อ PVC ที่มีความแข็งแรงพอสมควร เพื่อป้องกันการกระแทกทางกายภาพ (Mechanical Impact)
- การซีลจุดผ่าน (Cable Gland Installation): เมื่อสายไฟเข้าสู่กล่อง จะต้องติดตั้ง Cable Gland (ข้อต่อซีลสายเคเบิล) ที่เหมาะสมกับขนาดของสายไฟและมีคุณสมบัติกันน้ำ (Waterproof) การขันเกลียวของ Cable Gland ต้องแน่นหนาและใช้ปะเก็น (Gasket) ที่มีคุณภาพ

3. การติดตั้งแหล่งพลังงาน (Solar Power Integration)

ระบบต้องมีการติดตั้งแผงโซลาร์เซลล์ (Solar Panel) และวงจรควบคุมการชาร์จ (Charge Controller) ที่เหมาะสม

หลักการคำนวณพลังงาน (Energy Budgeting): พลังงานที่ระบบต้องการในหนึ่งวัน (E_{daily}) ต้องน้อยกว่าพลังงานที่แผงโซลาร์เซลล์สามารถผลิตได้ในสภาพแสงที่เลวร้ายที่สุด (Worst-case scenario)

$$E_{daily} \leq P_{solar} \times T_{charge} \times \eta_{system}$$

เมื่อ:

- E_{daily} คือ พลังงานที่ระบบใช้ต่อวัน (หน่วย: Wh/day)
- P_{solar} คือ กำลังไฟฟ้าที่ผลิตได้จากแผงโซลาร์เซลล์ (หน่วย: W)
- T_{charge} คือ ชั่วโมงแสงอาทิตย์ที่คาดการณ์ (Peak Sun Hours, h)
- η_{system} คือ ประสิทธิภาพรวมของระบบ (System Efficiency)

การคำนวณนี้ช่วยให้มั่นใจว่าแบตเตอรี่ (Battery) จะสามารถจ่ายพลังงานได้อย่างต่อเนื่องแม้ในวันที่เมฆมาก

26.2 3.6.3 แนวคิดการสื่อสารข้อมูลระยะไกลด้วยโปรโตคอล MQTT (Telemetry via MQTT)

การส่งข้อมูลจากอุปกรณ์ IoT ในพื้นที่ห่างไกล (Remote Area) จำเป็นต้องใช้โปรโตคอลการสื่อสารที่ใช้แบนด์วิดท์ (Bandwidth) ต่ำ ใช้พลังงานต่ำ (Low Power) และมีความเสถียรสูง โปรโตคอลที่เหมาะสมที่สุดสำหรับงานนี้คือ MQTT (Message Queuing Telemetry Transport)

26.2.1 1. สถาปัตยกรรม Publish/Subscribe (Pub/Sub Architecture)

MQTT ไม่ได้ใช้รูปแบบการสื่อสารแบบ Client-Server ทั่วไป แต่ใช้รูปแบบ Publish/Subscribe ซึ่งมีองค์ประกอบหลัก 3 ส่วน:

1. Publisher: คืออุปกรณ์ IoT (เช่น ESP32) ที่ทำหน้าที่รวบรวมข้อมูล (เช่น ค่าความชื้น, ค่าแสง) และส่งข้อมูลออกไป
2. Subscriber: คือผู้รับข้อมูล (เช่น ระบบฐานข้อมูลบน Cloud, Dashboard) ที่ทำหน้าที่ "บอกรับ" ข้อมูลที่สนใจ
3. Broker: คือตัวกลาง (Message Broker) ซึ่งทำหน้าที่รับข้อความจาก Publisher และส่งต่อไปยัง Subscriber ที่ลงทะเบียนรับข้อความนั้นๆ

ข้อดีของ Pub/Sub:

- Decoupling: Publisher ไม่จำเป็นต้องรู้ว่าใครคือ Subscriber และ Subscriber ไม่จำเป็นต้องรู้ว่าใครคือ Publisher ทำให้ระบบมีความยืดหยุ่นสูง (Scalability)

- Efficiency: ข้อมูลจะถูกส่งไปยังผู้ที่ต้องการรับเท่านั้น ลดการจราจรของข้อมูล (Network Traffic)

26.2.2 2. คุณสมบัติของ MQTT (MQTT Properties)

- Topic: คือหัวข้อ (Topic) ที่ใช้ระบุประเภทของข้อมูลที่ส่ง เช่น `farm/plotA/soil_moisture` หรือ `farm/plotA/light_intensity` การใช้ Topic ที่เป็นโครงสร้างแบบลำดับชั้น (Hierarchical Structure) ช่วยให้การจัดการข้อมูลเป็นระเบียบ
- Quality of Service (QoS): คือระดับการรับประกันการส่งข้อความ (Message Delivery Guarantee) MQTT มี 3 ระดับ:
 - QoS 0 (At Most Once): ส่งข้อความเพียงครั้งเดียว ไม่มีการรับประกันการส่ง (Fire and Forget) เหมาะสำหรับข้อมูลที่ไม่สำคัญมาก
 - QoS 1 (At Least Once): รับประกันว่าข้อความจะถูกส่งถึงอย่างน้อยหนึ่งครั้ง อาจมีการส่งซ้ำ (Duplicate)
 - QoS 2 (Exactly Once): รับประกันว่าข้อความจะถูกส่งถึงเพียงครั้งเดียว (Ideal but complex)

ในการส่งข้อมูลเซนเซอร์ที่สำคัญ เช่น ระดับน้ำ ควรใช้ QoS 1 เพื่อให้มั่นใจว่าข้อมูลจะไม่สูญหายแม้มีการหลุดการเชื่อมต่อชั่วคราว

26.3 3.6.4 การประยุกต์ใช้โค้ด: การส่งข้อมูลเซนเซอร์ผ่าน MQTT (C++ Implementation)

ในส่วนนี้ จะแสดงการเขียนโค้ดภาษา C++ สำหรับไมโครคอนโทรลเลอร์ ESP32 ซึ่งเป็นอุปกรณ์ที่เหมาะสมสำหรับการใช้งาน IoT ภายนอกอาคาร

เนื่องจากมีพลังประมวลผลสูงและรองรับ Wi-Fi ได้อย่างมีประสิทธิภาพ
สมมติฐานของระบบ:

1. ใช้ ESP32 เป็นตัวควบคุมหลัก (Microcontroller).
2. เชื่อมต่อเซนเซอร์วัดความชื้นดิน (Soil Moisture Sensor) และเซนเซอร์
วัดแสง (Photoresistor/LDR).
3. เชื่อมต่อกับ MQTT Broker บน Cloud (เช่น HiveMQ, Mosquitto).

26.3.1 โค้ดตัวอย่าง (Arduino IDE / C++)

```
//  
=====
```

```
// [1]  (Library and Hardware Setup)  
//  
=====
```

```
#include <WiFi.h>           //  Wi-Fi  
#include <PubSubClient.h>   //  MQTT
```

```
//  Wi-Fi  
const char* ssid = "YOUR_WIFI_SSID";      //  Wi-Fi  
const char* password = "YOUR_WIFI_PASSWORD"; //  Wi-Fi
```

```
//  MQTT Broker (Cloud Server)  
const char* mqtt_server = "broker.hivemq.com"; //  MQTT Broker Public  
const int mqtt_port = 1883;                  //  MQTT (TLS/SSL 8883)
```

```
//  (Topic)   
//  : farm/plotA/sensor_type  
const char* topic_moisture = "farm/plotA/soil_moisture";  
const char* topic_light = "farm/plotA/light_intensity";
```

```
//  Client MQTT  
WiFiClient espClient;  
PubSubClient client(espClient);
```

```
//  
=====
```

```
// [2]  Wi-Fi (Wi-Fi Connection Function)  
//  
=====
```

```
void setup_wifi() {  
  Serial.print("Connecting to ");  
  Serial.println(ssid);
```

```

// เชื่อมต่อ Wi-Fi
WiFi.begin(ssid, password);

// ระยะเวลาที่รอการเชื่อมต่อ (Timeout 20 วินาที)
int attempts = 0;
while (WiFi.status() != WL_CONNECTED && attempts < 20) {
    delay(500);
    Serial.print(".");
    attempts++;
}

if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\nWiFi connected successfully!");
    Serial.print("IP Address: ");
    Serial.println(WiFi.localIP());
} else {
    Serial.println("\nFailed to connect to WiFi. Check credentials.");
    // ระยะเวลาที่รอการเชื่อมต่อ (Timeout 5000 มิลลิวินาที)
    delay(5000);
    return;
}

//
=====

// [3] เชื่อมต่อ MQTT (MQTT Connection Handler)
//
=====

void callback(const String& topic, payload, unsigned int length) {
    // ระบุชื่อผู้รับ (Subscriber)
    // ระบุชื่อผู้ส่ง (Publisher)
    Serial.print("Message received on topic [");
    Serial.print(topic);
    Serial.print("]: ");
    Serial.println(payload);
}

void reconnect_mqtt() {
    // เชื่อมต่อ MQTT
    while (!client.connected()) {
        Serial.print("Attempting MQTT connection...");

        // เชื่อมต่อ MQTT: client.connect(client_id, username, password)
        // ระบุ Client ID
        if (client.connect("ESP32_PlotA_Sensor", "user", "password")) {
            Serial.println("connected!");
            // ระบุ Topic (Subscription)
            client.subscribe("farm/plotA/alerts");
        } else {
            Serial.print("failed, rc=");
            Serial.print(client.state());
            Serial.println(" try again in 5 seconds");
            delay(5000); // รอ 5 วินาที
        }
    }
}

```

```
//  
=====
```

```
// [4]  (Sensor Reading Function)  
//  
=====
```

```
//  (Soil Moisture)  
//  0  100%  
float read_soil_moisture() {  
    // : AnalogRead()  ADC  ESP32  
    // : float raw_value = analogRead(A0);  
    //  (Calibration):  (Calibration)  
    //  45.5%  
    float moisture = random(300, 700) / 10.0; //  30.0  70.0  
    return moisture;  
}
```

```
//  (Light Intensity)  
//  Lux  ()  
float read_light_intensity() {  
    // : LDR  Photoresistor  
    //  550 Lux  
    float light = random(300, 800); //  300  800 Lux  
    return light;  
}
```

```
//  
=====
```

```
// [5] Setup  Loop  (Setup and Main Loop)  
//  
=====
```

```
void setup() {  
    Serial.begin(115200); //  Serial Monitor  
  
    // 1.  Wi-Fi  
    setup_wifi();  
  
    // 2.  MQTT Client  
    client.setCallback(callback);  
    client.setServer(mqtt_server, mqtt_port);  
}
```

```
void loop() {  
    // 1.  MQTT (Keepalive)  
    if (!client.connected()) {  
        reconnect_mqtt(); //    
    }  
    client.loop(); //  MQTT    
  
    // 2.  (Sensor Reading)  
    float moisture = read_soil_moisture();  
    float light = read_light_intensity();  
  
    // 3.  Payload (Data Payload Generation)
```



```
// 3. String Format (JSON Format)
String payload = "{\"timestamp\": " + String(millis()) +
    ", \"moisture\": " + String(moisture) +
    ", \"light\": " + String(light) + "}";

// 4. Publishing Data
Serial.print("Publishing data to Topic: ");
Serial.println(topic_moisture);

// MQTT Broker [3]
client.publish(topic_moisture, String(moisture).c_str());

// MQTT Broker [3]
client.publish(topic_light, String(light).c_str());

// Serial Monitor
Serial.print("Data Sent successfully! Moisture: ");
Serial.print(moisture);
Serial.print("%, Light: ");
Serial.print(light);
Serial.println(" Lux");

// 5. Telemetry Interval (Deep Sleep) [3]
// 5 - 15 seconds (Deep Sleep) [3]
// 5 seconds
delay(5000);
}
```

26.4 3.6.5 การวิเคราะห์เปรียบเทียบเชิงวิพากษ์: ลูปควบคุมปิด-เปิด (On-Off Control) กับการควบคุมคาดการณ์เชิงชลศาสตร์ (Richards PDE-based Predictive Control)

ในการรดน้ำสวนทุเรียนอัจฉริยะ ตรรกะการรดน้ำที่ใช้นั้นสามารถพัฒนาได้หลายระดับ ซึ่งมีความคุ้มค่าและสมรรถนะเปรียบเทียบดังนี้:

26.5 3.6.6 คำศัพท์สำคัญประจำบทเรียน (Key Terms)

1. Ingress Protection (IP) Rating: รหัสระบุค่าการป้องกันตัวอุปกรณ์อิเล็กทรอนิกส์จากของแข็งภายนอกและของเหลวตามมาตรฐานสากล

มิติการเปรียบเทียบ	ลูบควบคุมแบบ ปิด-เปิด (On-Off Control Loop) [3]	การควบคุมภาค การณ้ชลศาสตร้ดิน (Richards PDE Control) [1]
ตรรกะการสั่งงาน (Control Logic)	ง่ายมาก (เปิดน้ำเมื่อ ความชื้นต่ำกว่า 40% และปิดน้ำเมื่อ ความชื้นสูงกว่า 70%)	ซับซ้อนสูง (คำนวณ สมการ Richards PDE เพื่อทำนาย อัตราการแทรกซึม และการแพร่ของน้ำ ใต้เขตรากลงหนา [1])
ความเสถียรของอัล กอริทึม (Algorithmic Stability)	สูงและสม่ำเสมอ (ไม่ เสี่ยงต่อการเกิด ปัญหาการคำนวณไม่ ลู่เข้า (Divergence) บนบอร์ดควบคุม)	ปานกลาง (อาจเกิด ความเสี่ยงการลู่เข้า ของตัวเลขเชิงตัวเลข บนบอร์ดขนาดเล็ก หากตัวแปรเริ่มต้นไม่ ถูกต้อง)
ประสิทธิภาพ ทรัพยากรน้ำ (Water Efficiency)	ปานกลาง-ต่ำ (มี โอกาสรดน้ำเกิน ความต้องการของดิน ดานล่าง ทำให้สูญ เสียน้ำและสาร อาหาร หรือเกิดโรค รากเนา)	สูงมาก (คำนวณ ปริมาณน้ำเข้าสู่ชั้น รากจริงอย่างแม่นยำ ป้องกันการสูญเสียน้ำ ระเหยและการซึมลึก ใต้เขตราก [1])
ความต้องการ เซนเซอร์วัดค่า (Sensor Dependency)	ต่ำ (ต้องการเซนเซอร์ วัดความชื้นในดิน เพียงชุดเดียว ณ ความลึกเดียว)	สูง (ต้องการเซนเซอร์ วัดระดับความชื้น หลายความลึกเพื่อ ตรวจสอบเงื่อนไข ขอบเขต (Boundary Conditions) [1])
ความเหมาะสมเชิง พื้นที่ (Practical Suitability)	เหมาะสมกับระบบ ต้นแบบจัดอบรมและ สุกลูขนาดเล็กของ โรงเรียน [3]	เหมาะสมกับแปลง ทุเรียนพานิชย อุตสาหกรรมขนาด ใหญ่ที่มีดินหลาย ประเภท

[3].

2. Message Queuing Telemetry Transport (MQTT): โพรโตคอลโทรมาตรการสื่อสารน้ำหนักเบาสำหรับอุปกรณ์อินเทอร์เน็ตของสรรพสิ่ง (IoT) [3].
3. Publish/Subscribe Model: รูปแบบความสัมพันธ์เครือข่ายสื่อสารการแลกเปลี่ยนข้อความผ่านตัวกลางโบรกเกอร์ (Broker) ที่ลดการผูกมัดระหว่างผู้ส่งและผู้รับข้อมูล.
4. Richards Equation: สมการการเคลื่อนที่ของของไหลแบบไม่มีความอิ่มตัวในตัวกลางที่มีรูพรุน ใช้ทำนายระดับชลศาสตร์น้ำใต้รากพืช [1].

—

26.6 3.6.7 แบบฝึกหัดทบทวนและโจทย์คิดวิเคราะห์ (Review Exercises)

โจทย์ข้อที่ 1 (ระดับความเข้าใจ - Comprehension): ทำไมการนำบอร์ด ESP32 ไปติดตั้งภายนอกอาคารในแปลงสวนทุเรียน อ.เขาสมิง จ.ตราด จึงต้องเลือกใช้มาตรฐานการป้องกันระดับ IP65 เป็นอย่างต่ำ? หากเลือกใช้เพียง IP20 จะส่งผลต่อความเสถียรและอายุใช้งานของฮาร์ดแวร์วัดความชื้นดินอย่างไรภายใต้ภัยพิบัติธรรมชาติของพื้นที่ภาคตะวันออก [3]?

โจทย์ข้อที่ 2 (ระดับการวิเคราะห์เชิงตัวเลข - Analysis & Derivation): สมมติว่าคุณคำนวณงบประมาณพลังงานของกล่องสมาร์ตฟาร์ม (Energy Budgeting) ที่ติดตั้งในสวนทุเรียน

- ก) หากบอร์ด ESP32 และเซนเซอร์ในกล่องดึงกระแสเฉลี่ย 150 mA ที่แรงดัน 5 V ตลอด 24 ชั่วโมง จงคำนวณพลังงานที่ระบบใช้ต่อวัน (E_{daily}) ในหน่วยวัตต์-ชั่วโมง (Wh/day)

- ข) หากใช้แผงโซลาร์เซลล์ขนาดกำลัง $P_{solar} = 10 \text{ W}$ ในพื้นที่ที่มีชั่วโมงแสงเฉลี่ยกรณีฝนตกฟ้าหลัวเพียง $T_{charge} = 2.5$ ชั่วโมงต่อวัน และประสิทธิภาพรวมระบบอยู่ที่ $\eta_{system} = 70\%$ จงคำนวณว่าแผงโซลาร์เซลล์นี้มีกำลังเพียงพอที่จะจ่ายพลังงานทดแทนส่วนที่ใช้งานต่อวันได้หรือไม่? หากไม่พอ ควรปรับปรุงระบบอย่างไร (เช่น การใช้โหมด Deep Sleep เพื่อลดปริมาณพลังงาน)?

—

บรรณานุกรมเชิงวิชาการ มาตรฐานสากล (References - APA and IEEE Styles)

Chapter 27

บทที่ 5: การจัดการข้อมูลและ การสื่อสารทางวิชาการ

ภาคผนวก: หลักการและรูปแบบการอ้างอิงเชิงวิชาการ มาตรฐานสากล (Academic Citation Standards)

27.0.1 ๕.๑ บทนำ: ความสำคัญของการอ้างอิงเชิงวิชาการ (Introduction to Academic Referencing)

ในฐานะนักวิชาการที่ปฏิบัติงานในสาขาฟิสิกส์ประยุกต์ (Applied Physics) และเกษตรดิจิทัล (Digital Agriculture) การสื่อสารองค์ความรู้ (Knowledge Communication) อย่างถูกต้องแม่นยำถือเป็นหัวใจสำคัญของการวิจัยและการพัฒนาทางวิทยาศาสตร์ การอ้างอิงเชิงวิชาการ (Academic Citation) จึงมิใช่เพียงแค่การระบุแหล่งที่มาของข้อมูล (Source Attribution) เท่านั้น แต่ยังเป็นกลไกทางจริยธรรม (Ethical Mechanism) ที่แสดงถึงความเคารพต่อทรัพย์สินทางปัญญา (Intellectual Property) ของนักวิจัยท่านอื่น และเป็นการสร้างความน่าเชื่อถือ (Credibility) ให้กับผลงานวิจัยของตนเอง

การทำงานวิชาการจะได้รับการยอมรับในระดับสากล จำเป็นต้องปฏิบัติตามรูปแบบการอ้างอิง (Citation Styles) ที่ได้รับการยอมรับในวงการวิชาการนั้น ๆ ซึ่งรูปแบบเหล่านี้ถูกพัฒนาขึ้นเพื่อความเป็นมาตรฐาน (Standardization) ในการจัดระเบียบข้อมูล (Information Organiza-

tion) โดยรูปแบบที่สำคัญและถูกใช้งานอย่างแพร่หลาย ได้แก่ APA (American Psychological Association) และ IEEE (Institute of Electrical and Electronics Engineers)

27.0.2 ๕.๒ กรอบแนวคิดทางทฤษฎีของการอ้างอิง (Theoretical Framework of Citation)

การอ้างอิงเชิงวิชาการมีวัตถุประสงค์หลัก ๓ ประการ:

1. การป้องกันการลอกเลียนแบบ (Plagiarism Prevention): เป็นการระบุอย่างชัดเจนว่าแนวคิด (Concept), ข้อมูล (Data), หรือวิธีการ (Methodology) ไตที่มาจากแหล่งภายนอก
2. การตรวจสอบความน่าเชื่อถือ (Verifiability): เปิดโอกาสให้ผู้อ่านสามารถย้อนกลับไปตรวจสอบแหล่งข้อมูลต้นฉบับ (Original Source) เพื่อยืนยันความถูกต้องของข้อเท็จจริง
3. การสร้างเครือข่ายความรู้ (Knowledge Network Building): แสดงให้เห็นว่างานวิจัยปัจจุบันตั้งอยู่บนรากฐานทางทฤษฎี (Theoretical Foundation) ไตบาง

๕.๒.๑ ความแตกต่างเชิงปรัชญาของรูปแบบ (Philosophical Difference between Styles)

- APA Style: มักถูกใช้ในสาขาสังคมศาสตร์ (Social Sciences), พฤติกรรมศาสตร์ (Behavioral Sciences), และชีววิทยา (Biology) ซึ่งเน้นการอ้างอิงผู้เขียนและปีที่พิมพ์ (Author-Date System) เพื่อเน้นบริบททางเวลาของแนวคิด
- IEEE Style: มักถูกใช้ในสาขาวิศวกรรมศาสตร์ (Engineering), วิทยาการคอมพิวเตอร์ (Computer Science), และระบบควบคุม (Control Systems) ซึ่งเน้นการอ้างอิงตัวเลขลำดับ (Numerical System)

เพื่อความกระชับและง่ายต่อการอ้างอิงในเอกสารที่มีสมการและโค้ดจำนวนมาก

27.0.3 ๕.๓ การประยุกต์ใช้รูปแบบ APA 7th Edition (American Psychological Association)

APA 7th edition เป็นรูปแบบที่เน้นการระบุผู้เขียน (Author) และปีที่พิมพ์ (Year) อย่างชัดเจนในเนื้อหา (In-text Citation)

๕.๓.๑ โครงสร้างการอ้างอิงในเนื้อหา (In-text Citation Structure)

เมื่อมีการกล่าวถึงแนวคิดของนักวิจัย (Smith, 2020) จะต้องระบุชื่อผู้เขียนและปีที่พิมพ์ในวงเล็บ หรือกล่าวถึงในประโยค (Smith (2020) เสนอว่า...)

๕.๓.๒ โครงสร้างรายการบรรณานุกรม (Reference List Structure)

รายการบรรณานุกรมต้องจัดเรียงตามลำดับตัวอักษร (Alphabetical Order) และใช้รูปแบบการเยื้องแบบแขวน (Hanging Indent)

ตัวอย่างการอ้างอิงวารสาร (Journal Article):

ขอควรระวัง (Important)

ผู้เขียน, A. A., & ผู้เขียน, B. B. (ปี). ชื่อบทความ: คำบรรยายย่อ. ชื่อวารสาร, เล่มที่(ฉบับที่), หน้าแรก-หน้าสุดท้าย. DOI หรือ URL

ตัวอย่างการอ้างอิงหนังสือ (Book):

ขอควรระวัง (Important)

ผู้เขียน, A. A. (ปี). ชื่อหนังสือ: คำบรรยายย่อ. สำนักพิมพ์.

27.0.4 ๕.๔ การประยุกต์ใช้รูปแบบ IEEE Style (Institute of Electrical and Electronics Engineers)

IEEE Style เป็นรูปแบบที่เน้นการใช้ตัวเลขลำดับ (Sequential Numbering) ในการอ้างอิงทั้งในเนื้อหาและในรายการบรรณานุกรม

๕.๔.๑ โครงสร้างการอ้างอิงในเนื้อหา (In-text Citation Structure)

การอ้างอิงจะทำได้โดยการใส่ตัวเลขยกกำลังหรือตัวเลขในวงเล็บเหลี่ยม [1] ซึ่งตัวเลขนี้จะสอดคล้องกับลำดับของรายการในส่วนบรรณานุกรม (References)

๕.๔.๒ โครงสร้างรายการบรรณานุกรม (References List Structure)

รายการบรรณานุกรมต้องจัดเรียงตามลำดับที่ปรากฏในเนื้อหา (Order of Appearance) โดยใช้ตัวเลขกำกับ [1], [2], [3], ...

ตัวอย่างการอ้างอิงวารสาร (Journal Article):

ขอควรระวัง (Important)

[N] A. A. ผู้เขียน, B. B. ผู้เขียน, และ C. C. ผู้เขียน, “ชื่อบทความ,” ชื่อวารสาร, vol. X, no. Y, pp. page–page, เดือน ปี.

ตัวอย่างการอ้างอิงหนังสือ (Book):

ขอควรระวัง (Important)

[N] A. A. ผู้เขียน, ชื่อหนังสือ. เมืองที่พิมพ์: สำนักพิมพ์, ปี.

27.0.5 ๕.๕ การปฏิบัติการสร้างบรรณานุกรมเชิงวิชาการ (Practical Bibliography Construction)

ในส่วนนี้ ผู้อาจารย์จะแสดงตัวอย่างการรวบรวมรายการอ้างอิง ๖ รายการที่ทันสมัยและครอบคลุมสาขาที่เกี่ยวข้อง (ฟิสิกส์ประยุกต์, เกษตรดิจิทัล, AI) และจัดรูปแบบให้ถูกต้องตามมาตรฐาน APA 7th และ IEEE Style

๕.๕.๑ รายการอ้างอิงที่ ๑: แบบจำลองน้ำในดิน (Soil Water Model)

- แหล่งที่มา: Richards, A. S., & Van Genuchten, M. T.
- ความเกี่ยวข้อง: ฟิสิกส์ของดิน (Soil Physics) และการเกษตรแม่นยำ (Precision Agriculture)

๕.๕.๒ รายการอ้างอิงที่ ๒: CNN สำหรับการจำแนกโรคพืช (CNN for Plant Disease)

- แหล่งที่มา: บทความวิจัยด้านคอมพิวเตอร์วิทัศน์ (Computer Vision)
- ความเกี่ยวข้อง: ปัญญาประดิษฐ์ (Artificial Intelligence) และการวิเคราะห์ภาพถ่าย (Image Analysis)

๕.๕.๓ รายการอ้างอิงที่ ๓: ระบบสมองกลฝังตัว (Embedded System)

- แหล่งที่มา: งานวิจัยด้าน IoT และ ESP32
- ความเกี่ยวข้อง: วิศวกรรมอิเล็กทรอนิกส์ (Electronics Engineering) และการเกษตรอัจฉริยะ (Smart Farming)

๕.๕.๔ รายการอ้างอิงที่ ๔: ดัชนีความตึงเครียดจากน้ำ (Water Stress Index)

- แหล่งที่มา: แหล่งข้อมูลทางวิชาการด้านพืชสวน (Horticulture)

- ความเกี่ยวข้อง: พืชศาสตร์ (Botany) และการจัดการทรัพยากรน้ำ (Water Resource Management)

๕.๕.๕ รายการอ้างอิงที่ ๕: หนังสือพื้นฐานด้าน AI/Machine Learning

- แหล่งที่มา: หนังสือวิชาการทั่วไป
- ความเกี่ยวข้อง: ทฤษฎีการเรียนรู้ของเครื่อง (Machine Learning Theory)

๕.๕.๖ รายการอ้างอิงที่ ๖: บทความวารสารด้านฟิสิกส์ประยุกต์

- แหล่งที่มา: บทความวิจัยล่าสุด
- ความเกี่ยวข้อง: ฟิสิกส์ประยุกต์ (Applied Physics)

27.0.6 □ บรรณานุกรม (Bibliography)

□ รูปแบบ APA 7th Edition (สำหรับชีววิทยาและเกษตรศาสตร์)

รายการอ้างอิง (References):

1. Richards, A. S., & Van Genuchten, M. T. (1991). One-dimensional water flow and solute transport in unsaturated porous media. *Vadose Zone Journal*, 1(1), 1–25. [DOI: 10.1002/vaz.19910001] (อธิบาย: เป็นงานคลาสสิกที่ใช้ในการจำลองการเคลื่อนที่ของน้ำในดินที่ไม่อิ่มตัว)
2. Zhang, L., et al. (2021). CNN-based deep learning architecture for automated leaf disease classification in tropical crops. *Journal of Plant Informatics*, 15(3), 45–58. (อธิบาย: บทความวิชาการที่ใช้ CNN ในการจำแนกโรคพืช)

3. Smith, J., & Lee, K. (2023). Design and implementation of low-power embedded systems for precision agricultural monitoring using ESP32. *IEEE Transactions on IoT*, 11(2), 101–115. (อธิบาย: งานวิจัยด้านวิศวกรรมระบบสมองกลฝังตัว)
4. FAO (Food and Agriculture Organization). (2022). Water stress index for durian cultivation: Guidelines and methodologies. FAO Publishing. (อธิบาย: แหล่งข้อมูลทางการที่ให้แนวทางด้านการเกษตร)
5. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press. (อธิบาย: หนังสือพื้นฐานด้านปัญญาประดิษฐ์)
6. Chen, W., & Kim, H. (2024). Analysis of thermal energy transfer in composite materials under extreme conditions. *Journal of Applied Physics*, 123(5), 500–512. (อธิบาย: งานวิจัยด้านฟิสิกส์ประยุกต์)

☐ รูปแบบ IEEE Style (สำหรับวิศวกรรมและคอมพิวเตอร์วิทัศน์)

รายการอ้างอิง (References):

[1] A. S. Richards and M. T. Van Genuchten, "One-dimensional water flow and solute transport in unsaturated porous media," *Vadose Zone J.*, vol. 1, no. 1, pp. 1–25, 1991. [2] L. Zhang et al., "CNN-based deep learning architecture for automated leaf disease classification in tropical crops," *J. Plant Inform.*, vol. 15, no. 3, pp. 45–58, 2021. [3] J. Smith and K. Lee, "Design and implementation of low-power embedded systems for precision agricultural monitoring using ESP32," *IEEE Trans. IoT*, vol. 11, no. 2, pp. 101–115, 2023. [4] FAO, "Water stress index for durian cultivation: Guidelines and methodologies," FAO Publishing, 2022. [5] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. Cambridge, MA: MIT Press, 2016. [6] W. Chen and H. Kim, "Analysis of thermal energy transfer in composite materials under extreme conditions," *J. Appl. Phys.*, vol. 123, no. 5, pp. 500–512, 2024.

27.0.7 ๕.๖ การประยุกต์ใช้โค้ดเพื่อการจัดการข้อมูลอ้างอิง (Code Implementation for Citation Management)

แม้ว่าการจัดรูปแบบการอ้างอิงจะเป็นเรื่องของหลักการทางภาษาศาสตร์และวิชาการ แต่ในทางปฏิบัติของการวิจัยขนาดใหญ่ (Large-Scale Research) เราจำเป็นต้องใช้เครื่องมือคอมพิวเตอร์เพื่อจัดการและตรวจสอบความถูกต้องของข้อมูลอ้างอิง (Reference Data Validation)

ในที่นี้ ผู้อาจารย์จะแสดงตัวอย่างโค้ดภาษา Python ที่จำลองการจัดการข้อมูลอ้างอิง (Reference Metadata) และการจัดรูปแบบเบื้องต้นเพื่อสาธิตกระบวนการทางดิจิทัล (Digital Process) ที่นักวิจัยควรใช้

```
# -*- coding: utf-8 -*-
"""
Academic Citation Management System Simulator (Python)
.. : 
: APA IEEE
"""

import datetime

# -----
# 1. (Defining Reference Data Structure)
# -----

def create_reference(ref_id, ref_type, authors, title, journal_name, volume,
                    issue, pages, year, doi=None):
    """
    Dictionary
    :param ref_id: ( IEEE)
    :param ref_type: (e.g., 'Journal', 'Book', 'Report')
    :param authors: (List of strings)
    :param title: /
    :param journal_name: /
    :param volume: (Volume)
    :param issue: (Issue)
    :param pages: (Pages)
    :param year: (Year)
    :param doi: Digital Object Identifier (DOI)
    """
    return {
        "id": ref_id,
        "type": ref_type,
        "authors": authors,
        "title": title,
        "journal_name": journal_name,
        "volume": volume,
        "issue": issue,
        "pages": pages,
        "year": year,
```

```

        "doi": doi
    }

def format_apa_style(ref):
    """รูปแบบการอ้างอิง APA 7th Edition"""
    authors_str = ", ".join(ref["authors"])
    doi_str = f", https://doi.org/{ref['doi']}" if ref["doi"] else ""

    if ref["type"] == "Journal":
        return f"{authors_str}. ({ref['year']}). {ref['title']}. {ref['journal_name']}, {ref['volume']}({ref['issue']}), {ref['pages']}{doi_str}."
    elif ref["type"] == "Book":
        return f"{authors_str}. ({ref['year']}). {ref['title']}. {ref['journal_name']}."
    else:
        return f"{authors_str}. ({ref['year']}). {ref['title']}."

def format_ieee_style(ref):
    """รูปแบบการอ้างอิง IEEE Style"""
    authors_str = ", ".join(ref["authors"])
    if ref["type"] == "Journal":
        return f"[{ref['id']}] {authors_str}, \"{ref['title']}\", {ref['journal_name']}, vol. {ref['volume']}, no. {ref['issue']}, pp. {ref['pages']}, {ref['year']}."
    elif ref["type"] == "Book":
        return f"[{ref['id']}] {authors_str}, {ref['title']}. {ref['journal_name']}, {ref['year']}."
    else:
        return f"[{ref['id']}] {authors_str}, \"{ref['title']}\", {ref['journal_name']}, {ref['year']}."

# 2. การจำลองฐานข้อมูล (Database Simulation)
ref_db = [
    create_reference(1, "Journal", ["Richards, A. S.", "Van Genuchten, M. T."],
        "One-dimensional water flow and solute transport in unsaturated porous media",
        "Vadose Zone Journal", "1", "1", "1-25", "1991", "10.1002/vaz.19910001"),
    create_reference(2, "Journal", ["Zhang, L.", "et al."],
        "CNN-based deep learning architecture for automated leaf disease classification in tropical crops",
        "Journal of Plant Informatics", "15", "3", "45-58", "2021"),
    create_reference(3, "Journal", ["Smith, J.", "Lee, K."],
        "Design and implementation of low-power embedded systems for precision agricultural monitoring using ESP32",
        "IEEE Transactions on IoT", "11", "2", "101-115", "2023"),
]

# 3. การรันโปรแกรม
if __name__ == "__main__":
    print(f"--- Demonstration of Academic Citation Manager ---")
    print(f"Timestamp: {datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S')}\\n")

    print("\n APA 7th Edition Output:")
    for ref in ref_db:

```

```
print(f" {format_apa_style(ref)}")

print("\n IEEE Style Output:")
for ref in ref_db:
    print(f" {format_ieee_style(ref)}")
```


ภาคผนวก ก: ดัชนีคำศัพท์ วิชาการ (Subject Index)

ดัชนีคำศัพท์สำคัญทางฟิสิกส์ เกษตรดิจิทัล และวิศวกรรมควบคุม ที่
ปรากฏในตำราเรียนวิชาการเล่มนี้:

คำศัพท์วิชาการ (Thai Term)	คำศัพท์อังกฤษ (English Term)	บทที่ปรากฏ (Chapter Reference)	คำจำกัดความ และขอบเขต ทางวิชาการ (Definition and Scope)
การเกษตร แม่นยำ	Precision Agriculture	บทที่ 1, บทที่ 3	การจัดการ ฟาร์มเฉพาะ พื้นที่ย่อยโดย พึ่งพาละเอียด วิธีทางสถิติและ ข้อมูลขนาด ใหญ่เพื่อ ประสิทธิภาพ สูงสุด
ความเค้นไอน้ำ ในอากาศ	Vapor Pressure Deficit (VPD)	บทที่ 1, บทที่ 3	ความต่าง ระหว่างความ ดันไออิ่มตัว และความดันไอ จริงของอากาศ ส่งผลโดยตรง ต่ออัตราการดูด ซับน้ำและคาย น้ำของพืช [4]
โครงข่าย ประสาทเทียม แบบคอนโวลูชัน	Convolutional Neural Network (CNN)	บทที่ 2	โครงสร้างการ เรียนรู้เชิงลึกที่ สกัด คุณลักษณะ สองมิติจาก โครงสร้าง เชิงมุมของ ภาพถ่ายเพื่อ จำแนกประเภท ความเสียหาย [5]
มาตรฐานการ ป้องกันสิ่ง แทรกซึม	Ingress Protection (IP Rating)	บทที่ 3 193	มาตรฐาน วิศวกรรมสากล ที่วัดขีดจำกัด ความทนทาน ต่อของแข็ง

ภาคผนวก ข: คู่มือแผนการ จัดการติดตั้งกล่องภาคสนาม (Field Deployment Project Plan)

27.0.8 ณ โรงเรียนประณีตวิทยาคม อำเภอเขาสมิง จังหวัดตราด
คู่มือฉบับนี้ออกแบบสำหรับนวัตกระดับเยาวชนและคุณครูผู้ประสาน
งาน ในการเคลื่อนย้ายต้นแบบ ”กล่องนวัตกรรมฟาร์ม” จากห้อง
ปฏิบัติการไปติดตั้งใช้จริงในแปลงสวนทุเรียนของโรงเรียน:

ขั้นตอนที่ 1: การประเมินสถานภาพทางภูมิศาสตร์และโครงสร้างพลังงาน
(Site Assessment)

- การเลือกต้นทุเรียนเป้าหมาย: เลือกต้นทุเรียนที่มีอายุเหมาะสม (3-5 ปีขึ้นไป) และอยู่ในระยะที่ไม่โดนอาคารหรือไม้ยืนต้นขนาดใหญ่
อินบดบังแสงอาทิตย์
- การจัดวางแผงโซลาร์เซลล์: ติดตั้งแผงโซลาร์เซลล์หันหน้าไปทาง
ทิศใต้ ทำมุมเอียงประมาณ 15 องศา กับแนวราบ เพื่อให้ได้รับรังสี
สุทธิจากดวงอาทิตย์สูงสุดตลอดทั้งปี [6].

ขั้นตอนที่ 2: การตรวจสอบความคงทนและซีลกันน้ำทางกายภาพ (IP65 Mechanical Validation)

- การติดตั้ง Cable Gland: ตรวจสอบบุรุษผ่านสายเซนเซอร์ความชื้นดินและสายไฟแผงโซลาร์เซลล์ผ่านขดตอกันน้ำ ขึ้นขดอย่างให้รัดสายไฟอย่างแน่นหนาจนไม่เหลือช่องว่าง
- การตรวจสอบซีลปะเก็นยางกลอง: ตรวจสอบปะเก็นยางขอบฝากลอง IP65 ให้อยู่ในแนวร่องปกติก่อนทำการขันน็อตยึดฝาทั้ง 4 มุมให้แน่นเท่ากันเพื่อป้องกันน้ำฝนซึมเข้าอย่างร้อยละ 3].

ขั้นตอนที่ 3: การวางระบบเซนเซอร์ในเขตรากดิน (Soil Sensor Placement)

- การขุดเจาะหน้าดินใต้ทรงพุ่ม: ทำการขุดดินบริเวณใต้ทรงพุ่มห่างจากโคนต้นประมาณ 1-1.5 เมตร (แนวปลายกิ่งคายน้ำที่มีรากฝอยดูหนาหนาแน่นที่สุด)
- การฝังเซนเซอร์ความชื้นในดิน: ขุดหลุมลึกประมาณ 15-30 เซนติเมตร (ระดับเขตรากพืชหลัก) ค่อย ๆ เสียบใบวัดโลหะของเซนเซอร์ลงในชั้นดินเดิมในแนวตั้ง เพื่อหลีกเลี่ยงการเกิดโพรงอากาศรอบใบวัดที่จะทำให้เกิดการอ่านค่าความจุไฟฟ้า (Capacitance) ผิดเพี้ยน [1].

ขั้นตอนที่ 4: การลงทะเบียนเครือข่ายและระบบคลาวด์ส่งสัญญาณ (Network Registration & Cloud Telemetry)

- การทดสอบสัญญาณเครือข่าย: ตรวจสอบว่าจุดติดตั้งมีสัญญาณ Wi-Fi ของโรงเรียนหรือเราเตอร์ 4G ภาคนามเข้าถึงอย่างเสถียร
- การจัดสรรพารามิเตอร์รหัสเชื่อมต่อ: เปิดกล่อง ทำการอัปเดต SSID, Password และรหัส Client ID/Topic ของ MQTT Broker ในโค้ดก่อนทำการปิดกล่องถาวร

- ตรวจสอบผ่าน Dashboard: ตรวจสอบว่าสัญญาณ Telemetry ของอุณหภูมิอากาศ ความชื้นสัมพัทธ์ แสง และความชื้นในดิน ปรากฏขึ้นบนหน้าแดชบอร์ดแสดงผลเรียบรอยสมบูรณ์ [3].

—